

Chapter 16. Function Basics

In **Part III**, we studied basic procedural statements in Python. Here, we'll move on to explore a set of additional statements and expressions that we can use to create functions of our own.

In simple terms, a *function* is a package of code invoked by name. It labels and groups a set of statements so they can be run more than once in a program. A function also can compute a result value, and lets us specify parameters that serve as inputs and may differ each time the function's code is run. Wrapping an operation in a function makes it a generally useful tool, which we can apply in a variety of contexts.

On a more pragmatic level, functions are the alternative to programming by *cutting and pasting*—rather than having multiple redundant copies of an operation's code, we can factor it into a single function. In so doing, we reduce our future work radically: if the operation must be changed later, we have only one copy to update in the function, not many scattered throughout the program.

Functions are also the most basic program structure Python provides for maximizing code *reuse*, and lead us to the larger notions of program *design*. As you'll see, functions let us split complex systems into manageable parts. By implementing each part as a function, we make it both reusable and easier to code.

Table 16-1 abstractly previews the function-related tools we'll study in this part of the book—a set that includes call expressions, two ways to make functions (`def` and `lambda`), two ways to manage scope visibility (`global` and `nonlocal`), two ways to send results back to callers (`return` and `yield`), and tools to pause for results (`async` and `await`). While this comprises a feature-rich topic, you'll find that its commonly used core is straightforward.

Table 16-1. Function-related statements and expressions

Statement or expression	Examples
-------------------------	----------

Call expressions	<code>myfunc('hack', tool=python, *versions)</code>
<code>def</code>	<code>def printer(message): print('Hello', message)</code>
<code>return</code>	<code>def adder(a, b=1, *c): return a + b + c[0]</code>
<code>lambda</code>	<code>funcs = [lambda x: x**2, lambda x: x**3]</code>
<code>global</code>	<code>x = 'old' def changer(): global x; x = 'new'</code>
<code>nonlocal</code>	<code>def outer(): x = 'old' def changer(): nonlocal x; x = 'new'</code>
<code>yield</code>	<code>def squares(x): for i in range(x): yield i ** 2</code>
Generator expressions	<code>(i ** 2 for i in range(x))</code>
<code>async/await</code>	<code>async def consumer(a, b): await producer(b)</code>
Decorators and annotations	<code>@tracer def func(a: 'hack' = None) -> None</code>

Why Use Functions?

Before we get into the details, let's establish a clearer picture of what functions are all about. Functions are a nearly universal program-structuring device. You may have come across them before in other languages, where they may have been called *subroutines* or *procedures*. As a brief introduction, functions serve two primary development roles, and serve as the basis of other tools:

Maximizing reuse and minimizing redundancy

As in most programming languages, Python functions are the simplest way to package logic you may wish to use in more than one place and more than one time. Up until now, almost all the code we've been writing has run immediately. Functions allow us to defer and generalize code to be used arbitrarily many times later. Because they allow us to code an operation in a

single place and use it in many others, functions are also a *factoring* tool: they enable us to reduce code redundancy in our programs, and thereby reduce maintenance effort.

Dividing and conquering

Functions also provide a tool for splitting systems into pieces that have well-defined roles and *manageable* scopes. For instance, to make a pizza from scratch, you would start by mixing the dough, rolling it out, adding toppings, baking it, and so on. If you were programming a pizza-making robot, functions would help you divide the overall “make pizza” task into smaller parts—one function for each subtask in the process. Because it’s easier to implement the smaller tasks in isolation than it is to implement the entire process at once, this makes larger tasks more practical.

Implementing object methods

In general, functions are about *procedure*—how to do something, rather than what you’re doing it to. Nevertheless, when paired with an implied subject, they can also be used to code object-specific behavior known as methods. You’ll see why this distinction matters in **Part VI**, when we start making new objects with classes. As you’ll find, classes are largely just packages of the functions you’ll learn to code here.

In this part of the book, we’ll explore the tools used to code functions in Python: function basics, scope rules, and argument passing, along with a few related concepts such as generators, coroutines, and functional tools. Because its importance begins to become more apparent at this level of coding, we’ll also revisit the notion of polymorphism, which was introduced earlier in the book. As you’ll see, functions don’t imply much new syntax, but they do lead us to some bigger programming ideas.

Function Coding Overview

Although it wasn't made very formal, we've already *used* functions in earlier chapters. For instance, we called the built-in `open` function to make a file object, invoked the `len` built-in function to ask for the number of items in a collection object, and employed tools like `zip` and `range` for iteration tasks.

In this chapter, we will begin exploring how to write *new* functions in Python. Functions we write behave the same way as the built-ins we've already seen: they are called in expressions, are passed values, and return results. But writing new functions requires the application of a few additional tools and ideas that haven't yet been introduced. Moreover, functions behave very differently in Python than they do in compiled languages like C.

Basic Function Tools

As a road map and brief rundown on what we'll study in this part of the book, here are the basic components and concepts in Python's function toolbox:

- **def creates a function and assigns it to a name.** Python functions are coded with `def` statements. When Python reaches and runs a `def`, it generates a new function object and assigns it to the function's name. As with all assignments, the function name becomes a reference to the function object. There's nothing magical about the name of a function—as you'll see, the function object can be assigned to other names, stored in a list, and so on. Function objects may also have arbitrary user-defined *attributes* attached to them to record data.
- **def is executable code.** Unlike functions in compiled languages, `def` is an executable statement—your function does not exist until Python runs its `def`. In fact, it's legal (and even occasionally useful) to nest `def` statements inside `if` statements, `for` loops, and even other `defs`. In typical usage, `def` statements are coded at the top level of module files, and are automatically run to make functions when their modules are imported.
- **return sends a result object back to the caller.** When a function is

called, the caller normally stops until the function finishes its work and returns control to the caller. Functions that compute a value send it back to the caller with a `return` statement, and the returned value becomes the result of the function call. A `return` without a value simply returns to the caller (and sends back `None`, the default result).

- **lambda creates a function but returns it as a result.** Function objects may also be created with the `lambda` expression, a feature that allows us to code *inline* function definitions in places where a `def` statement won't work syntactically. Like `def`, functions made this way are created when the `lambda` is reached and run. Because `lambda` is not typically coded at the top level of a module, though, such functions are made during program runs, not imports. `lambda` is an optional convenience best seen as a sidebar to the `def` statement.

Advanced Function Tools

In addition, Python functions can leverage more advanced tools like the following, which we'll take up after we've introduced the basics:

- **global declares module-level variables that are to be assigned.** By default, all names assigned in a function are local to that function and exist only while the function runs. To assign a name in the enclosing module, functions need to list it in a `global` statement. More generally, names are always looked up in *scopes*—places where variables are stored—and names are bound to scopes per the location of assignments in your code.
- **nonlocal declares enclosing-function variables that are to be assigned.** In the same category as `global`, the `nonlocal` statement allows a function to assign a name that exists in the scope of a syntactically enclosing `def` statement. This allows enclosing functions to serve as a place to retain *state*—information remembered between function calls—without using shared global names.
- **yield sends a result object back to the caller, but remembers where**

it left off. Functions known as *generators* may also use the `yield` statement to send back a value and suspend their state such that they may be resumed later, to produce a series of results over time. Along with their *generator expression* kin, such functions save space and avoid delays just like the built-in iterables we met in the prior part of this book.

- **`await/async` pause a waiting function so that other tasks may run.** Functions known as *coroutines* can suspend their execution until a required result is available. This is an advanced applications-level topic, but allows code to use language tools to get other work done while waiting on a blocking event like slow IO.

General Function Concepts

Finally, along the way we'll explore a handful of core concepts that span functions of all kinds:

- **Arguments are passed by assignment (object reference).** In Python, arguments are passed to functions by assignment—which, as we've learned, means by object reference. As you'll see, in Python's model the caller and function share *objects* by references, but there is no name aliasing. Changing an argument name within a function does not also change the corresponding name in the caller, but changing passed-in mutable objects in place can change objects shared by the caller, and serve as a function result.
- **Arguments are passed by position, unless you say otherwise.** Values you pass in a function call match argument names in a function's definition from left to right by default. For flexibility, function *calls* can also pass arguments by name with *name=value* keyword syntax and unpack arbitrarily many arguments to send with **args* and ***args* starred-value notation. Function *definitions* use the same syntax forms to specify argument defaults and collect arbitrarily many arguments received.
- **Arguments, return values, and variables are not declared.** As with

everything in Python, there are no type constraints on functions. In fact, nothing about a function needs to be declared ahead of time: you can pass in arguments of any type, return any kind of object, and so on. As a consequence, a single function can often be applied to a variety of object types—any objects that have a compatible *interface* (support for expected methods and expressions) will do, regardless of their specific types. This makes code flexible by design.

- **Attributes, annotations, and decorators support advanced roles.**

Functions can optionally be augmented with both user-defined attributes and other tools that extend their roles. Annotations of arguments and results, for example, can serve a variety of goals. Among these, *type hinting* (noted in [Chapter 6](#)) gives suggested object types but is unused by Python itself and simply serves as a weighty form of documentation, which we won't cover further in this book. More usefully, functions can also be prefixed with @ decorators that add extra layers of logic but are used in ways that merit largely deferring until [Part VI](#) when we've also learned about classes.

If some of the preceding words didn't sink in, don't worry—we'll explore most of these concepts with real code in this part of the book. Let's get started by expanding on some of the basics by coding a few abstract examples.

def Statements

The `def` statement creates a function object and assigns it to a name. Its general format and usage is as follows:

```
def name(arg1, arg2,... argN):      # Define a function
    statements                      # Function body

name(val1, val2,... valN)          # Call it later in an expression
```

As with all compound statements, `def` consists of a *header* line followed by a block of other statements, usually indented (though a simple statement after the colon works as usual). The statement block becomes the function's *body*—that is, the code Python executes each time the function is later called.

The `def` header line specifies a function *name* that is assigned the new function object, along with a list of zero or more *arguments* (sometimes called *parameters*) enclosed in parentheses (even for zero arguments). The argument names in the header are *assigned* to the objects passed in parentheses at the point of call elsewhere in your code.

To *call* the function later in your program and run its body, you can use the function *name* assigned by `def`, passing in zero or more *values* (objects) to match the arguments listed in the `def` header. A call expression can also denote the function with other types of object references; it need not necessarily be the name coded in the `def` header.

return Statements

Function bodies often contain one or more `return` statements that make sense only inside a `def`:

```
def name(arg1, arg2,... argN):  
    ...  
    return result                # The result of the call expression
```

The Python `return` statement can show up anywhere in a function body. When reached, it *ends* the function call and sends a *result* back to the caller to serve as the result of the call expression. The `return` statement consists of an optional object-value expression that gives the result. If the value is omitted, `return` sends back a `None` by default.

The `return` statement itself is optional too; if it's not present, the function exits when the control flow falls off the end of the function body. Technically, a function *without* a `return` statement also returns the `None` object automatically, but this return value is usually ignored at the call, by using a call-expression *statement* of [Chapter 11](#).

Functions may also contain `yield` statements used to produce a series of values over time, as well as `await` and `async for/with` statements used to suspend the function's execution, but we'll defer discussion of these more advanced tools until we survey generator and coroutine topics in [Chapter 20](#).

def Executes at Runtime

The Python `def` is a true executable statement: when it runs, it creates a new function object and assigns it to a name. (Remember, all we have in standard Python is *runtime*; there is no such thing as a separate compile time.) Because it's a statement, a `def` can appear anywhere a statement can—even *nested* in other statements. For instance, although `defs` are normally top-level code run when the module enclosing them is imported, it's also completely legal to nest a function `def` inside an `if` statement to select between alternative definitions:

```
if test:
    def func():          # Define func this way
        ...
else:
    def func():          # Or else this way
        ...
...
func()                  # Call the version selected and built
```

One way to understand this code is to realize that the `def` is much like an `=` statement: it simply assigns a name (like `func` here) at runtime. Unlike in compiled languages such as C, Python functions do not need to be fully defined before the program runs. More generally, `defs` are not run until they are reached, and the code *inside* `defs` is not run until the functions are later called.

Because function definition happens at runtime, there's nothing special about the function name. What's important is the object to which it refers. For example:

```
othername = func        # Assign function object
othername()              # Call func again
```

Here, the function was assigned to a different name and called through the new name. Like everything else in Python, functions are just *objects*; they are recorded explicitly in memory at program execution time. In fact, besides calls, functions allow arbitrary *attributes* to be attached to record information for later use:

```
def func(): ...          # Create + assign function object
func()                   # Call object via its name
func.attr = value        # Attach attributes
```

In other words, functions are *first-class objects*, to borrow a term introduced in [Chapter 9](#). While they don't support some operations that other objects do, functions inhabit the same category as every object. As you'll see, passing them about and storing them in other objects is both syntactically legal and surprisingly useful.

lambda Makes Anonymous Functions

In addition to `def`, you can make a new function with the `lambda` expression. It's coded and might be used like this:

```
name = lambda arg1, arg2,... argN: expression

name(val1, val2,... valN)
```

Like `def`, `lambda` makes a new function object to be called later. It begins with the word `lambda`, followed by an arguments-list *header* that works the same as in `def` but is coded without parentheses. Unlike `def`, the code after the `:` in `lambda` is a single *expression*—it cannot contain statements and is the *implied* body and return value of the function that `lambda` makes. We don't need to say `return` in a `lambda` (and we can't).

Also unlike `def`, `lambda` does not itself assign the new function to a name, but simply *returns* it as the result of the whole `lambda` expression. This snippet manually assigns the function to a name through which it is called, but that's optional: the result might also be saved in another object or passed for use elsewhere. Because of this, `lambda` is usually called an “anonymous” function—one that's unnamed.

You'll learn more about `lambda` later and see it in action along the way. But your first question may be, Why have a version of `def` whose code is limited to just one expression? In short, `lambda` can be used in places that `def` cannot, including in calls and object literals where we want to embed small bits of deferred and runnable code. While `def` handles larger tasks and also supports embedding functions by name, `lambda` is an optional amenity in simpler roles.

And if you're keeping track, we've now seen *four* expression equivalents for

more general statements—the `lambda` for `def`, the `:=` named assignment for `=`, the `if/else` ternary for `if`, and the comprehension for `for`. Although lambdas are limited to expressions, they can use any of these expression equivalents to code assignments, logic, and loops. All these expression forms were accumulated over time, but are convenient alternatives in practical code.

A First Example: Definitions and Calls

Apart from their runtime flavor (which tends to seem most novel to programmers with backgrounds in compiled languages), Python functions are straightforward to use. Let's code a first real example to demonstrate the fundamentals, from both sides of the function equation: *definition* (the `def` or `lambda` that creates a function) and *calls* (the expressions that tell Python to run the function's body).

Definition

Here's a definition typed interactively that defines a function named `times`, which returns the product of its two arguments (it's not much, but it demos the basic bits). As usual "... prompts are omitted here for copy and paste:

```
>>> def times(x, y):      # Create and assign function
    return x * y          # Body executed when called
```

When Python reaches and runs this `def`, it creates a new function object that packages the function's code, and assigns this object to the name `times`. Typically, such a statement is coded in a module file and runs when the enclosing file is imported; for something this small, though, the interactive REPL suffices.

As an aside, a `lambda` can have the same effect if we assign its result to a name, though it's typically used in more focused roles than this, and often not as top-level code. Try the `def`, `lambda`, or both if you're working along—they both make function objects that work the same:

```
>>> times = lambda x, y: x * y
```

Calls

Both `def` and `lambda` make a function but do not call it. After either has run, you can *call* (i.e., run) the function in your program by adding parentheses after the function's name. The parentheses may optionally contain one or more object arguments, to be passed (i.e., assigned) to the names in the function's header:

```
>>> times(2, 4)           # Arguments in parentheses
8
```

This expression passes two arguments to `times`. As mentioned previously, arguments are passed by assignment, so in this case the name `x` in the function header is assigned the object 2, `y` is assigned 4, and the function's body is run.

For this function, the body is just a `def`'s return statement or a `lambda`'s expression, that sends back the result as the value of the call expression. The returned object was printed here interactively (as in most languages, $2 * 4$ is 8 in Python), but if we needed to use it later we could instead assign it to a variable. For example:

```
>>> x = times(3.14, 4)    # Save the result object
>>> x
12.56
```

Now, watch what happens when the function is called a third time, with very different kinds of objects passed in:

```
>>> times('Py', 4)       # Functions are "typeless"
'PyPyPyPy'
```

This time, our function means something wholly different. In this third call, a string and an integer are passed to `x` and `y`, instead of two numbers. Recall that `*` works on both numbers and sequences; because we don't constrain the types of variables, arguments, or return values in Python, we can use `times` to either *multiply* numbers or *repeat* sequences.

In other words, what our `times` function means and does depends on what we pass into it. This is a core idea in Python (and perhaps the key to using the

language well), which merits a separate callout here.

Polymorphism in Python

As we just saw, the very meaning of the expression `x * y` in our simple `times` function depends completely upon the kinds of objects that `x` and `y` are—thus, the same function can perform multiplication in one instance and repetition in another. Python leaves it up to the *objects* to do something reasonable for the syntax. Really, `*` is just a dispatch mechanism that routes control to the objects being processed.

This sort of type-dependent behavior is known as *polymorphism*, a term we first met in [Chapter 4](#) that essentially means that the meaning of an operation depends on the objects being operated upon. Because it's a dynamically typed language, polymorphism runs rampant in Python. In fact, *every* operation is a polymorphic operation in Python: printing, indexing, the `*` operator, and much more.

This is deliberate, and it accounts for much of the language's conciseness and flexibility. A single function, for instance, can generally be applied to a whole category of object types automatically. As long as those objects support the expected *interface* (a.k.a. protocol), the function can process them. That is, if the objects passed into a function have the expected methods and expression operators, they are plug-and-play compatible with the function's logic.

Even in our simple `times` function, this means that *any* two objects that support a `*` will work, no matter what they may be, and no matter when they are coded. This function will work on two numbers (performing multiplication), or a string and a number (performing repetition), or any other combination of objects supporting the expected interface—even class-based objects we have not even imagined yet.

Moreover, if the objects passed in do *not* support this expected interface, Python will detect the error when the `*` expression is run and raise an exception automatically. It's therefore usually pointless to code error checking in such code ourselves. In fact, doing so would limit our function's utility, as it would be restricted to work only on objects whose types we test for:

```
>>> times('not', 'quite')
```

```
TypeError: can't multiply sequence by non-int of type 'str'
```

This turns out to be a crucial philosophical difference between Python and statically typed languages like C++ and Java: in Python, your code is *not supposed to care* about specific data types. If it does, it will be limited to working on just the types you anticipated when you wrote it, and it will not support other compatible object types coded now or in the future. Although it is possible to test for types with tools like the `type` built-in function, doing so breaks your code's flexibility. By and large, we code to object *interfaces* in Python, not data types.

Of course, some programs have unique requirements, and this polymorphic model of programming means we have to *test* our code to detect some errors that statically typed compiled languages might be able to detect earlier. In exchange for an initial bit of extra testing, though, we radically reduce the amount of code we have to write, and radically increase our code's flexibility. As you'll come to find with time, it's a resounding net win in practice.

A Second Example: Intersecting Sequences

Next, let's explore a second function example that does something a bit more useful than multiplying arguments and further illustrates function basics.

In [Chapter 13](#), we coded a `for` loop that collected items held in common in two strings (and called it nearly intersection, except for duplicates). We noted there that the code wasn't as useful as it could be because it was set up to work only on specific variables and could not be rerun later. Of course, we could copy the code and paste it into each place where it needs to be run, but this solution is neither good nor general—we'd still have to edit each copy to support different sequence names, and changing the algorithm would then require changing multiple copies.

Definition

By now, you can probably guess that the solution to this dilemma is to package the `for` loop inside a function. Doing so offers a number of advantages:

- Putting the code in a function makes it a tool that you can run as many times as you like.
- Because callers can pass in arbitrary arguments, functions are general enough to work on any two sequences (or other iterables) you wish to intersect.
- When the logic is packaged in a function, you have to change code in only *one* place if you ever need to change the way the intersection works.
- Coding the function in a module file means it can be imported and reused by any program run on your machine.

In effect, wrapping the code in a function makes it a general utility, as captured by [Example 16-1](#).

Example 16-1. inter1.py

```
def intersect(seq1, seq2):
    res = []                # Start empty
    for x in seq1:          # Scan seq1
        if x in seq2:       # Common item?
            res.append(x)   # Add to end
    return res
```

The transformation from the simple code of [Chapter 13](#) to this function is straightforward; we’ve just nested the original logic under a `def` header and made the objects on which it operates passed-in parameter names. Because this function computes a result, we’ve also added a `return` statement to send a result object back to the caller.

Calls

Before you can call a function, you have to make it. To do this, run its `def` statement, either by typing it interactively or by coding it in a module file and importing the file. You can paste it at a REPL as a whole, but we’ll import it from a file here (per [Chapter 3](#), make sure you can see it in your REPL by working in the file’s folder if needed). Once you’ve run the `def` by such means, you can call the function by passing any two sequence objects in parentheses:

```
>>> from inter1 import intersect    # Get function from module
>>> s1 = 'HACK'
>>> s2 = 'CHOK'
>>> intersect(s1, s2)               # Pass two strings
['H', 'C', 'K']
```

Here, we’ve passed in two strings, and we get back a list containing the characters in common. The algorithm the function uses is simple: “For every item in the first argument, if that item is also in the second argument, append the item to the result.” It’s a little shorter to say that in Python than in English, but it works out the same.

To be fair, our `intersect` function could probably be quicker (it executes naive nested loops), isn’t really mathematical intersection (there may be duplicates in the result), and isn’t required at all (as we’ve seen, Python’s set objects provide a built-in intersection operation kicked off with `&`). Indeed, the function could be replaced with a single list comprehension expression, as it exhibits the classic collector-loop code pattern:

```
>>> [x for x in s1 if x in s2]
['H', 'C', 'K']
```

Which also works as a loop inside a `lambda` body expression—despite being five lines in the `def`. This suffices as a demo, but doesn’t make much sense in a simple role like this (again, we’ll put lambdas to better use later):

```
>>> intersect = lambda seq1, seq2: [x for x in seq1 if x in seq2]
>>> intersect(s1, s2)
['H', 'C', 'K']
```

However it’s coded, though, this function does the job: its single piece of code can apply to an entire range of object types, as the next section explains. In fact, we’ll improve and extend this to support arbitrarily many operands in [Chapter 18](#), after we uncover more about argument passing modes.

Polymorphism Revisited

Like all good functions in Python, `intersect` is polymorphic. That is, it works on arbitrary types, as long as they support the expected object interface:


```
>>> x = intersect([1, 2, 3], (1, 4))    # Pass mixed types
>>> x                                   # Saved result object
[1]
```

This time, we passed in different types of objects to our function—a list and a tuple (mixed types)—and it still picked out the common items. Because you don’t have to specify the types of arguments ahead of time, the `intersect` function happily iterates through any kind of objects you send it, as long as they support the expected interfaces.

For `intersect`, this means that the first argument has to support the `for` loop, and the second has to support the `in` membership test, in both `def` and `lambda` flavors. Any two such objects will work, regardless of their specific types—that includes physically stored sequences like strings and lists; all the iterable objects we met in [Chapter 14](#), including files and dictionaries; and even any class-based objects we code that apply operator overloading techniques we’ll discuss later in the book.¹

And here again, if we pass in objects that do *not* support these interfaces (e.g., numbers), Python will automatically detect the mismatch and raise an exception for us—which is exactly what we want, and the best we could do on our own if we coded explicit type tests:

```
>>> intersect([1, 2, 3], 1)
TypeError: argument of type 'int' is not iterable
```

By not coding type tests and allowing Python to detect the mismatches for us, we both reduce the amount of code we need to write, and avoid artificially limiting our code’s scope.

Segue: Local Variables

Perhaps the most interesting part of this example, though, is its names. It turns out that the variable `res` inside the `def` version of `intersect` is what in Python is called a *local variable*—a name that is visible only to code inside the function `def` and that exists only while the function call runs. In fact, because all names *assigned* in any way inside a function are classified as local variables by default, nearly all the names in the `def` qualify:

- `res` is obviously assigned, so it is a local variable.
- Arguments are passed by assignment, so `seq1` and `seq2` are, too.
- The `for` loop assigns items to a variable, so the name `x` is also local.

All these local variables appear when the function is called and disappear when the function exits—the `return` statement at the end of `intersect` sends back the result *object*, but the *name* `res` goes away. The same goes for argument names in the `lambda`, though its compression hides its loop variable (sans a nested `:=` to export it).

Because of this, a function's variables don't clash with names elsewhere, but also won't remember values between calls. Although the object returned by a function lives on, retaining other *state information* requires other techniques. To fully explore the notion of locals and state, though, we need to move on to the scopes coverage of the next chapter.

Chapter Summary

This chapter introduced the core ideas behind function definition—the syntax and operation of the `def` and `return` statements and `lambda` expression, the behavior of function call expressions, and the notion and benefits of polymorphism in Python functions. As we saw, a `def` and `lambda` are executable code that create a function object at runtime; when the function is later called, objects are passed into it by assignment (which means object reference in Python), and computed values are sent back by `return` in `def` and implicitly in `lambda`. We also began exploring the concepts of local variables and scopes, but saved the details for [Chapter 17](#). First, though, a quick quiz.

Test Your Knowledge: Quiz

1. What is the point of coding functions?
2. At what time does Python create a function?
3. What does a `def` function return if it has no `return` statement in it?
4. When does the code nested inside the function definition run?
5. What's wrong with checking the types of objects passed into a function?

Test Your Knowledge: Answers

1. Functions are the most basic way of avoiding code *redundancy* in Python—factoring code into functions means that we have only one copy of an operation's code to update in the future. Functions are also the basic unit of code *reuse* in Python—wrapping code in functions makes it a reusable tool, callable in a variety of programs. Finally, functions allow us to *divide* a complex system into manageable parts, each of which may be developed individually. Functions are also used for object *methods*, but we're postponing this role until we study classes.

2. A function is created when Python reaches and runs a `def` statement, which creates a function object and assigns it to the function's name. This normally happens when the enclosing module file is imported by another module (recall that imports run the code in a file from top to bottom, including any `defs`), but it can also occur when a `def` is typed interactively or nested in other statements, such as `ifs`. Functions are also created when a `lambda` is reached and run, though this doesn't normally happen during an import operation.
3. A `def` function returns the `None` object by default if the control flow falls off the end of the function body without running into a `return` statement. Such functions are usually called with expression statements, as assigning their `None` results to variables is generally pointless. A `return` statement with no expression in it also returns `None`. A `lambda` implicitly returns its expression's result, so it has no default.
4. The function *body* (the code nested inside the function `def` statement or following the colon in `lambda`) is run when the function is later invoked with a call expression. The body runs anew each time the function is called.
5. Checking the types of objects passed into a function effectively breaks the function's *flexibility*, constraining the function to work on specific types only. Without such checks, the function would likely be able to process an entire array of object types—any objects that support the interface expected by the function will work. (The term *interface* means the set of methods and expression operators the function's code runs.)

¹ This code will always work if we intersect files' contents obtained with `file.readlines()`. It may not work to intersect lines in open input files directly, though, depending on the file object's implementation of the `in` operator or general iteration. Files must generally be rewound (e.g., with a `file.seek(0)` or another `open`) after they have been read to end-of-file once, and so are single-pass iterators. As you'll see in [Chapter 30](#) when we study operator overloading, objects implement the `in` operator either by providing the specific `__contains__` method or by supporting the general iteration protocol with the `__iter__` or older `__getitem__` methods; classes can code these methods arbitrarily to define what iteration means for their data.

Chapter 17. Scopes

The preceding chapter introduced basic function definitions and calls. As we saw, Python’s core function model is simple to use, but even simple function examples quickly lead to questions about the meaning of variables in code. This chapter moves on to present the details behind Python’s *scopes*—the places where variables are defined and looked up. Like module files, scopes help prevent same-name clashes across a program’s code: names defined in one program unit don’t interfere with names in another.

As you’ll learn here, the place where a name is assigned in your code is crucial to determining what the name means. You’ll also find that scope usage can have a major impact on program maintenance effort; overuse of *globals*, for example, is a generally bad idea. On the plus side, you’ll also learn that scopes can provide a way to retain *state information* between function calls, and they offer an alternative to classes in some roles.

Python Scopes Basics

Now that we’re starting to write our own functions, we need to get more formal about what names mean in Python. When we use a name in a program, Python creates, changes, or looks up the name in what is known as a *namespace*—a place where names live. And when we talk about names in relation to code, namespaces correspond to *scopes*: the location of a name’s assignment in your source code determines the scope of the name’s visibility in your program.

Just about everything related to names, including scope classification, happens at assignment time in Python. As we’ve seen, names in Python spring into existence when they are first assigned values, and they must be assigned before they are used. Because names are not declared ahead of time, Python uses the location of the assignment of a name to associate it with (i.e., *bind* it to) a particular namespace and scope. In other words, the place where you assign a name in your source code determines the namespace it will live in, and hence its scope of visibility.

Besides packaging code for reuse, functions add an extra namespace layer to your programs to minimize the potential for collisions among variables of the same name—by default, all names *assigned* inside a function are associated with that function’s namespace, and no other. This rule means that:

- Names assigned inside a `def` can be seen only by the code *within* that `def`. You cannot even refer to such names from outside the function.
- Names assigned inside a `def` do not clash with variables outside the `def`, even if the same names are used elsewhere. A name `X` assigned *outside* a given `def` (i.e., in a different `def` or at the top level of a module file) is a completely different variable from a name `X` assigned *inside* that `def`.
- Names assigned in a `lambda` work the same way, though assignment in their expressions can happen only for arguments and `:=` named assignments, and is much less common than in `def`.

In all cases, the scope of a variable (where it can be used) is wholly determined by where it is assigned in your source code, and has nothing to do with which functions call which. In fact, as you’ll learn in this chapter, variables may be assigned in three different places, corresponding to three different scopes:

- If a variable is assigned inside a `def` or `lambda`, it is *local* to that function.
- If a variable is assigned in an enclosing `def` or `lambda`, it is *nonlocal* to nested functions.
- If a variable is assigned outside all `defs` and `lambdas`, it is *global* to the entire file.

We call this *lexical scoping* because variable scopes are determined entirely by the locations of the variables in the source code of your program files, not by function calls. Hence, a simple visual inspection is enough to make the call.

For example, in the following module file, the `X = 99` assignment creates a *global* variable named `X` (visible everywhere in this file), but the `X = 88`

assignment creates a *local* variable *X* (visible only within the `def` statement). If we refer to *X* inside the function, we'll get the function's *X*, not the global:

```
X = 99                                # Global (module) scope X

def func():
    X = 88                            # Local (function) scope X: a different variable
```

Even though both variables are named *X*, their scopes make them different. The net effect is that function scopes help to avoid name clashes in your programs and help to make functions more self-contained program units—their code need not be concerned with names used elsewhere.

Scopes Overview

Before we started writing functions, none of the code we wrote was nested in a `def`, so the names we used either lived in a module or were built-ins predefined by Python like `open` and `zip`. This includes code typed at the REPL: the interactive prompt is technically a module named `__main__` that prints results and doesn't correspond to a real file; in all other ways, though, it's like the top level of a module file.

Functions, though, provide nested namespaces (scopes) that localize the names they use, such that names inside a function won't clash with those outside it (in a module or another function). Specifically, functions define a *local scope* and modules define a *global scope* with the following properties:

- **The enclosing module is a global scope.** Each module is a global scope—that is, a namespace in which variables created by assignment at the top level of the module file live. Global variables become attributes of a module object to the outside world after imports but can also be used as simple variables within the module file itself.
- **The global scope spans a single file only.** Don't be fooled by the word “global” here—names at the top level of a file are global to code within that single file only. There is no notion of an all-encompassing global scope that corresponds to user code in Python (built-ins are truly global, but not user defined). Instead, names are partitioned into modules, and

you must always import a module explicitly if you want to be able to use the names its file defines. When you hear “global” in Python, think “module.”

- **Assigned names are local unless declared global or nonlocal.** By default, all the names assigned inside a function definition are put in its local scope—the namespace associated with the function itself. To assign a name that lives at the top level of the module enclosing a function, declare it in a `global` statement inside the function. To assign a name that lives in an enclosing `def` instead, declare it in a `nonlocal` statement. Because `lambda` bodies are limited to expressions, these two statements are just for `def`.
- **All other names are enclosing function locals, globals, or built-ins.** Names *not* assigned a value in the function definition are assumed to be either locals defined in a physically *enclosing* function, *globals* that live in the enclosing module’s namespace, or *built-ins* in the predefined built-ins module that Python provides. Enclosing scopes can arise for any `def` and `lambda` combo, though `def` can’t be coded in `lambda`.
- **Each call to a function creates a new local scope.** Every time you call a function, you create a *new* local scope—that is, a namespace in which the names created inside that function will live, barring `global` or `nonlocal` statements. You can think of each `def` statement and `lambda` expression as defining a new local scope, but the local scope actually corresponds to a function *call*, for reasons the next bullet explains.
- **Per-call scopes matter in recursion and closures.** It’s crucial that each active call receive its own copy of the function’s local variables when using *recursion*, an advanced technique that allows functions to loop by calling themselves, and noted briefly in [Chapter 9](#) for comparisons. Recursion is useful in functions we write as well, to process structures whose shapes can’t be predicted ahead of time; we’ll explore it more fully in [Chapter 19](#). Per-call scopes are also required for the *closure* functions covered ahead here: each call gets a fresh packet of the function’s locals, which can remember state-information objects to be used later.

Beyond those basics, there are three subtleties worth underscoring. First, as noted, code typed at the Python *interactive prompt* lives in a module, too, and follows the normal scope rules: names assigned there are global variables, accessible to the entire interactive session. You'll learn more about modules in the next part of this book.

Second, also bear in mind that *any type of assignment* classifies a name as local or global, based on where the assignment appears. This includes `=` statements and `:=` expressions; module names in `import`; function and argument names in `def`; names in `class` covered later; and so on. If you assign a name in any way within a function, it will be local to that function by default. This includes its arguments listed in its header, but also names in its body's code.

Third, *in-place changes* to objects do not classify names as locals; only actual *name* assignments do. For instance, if the name `L` is assigned to a list at the top level of a module, a statement `L = X` within a function will classify `L` as a local, but `L.append(X)` will not. In the latter case, we are changing the list object that `L` references, not `L` itself—hence, `L` is found in the global scope as usual, and Python happily modifies it without requiring a `global` declaration. As usual, it helps to keep the distinction between names and objects clear: changing an object is not an assignment to a name.

Name Resolution: The LEGB Rule

If the prior section sounds complicated, it really boils down to three simple rules. Within a `def` statement:

- Name *assignments* create or change local names by default.
- Name *references* search at most four scopes: local, then enclosing functions (if any), then global, then built-in.
- Names declared in `global` and `nonlocal` statements map assigned names to enclosing module and function scopes, respectively.

The same goes for `lambda`, except for the last bullet: it doesn't support statements.

In other words, all names assigned inside a function `def` statement or `lambda` expression are locals by default. Functions can freely use names assigned in syntactically enclosing functions and the global scope, but they must declare such nonlocals and globals in order to change them.

Python's name-resolution scheme is usually called the *LEGB rule*, after the names of the scopes it searches:

- When you use an *unqualified* name (not after a `“.”`) inside a function, Python searches up to four scopes—the local (*L*) scope of the function itself, then the local scopes of any enclosing (*E*) `defs` and `lambdas`, then the global (*G*) scope of the surrounding module, and finally the built-in (*B*) scope—and stops at the first place the name is found. If the name is not found during this search, Python reports an error.
- When you assign a name *inside* a function (instead of just referring to it in an expression), Python always creates or changes the name in the assigner's local scope, unless it's declared to be global or nonlocal there.
- When you assign a name *outside* any function (i.e., at the top level of a module file, or at the interactive prompt), the local and global scope are one and the same—the module's namespace.

Because names must be assigned before they can be used (as we saw in [Chapter 6](#)), there are no automatic components in this model: assignments always determine name scopes unambiguously. [Figure 17-1](#) illustrates Python's four scopes and LEGB rule. Note that the second scope lookup layer, *E*—the scopes of enclosing `defs` or `lambdas`—can technically correspond to more than one lookup level. This layer only comes into play when you nest functions within functions for reasons you'll meet ahead, and is enhanced by the `nonlocal` statement.¹

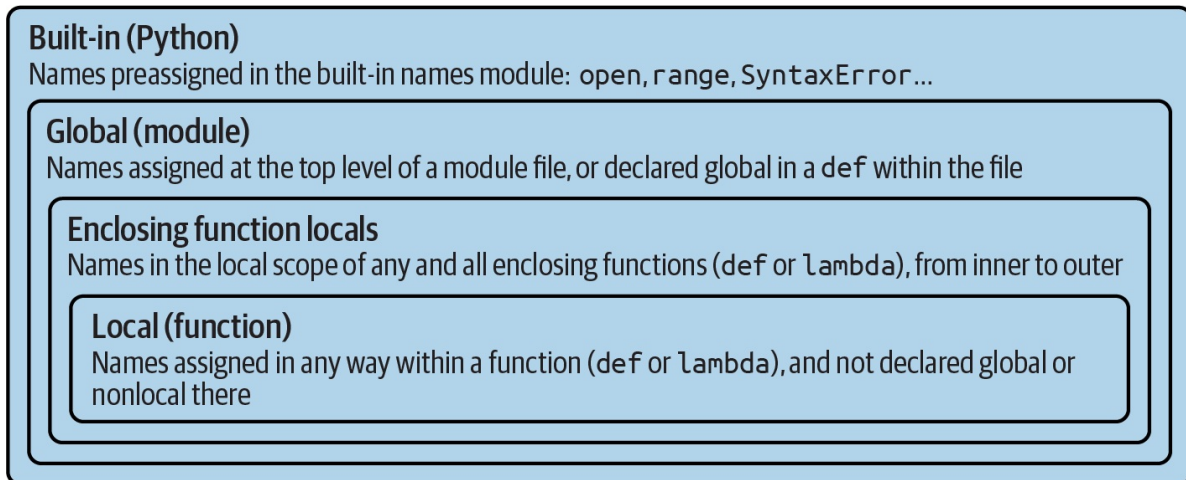


Figure 17-1. The LEGB scope lookup rule for names

Also keep in mind that these rules apply only to simple *variable* names (e.g., `name`). In Parts [V](#) and [VI](#), you'll see that qualified *attribute* names (e.g., `object.name`) live in particular objects and follow a completely different set of lookup rules than those covered here. References to attribute names following periods (`.`) search one or more *objects*, not *scopes*, and in fact may invoke something called *inheritance* in Python's OOP model; more on this in [Part VI](#).

Preview: Other Python scopes

Though obscure at this point in the book, there are technically three more scopes in Python—temporary loop variables in comprehensions, exception reference variables in `try` handlers, and local scopes in `class` statements. The first two of these are special cases that rarely impact real code, and the third falls under the LEGB umbrella rule.

Importantly, most statement blocks and other constructs do not localize the names used within them. This includes loop variables in `for` loop statements, and the targets of `:=` named assignment, which works the same as all others with regard to scopes. There are, however, some boundary cases whose variables are not available to (and do not clash with) surrounding code, and which involve topics covered in full elsewhere in this book:

- *Comprehension loop variables*: The variable (or variables) `X` in `[X for X in I]` used to refer to the current iteration item in comprehension expressions. Because they might clash with other names and reflect

internal state in generators, such names are local to the expression itself in all comprehension forms: list, dictionary, set, and generator. By contrast, *for* loop *statements* never localize their loop variable (e.g., *X* in *for X in I:*) to the statement block as noted, despite the similarity. Moreover, names assigned by *:=* within a comprehension *do* leak out to the enclosing scope, per the info and demos in [Chapter 20](#).

- *Exception variables*: The variable *X* in *except E as X* used to reference the raised exception in a *try* statement handler. Because this might defer garbage collection's memory recovery, such variables are local to that *except* block, and in fact are *removed* from the containing scope when the block is exited (even if you've used them earlier in your code!). You'll learn all about *try* statements in [Chapter 34](#).
- *Named assignments in lambda*: The variable *X* in *X := Y* used in assignment expressions. In terms of scopes, the *:=* works the same as an unnested *=* assignment statement in general. Keep in mind, though, that a *lambda* function expression creates a new local function scope just like a *def* statement. Hence, any names assigned by a *:=* nested in a *lambda* are local variables that can be used within the body of the *lambda* itself, but are not available to code outside the *lambda* expression. That said, this is likely a rare coding pattern in most code (and pushes the envelope on complexity). For a refresher on *:=*, see [Chapter 11](#).

Most of these contexts *augment* the LEGB rule, rather than modifying it. Loop variables assigned in a comprehension, for example, are simply bound to a further nested and special-case local scope; all other names referenced within these expressions follow the usual LEGB lookup rules.

It's also worth noting that the *class* statement you'll meet in [Part VI](#) creates a new *local* scope, too, for the names assigned inside the top level of its block. As for *def*, names assigned inside a *class* don't clash with names elsewhere, and follow the LEGB lookup rule, where the *class* block is the *L* level. As for imported modules, these class-local names also morph into class object attributes after the *class* statements ends. For all practical purposes, classes are a local

scope (despite some rare border cases that likely qualify as glitches, and can be safely omitted here).

Unlike functions, though, `class` names are not created per *call*: class object calls generate *instances*, which inherit names assigned in the `class` and record per-object state as per-instance attributes of their own. As you'll also learn in [Chapter 29](#), although the LEGB rule is used to resolve names used in both the top level of a class itself as well as the top level of method functions nested within it, classes themselves are *skipped* by scope lookups—their names must be fetched as object attributes. Hence, because Python searches enclosing functions for referenced names, but not enclosing classes, the LEGB rule sketched in [Figure 17-1](#) still applies to OOP code.

Scopes Examples

But enough theory! Let's step through a live example that demos scope ideas. Suppose we wrote the code in [Example 17-1](#) and saved it in a module file named `scopes101.py`.

Example 17-1. scopes101.py

```
# Global scope
X = 99                # X and func assigned in module: global

def func(Y):          # Y and Z assigned in function: locals
    # Local scope
    Z = X + Y         # X is a global when referenced here
    return Z

func(1)               # func in module: result=100 (not printed)
```

To see this example's result, import and call its function; the file's global `func` is a module attribute to importers (be sure to run this in the same folder as the file if it matters for imports in your REPL, as introduced in [Chapter 3](#)):

```
>>> import scopes101
>>> scopes101.func(1)
100
```

This module and the function it contains use a number of names to do their business. Applying Python's scope rules, we can classify these names as follows:

Global names: X, func

X is global because it's assigned at the top level of the module file; it can be referenced inside the function as a simple unqualified variable without being declared global. func is global for the same reason; the `def` statement makes a function object and assigns it to the name func at the top level of the module.

Local names: Y, Z

Y and Z are local to the function (and exist only while the function runs) because they are both assigned values in the function definition: Z by virtue of the `=` statement, and Y because arguments are always passed by assignment. Hence both are assigned during a function call, and both are locals.

The underlying rationale for this name-segregation scheme is that local variables serve as *temporary* names that you need only while a function is running. For instance, in the preceding example, the argument Y and the addition result Z exist only inside the function; these names don't interfere with the enclosing module's namespace—or any other function, for that matter. In fact, local variables are removed from memory when the function call exits, and objects they reference may be *garbage-collected* if not referenced elsewhere. This is an automatic internal step, but it helps minimize memory requirements.

The local/global distinction also makes functions easier to understand, as most of the names a function uses appear in the function itself, not at an arbitrarily distant place in a module file. Also, because you can be sure that local names will not be changed by some remote function in your program, they tend to make programs easier to debug and modify. In the following sort of code, for instance, each function's variable cannot be changed—or even seen—anywhere else:

```
def func1():  
    X = 'hack'          # This X...
```

```
def func2():  
    X = 'code'           # ...is not the same as this X
```

The net effect helps make functions self-contained units of software, by design.

The Built-in Scope

We’ve been talking about the built-in scope in the abstract, but it’s a bit simpler than you may think. Really, the built-in scope is just a built-in module called `builtins`, but you have to import `builtins` to query built-ins because the name `builtins` is not itself a built-in...

Yes, seriously! The built-in scope is implemented as a standard-library module named `builtins`, but that name itself is not placed in the built-in scope, so you have to import it in order to inspect it. Once you do, you can run a `dir` call to see which names are predefined (run this on your own for full fidelity: it can vary across Python versions):

```
>>> import builtins  
>>> dir(builtins)  
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',  
'BaseExceptionGroup', 'BlockingIOError', 'BrokenPipeError', 'BufferError',  
...many more names omitted: 158 total in 3.12..  
'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed',  
'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum',  
'super', 'tuple', 'type', 'vars', 'zip']
```

The names in this list constitute the built-in scope in Python. Roughly the first half are built-in exceptions, and the second half are built-in functions. Also in this list are the special names `None`, `True`, `False`, and `Ellipsis` we met earlier. Because Python automatically searches this module last in its LEGB lookup (it’s *B*), you get all the names in this list “for free.” That is, you can use them without importing any modules. Thus, there are really two ways to refer to a built-in function—by taking advantage of the LEGB rule, or by manually importing the `builtins` module:

```
>>> zip                                     # The normal way, per LEGB's B  
<class 'zip'>
```

```
>>> import builtins          # The hard way: for customizations
>>> builtins.zip
<class 'zip'>

>>> zip is builtins.zip      # Same object, different lookup routes
True
```

The second of these approaches, though more to type, can be useful in advanced roles you’ll meet in the sidebar **“Why You Will Care: Customizing open”**.

Redefining built-in names: For better or worse

Careful readers might notice that because the LEGB lookup procedure takes the *first* occurrence of a name that it finds, names in the local scope may override variables of the same name in both the global and built-in scopes, and global names may override built-ins. A function can, for instance, create a local variable called `open` by assigning to it:

```
def hider():
    open = 'text'          # Local variable, hides built-in here (L before B)
    ...
    open('data.txt')      # Error: this no longer opens files in this scope!
```

However, this will *hide* the built-in function called `open` that lives in the built-in (outer) scope, such that the name `open` will no longer work within the function to open files—it’s now a string, not the opener function. This is sometimes called “shadowing” a built-in. This isn’t a problem if you don’t need to open files in this function, but triggers an error if you attempt to open through this name in this scope; your `open` is no longer the built-in `open`.

This can even occur more simply at the interactive prompt, which works as a global, implicit-module scope:

```
>>> open = 99              # Assign in global scope, hides built-in here too
```

That being said, there is nothing inherently *wrong* with using a built-in name for variables of your own, as long as you don’t need the original built-in version. After all, if these were truly off limits, we would need to memorize the entire built-in names list and treat all its names as reserved. With some 150 usable names in this module in Python 3.12, that would be far too restrictive and

daunting:

```
>>> len(dir(builtins)), len([x for x in dir(builtins) if not x.startswith('__')])
(158, 150)
```

In fact, there are times in advanced programming where you may really *want* to replace a built-in name by redefining it in your code—to define a custom `open` that augments access attempts, for instance (again, more in the sidebar at the end of the chapter). Exception: the special names `None`, `True`, and `False` in the built-in scope are also treated as reserved words today, so they can no longer be reassigned (fun though it once was!). All other built-in names, though, are fair game.

Nevertheless, redefining a built-in name is often a bug, and a nasty one at that, because Python will not issue a warning message about it. You may find tools on the web that can warn you of such mistakes, but knowledge may be your best defense on this point: don't redefine a built-in name you need. If you *accidentally* reassign a built-in name at the interactive prompt this way, though, you can either restart your session or run a `del name` statement to remove the redefinition from your scope, thereby restoring the original in the built-in scope per LEGB.

Note that functions can similarly hide *global* variables of the same name with locals, but this is more broadly useful, and in fact is much of the point of local scopes—because they minimize the potential for name clashes, your functions are self-contained namespaces:

```
X = 88                                # Global X

def func():
    X = 99                            # Local X: hides global, but we want this here

func()
print(X)                             # Prints 88: unchanged
```

Here, the assignment within the function creates a local `X` that is a completely different variable from the global `X` in the module outside the function. As a consequence, though, there is no way to *change* a name outside a function without adding a `global` or `nonlocal` declaration to the `def`; the next section

takes up the former.

NOTE

More built-in tongue twisters: Technically, the name `__builtins__` is preset in most global scopes, including the interactive session, to reference the module known as `builtins`, so you can often use `__builtins__` without an import, but cannot run an import on the name `__builtins__` itself—it's a preset variable, not a module's name. Thus, `builtins is __builtins__` is `True` after you import `builtins`. The upshot is that we can often inspect the built-in scope by simply running `dir(__builtins__)` sans imports, but are advised to use `builtins` for real work and customization. Who said documenting this stuff was easy?

The global Statement

The `global` statement and its `nonlocal` cousin are the only declaration statements in Python that are actually used by Python (for contrast, see [Chapter 6](#)'s discussion of unused type hinting). They are not type or size declarations, though; they are *namespace declarations*. The `global` statement, for instance, tells Python that a function plans to change one or more global names—that is, names that live in the enclosing module's scope (namespace).

We've talked about `global` in passing already. Here's a summary:

- Global names are variables assigned at the top level of the enclosing module file.
- Global names must be declared only if they are assigned within a function.
- Global names may be referenced within a function without being declared, per the LEGB rule.

In other words, `global` allows us to *change* names that live outside a `def` at the top level of a module file. As you'll see later, the `nonlocal` statement is almost identical but applies to names in an enclosing `def`'s local scope, rather than names in the enclosing module.

The `global` statement, usable in `def` but not `lambda`, simply consists of the

reserved word `global`, followed by one or more names separated by commas. All the listed names will be mapped to the enclosing module's scope when assigned or referenced within the function body. For instance, the following is a takeoff on the preceding example:

```
X = 88                                # Global X

def func():
    global X
    X = 99                            # Global X: outside def

func()
print(X)                             # Prints 99
```

We've added a `global` declaration to the example here, such that the `X` inside the `def` now refers to the `X` outside the `def`; they are the same variable this time, so changing `X` inside the function changes the `X` outside it. Here is a slightly more involved example of `global` at work:

```
y, z = 1, 2                          # Global variables in module
def all_global():
    global x                          # Declare globals assigned
    x = y + z                         # No need to declare y or z: LEGB rule
```

Here, `x`, `y`, and `z` are all globals inside the function `all_global`. Names `y` and `z` are global because they aren't assigned in the function, and `x` is global because it was listed in a `global` statement to map it to the module's scope explicitly. Without the `global` here, `x` would be considered local by virtue of the assignment.

Notice that `y` and `z` are not declared global; Python's LEGB lookup rule finds them in the module (`G`) automatically. Also notice that `x` does not even exist in the enclosing module before the function runs; in this case, the first assignment in the function creates `x` in the module. All of which works when needed, but you really should try to avoid using globals like this whenever possible—as the next section will explain.

Program Design: Minimize Global Variables

Functions in general, and global variables in particular, raise some larger design questions. How, for example, should functions communicate? Although some answers will become more apparent when you begin writing larger functions of your own, a few guidelines up front might spare you from problems later. In general, functions should rely on *arguments* and *return* values instead of globals, but this may not at all be obvious to newcomers to programming.

By default, names assigned in functions are locals, so if you want to change names outside functions you have to write *extra* code (e.g., `global` statements). This is deliberate—you have to say more to do the potentially “wrong” thing. Although there are times when globals are useful (and even required), variables assigned in a `def` are local by default because that is normally the best policy. Changing globals can lead to well-known software-engineering problems: because the variables’ values are dependent on the *order* of calls to arbitrarily distant functions, programs can become difficult to debug, or understand at all.

Consider this module file, for example, which is presumably imported and used elsewhere:

```
X = 99

def func1():
    global X                # Change global X when called
    X = 88

def func2():
    global X                # But so does this, and when?
    X = 77
```

Now, imagine that it is your job to modify or reuse this code. What will the value of `X` be here? Really, that question has no meaning unless it’s qualified with a point of reference in *time*—the value of `X` is timing-dependent, as it depends on which function was called last (something we can’t tell from this file alone).

The net effect is that to understand this code, you have to trace the flow of control through the *entire program*. Hence, if you need to reuse or modify the code, you have to keep the entire program in your head all at once. In fact, you can’t really use one of these functions without bringing along the other. They are dependent on—that is, *coupled* with—the global variable. And that is the problem with globals: they generally make code more difficult to understand and

reuse than code consisting of self-contained functions that rely on locals.

On the other hand, short of using tools like the nested scope closures covered ahead or OOP with classes covered later, global variables are the most basic way to retain shared *state information*—information that a function needs to remember for use the next time it is called. Local variables disappear when the function returns, but globals do not. As you’ll see later, other techniques can achieve this, too, and allow for multiple copies of the retained information; but they are more complex than pushing values out to the global scope for retention in simple cases where this applies.

Moreover, some programs designate a single module to collect shared globals; as long as this is expected, it is not as harmful. Programs that use multithreading for parallel processing also commonly depend on global variables—they become shared memory between functions running in parallel threads, and so act as a communication device.²

For now, though, and especially if you are relatively new to programming, avoid the temptation to use globals whenever you can—they tend to make programs difficult to understand and reuse, and won’t work for cases where one copy of saved data is not enough. That’s why they are not the default in Python. Try to communicate with passed-in arguments and return values instead. Six months from now, both you and your coworkers may be glad you did.

Program Design: Minimize Cross-File Changes

While we’re on the subject of globals, here’s another related design note: although we *can* change global variables in another file directly, we usually *shouldn’t*. Module files were introduced in [Chapter 3](#) and are covered in depth in the next part of this book, but their basics are simple. To demo their relationship to scopes, consider these two module files:

```
# first.py
X = 99                                     # This code doesn't know about second.py

# second.py
import first
print(first.X)                            # OK: references a name in another file
first.X = 88                             # But changing it can be too subtle and implicit
```

The first defines a variable `X`, which the second prints and then changes by assignment. Notice that we must import the first module into the second to get to its variable at all—as we’ve learned, each module is a self-contained namespace (package of variables), and we must import one module to see inside it from another. That’s the main purpose of modules: by segregating variables on a per-file basis, they avoid name collisions across files, in much the same way that local variables avoid name clashes across functions.

Really, though, in terms of this chapter’s topic, the global *scope* of a module file becomes the attribute *namespace* of the module object once it is imported—importers automatically have access to all of the file’s global variables, because a file’s global scope morphs into an object’s attribute namespace when it is imported.

After importing the first module, the second module prints its variable and then assigns it a new value. Referencing the module’s variable to print it is fine—this is how modules are linked together into a larger system normally. The problem with the assignment to `first.X`, however, is that it is far too implicit: whoever’s charged with maintaining or reusing the first module probably has no clue that some arbitrarily far-removed module on the import chain can change `X` at runtime. In fact, the second module may be in a different folder, and so difficult to notice at all.

Although such cross-file variable changes are always possible in Python, they are usually much more subtle than you will want. Again, this sets up too strong a *coupling* between two components—because the files are both dependent on the value of the variable `X`, it’s difficult to understand or reuse one file without the other. Such implicit cross-file dependencies can lead to inflexible code at best, and surprising bugs at worst.

Here again, and generally speaking, don’t do that—the better way to communicate across file boundaries is to call functions, passing in arguments and getting back return values. In this specific case, we would probably be better off coding an *accessor function* to manage the change:

```
# first.py
X = 99

def setX(new):          # Accessors make external changes explicit
```

```

    global X                                # And can manage access in a single place
    X = new

# second.py
import first
first.setX(88)                             # Call the function instead of changing directly

```

This requires more code and may seem like a trivial change, but it makes a huge difference in terms of readability and maintainability—when a person reading the first module by itself sees a function, that person will know that it is a point of *interface* and will expect the change to the X. In other words, it removes the element of surprise that is rarely a good thing in software projects. Although we cannot prevent cross-file changes from happening (sans obscure hacks that we’ll omit here), common sense dictates that they should be minimized unless widely known across the program.

Other Ways to Access Globals

Interestingly, because global-scope variables morph into the attributes of a loaded module object, we can emulate the `global` statement by importing the enclosing module and assigning to its attributes, as in the module file in [Example 17-2](#). Code in this file accesses its enclosing module, first by importing it, and then by indexing the `sys.modules` dictionary that records all loaded modules and is used in advanced roles (there’s more on this dictionary in [Part V](#)).

Example 17-2. thismod.py

```

"Change a global three ways"

var = 99                                # Global variable == module attribute

def local():
    var = 0                             # Change local var

def glob1():
    global var                           # Declare global (normal)
    var += 1                             # Change global var

def glob2():
    var = 0                             # Change local var
    import thismod                       # Import myself
    thismod.var += 1                     # Change global var

```

```
def glob3():
    var = 0                                # Change local var
    import sys                             # Import system table
    thismod = sys.modules['thismod']       # Get module object (or use __name__)
    thismod.var += 1                       # Change global var

def test():
    print(var)
    local(); glob1(); glob2(); glob3()
    print(var)
```

When we import and call this module’s `test` to invoke its other functions, this adds 3 to the global variable `var`—only its first function, `local`, does not impact the global:

```
>>> import thismod
>>> thismod.test()
99
102
>>> thismod.var
102
```

All these global-access techniques work, and illustrate the equivalence of the global scope to module attributes, but they’re noticeably more work than using the `global` statement to make your intentions explicit.

As we’ve seen, `global` allows us to easily change names in a module outside a function. It has a close relative named `nonlocal` that can be used to change names in enclosing functions, too—but to understand how that can be useful, we first need to explore function nesting in general, the topic we turn to next.

Nested Functions and Scopes

So far, this chapter has largely omitted one part of Python’s scope rules on purpose, because it’s relatively uncommon to encounter it in practice. However, it’s time to take a deeper look at layer *E* in the LEGB lookup rule. This layer was added during Python 2.X’s reign. It takes the form of the local scopes of any and all enclosing function’s local scopes. Enclosing scopes are sometimes also called *statically nested scopes*. Really, the nesting is a *lexical* one—nested scopes correspond to physically and syntactically nested code structures in your

program's source code text.

Nested Scopes Overview

With the addition of nested function scopes, variable lookup rules become slightly more complex. Within a function:

- A *name reference* (`X`) looks for the name `X` first in the current local scope (function); then in the local scopes of any lexically enclosing functions in your source code, from inner to outer; then in the current global scope (the module file); and finally in the built-in scope (which we've seen means the built-in module `builtins`). `global` declarations in a `def` make the search begin in the global (module file) scope instead.
- A *name assignment* (e.g., `X = value`) creates or changes the name `X` in the current local scope, by default. If `X` is declared `global` within the function, the assignment creates or changes the name `X` in the enclosing module's scope instead. If, on the other hand, `X` is declared `nonlocal` within a `def` function, the assignment changes the name `X` in the local scope of the closest enclosing function that assigns the same name.

Notice that the `global` declaration still maps variables to the enclosing module. When nested functions are present, variables in enclosing functions may be referenced, but require `nonlocal` declarations to be changed. Also note that, unlike `global`, `nonlocal` does not create a name in an enclosing `def` if it's not assigned there; `nonlocal` makes sense only if the variables it names are also assigned by an enclosing `def`—and is an error to use otherwise.

This section is primarily concerned with the first bullet in the preceding list: nested scope *references*. We'll explore nested *assignments* and `nonlocal` in a moment, but first we need to grok function nesting in general.

Nested Scopes Examples

Let's turn to code to illustrate some of the preceding points. Here is what an enclosing function scope looks like (type this into a script file or at the interactive prompt to run it live):

```

X = 99                                # Global scope name: not used

def f1():
    X = 88                            # Enclosing def local
    def f2():
        print(X + 1)                 # Reference made in nested def
        f2()

f1()                                  # Prints 89: enclosing def local + 1

```

First off, this is legal Python code: the `def` is simply an executable statement, which can appear anywhere any other statement can—including nested in another `def`. Here, the nested `def` runs while a call to the function `f1` is running; it makes a function and assigns it to the name `f2`, a local variable within `f1`'s local scope. In a sense, `f2` is a temporary function that lives only during the execution of (and is visible only to code in) the enclosing `f1`. As such, its name won't clash with any other part of the program—which is one reason to nest it this way.

But notice what happens inside `f2`: when it uses the variable `X`, it refers to the `X` that lives in the *enclosing* `f1` function's local scope. Because functions can access names in all physically enclosing functions, the `X` in `f2` is automatically mapped to the `X` in `f1`, by the LEGB lookup rule's level *E*.

In fact, this enclosing scope lookup works even if the enclosing function's call has already *ended*. For example, the following code defines a function that makes and *returns* another function, and introduces a common nesting role:

```

def f1():
    X = 88
    def f2():
        print(X + 1)                 # Remembers X in enclosing def scope
        return f2                   # Return f2 but don't call it

action = f1()                       # Make, return function
action()                            # Call it now: prints 89

```

In this code, the call to `action` is really running the function we named `f2` when `f1` ran. This works naturally because functions are objects in Python like everything else and can be passed back as return values from other functions.

More subtly, `f2` “remembers” the enclosing scope’s `X` in `f1`—even though `f1` is no longer active when `f2` is run. Though abstract here, this memory behavior turns out to be one of the main reasons to nest functions, and warrants a closer look in the next section.

Closures and Factory Functions

Depending on whom you ask, the preceding example’s behavior is sometimes called a *closure* or a *factory* function—the former describing a *functional programming* technique, and the latter denoting a *design pattern*. In code, a factory function creates and returns a function that has a stateful closure, but the two terms are intertwined.

Whatever the label, the function object in question remembers values in enclosing scopes regardless of whether those scopes are still active for a call. In effect, it has attached packets of memory (a.k.a. *state retention*), which are made anew for each copy of the nested function created, and often provide a simple alternative to classes in this role.

Closures (a.k.a. factory functions) are sometimes used by programs that need to generate event handlers on the fly in response to conditions at runtime. For instance, imagine a GUI that must define actions according to user inputs that cannot be anticipated when the GUI is built and vary per action. In such cases, we need a function that creates and returns another function, with information (i.e., state) that differs per function made.

To demo this live in simplified terms, consider the following function, typed at the interactive prompt:

```
>>> def maker(N):  
    def action(X):  
        return X ** N  
    return action
```

Make and return action
action retains N from enclosing scope

This defines an outer function that simply makes and returns a nested function, without calling it—`maker` makes `action`, but simply returns `action` without running it. If we then call the *outer* function:

```
>>> f = maker(2)
```

Pass 2 to argument N

```
>>> f
<function maker.<locals>.action at 0x101ae7ba0>
```

What we get back is a reference to the *nested* function built—the one created and assigned to `action` when the nested `def` runs. If we now call what we got back from the outer function:

```
>>> f(3)          # Pass 3 to action's X
9                # And maker's N remembers 2: 3 ** 2
>>> f(4)          # Pass 4 to X, N is still 2: 4 ** 2
16
```

We invoke the nested function—the one called `action` within `maker`. In other words, we're calling the nested function that `maker` created and passed back.

Perhaps the most unusual part of this, though, is that the nested function *remembers* integer 2, the value of the variable `N` in `maker`, even though `maker` has returned and exited by the time we call `action`. In effect, `N` from the enclosing local scope is retained as state information attached to the generated `action`, which is why we get back its argument squared whenever it is later called.

Just as important, if we now call the outer function again, we get back a *new* nested function with *different* state information attached. In the following, we compute the argument cubed instead of squared when calling the new function, but the original still squares as before:

```
>>> g = maker(3)  # g remembers 3, f remembers 2
>>> g(4)          # 4 ** 3
64
>>> f(4)          # 4 ** 2
16
```

This works because each call to a closure/factory function like this gets its *own* set of state information. In our demo, the function we assign to name `g` remembers 3, and `f` remembers 2, because each has its own state information retained by the variable `N` in `maker`. In a GUI, `N` might be a username, email address, or other per-function state.

This is all literal inside Python. Nested functions have an attached closure

storage area for the enclosing scope names they use. It's available as the `__closure__` attribute of such functions (and fetching `cell_contents` after indexing this tuple yields a state item), but we don't need to understand Python internals to use closures in our code.

This is also a somewhat advanced technique that you may not see commonly in most code, and may be more popular among programmers with backgrounds in functional programming languages. On the other hand, enclosing scopes are often employed by the `lambda` function-creation expressions introduced in the prior chapter—because `lambda` is an expression, it is almost *always* nested in a `def`. For example, a `lambda` can serve in place of a `def` in our demo, and also relies on enclosing scope references to retain state—like `N` in the following variation:

```
>>> def maker(N):
    return lambda X: X * N          # lambda functions retain state too

>>> h, i = maker(2), maker(3)      # Make two closure functions

>>> h('Py'), i('Py')              # Run lambdas: ('Py' * 2) and ('Py' * 3)
('PyPy', 'PyPyPy')
```

For a more tangible example of closures at work, see the sidebar “[Why You Will Care: Customizing open](#)”. It uses similar techniques to store information for later use in an enclosing scope. As you'll also see after the next section's closer, closures become more useful when their state becomes changeable with `nonlocal`.

Arbitrary Scope Nesting

Finally, this tour of nested function scopes would be remiss if it didn't point out that scopes may nest arbitrarily, though only enclosing functions (not classes, described in [Part VI](#)) are searched when names are referenced:

```
>>> def f1():
    x = 99
    def f2():
        def f3():
            print(x)          # Found in f1's local scope!
        f3()
```

`f2()`

```
>>> f1()  
99
```

This does work: Python will search the local scopes of *all* enclosing `defs`, from inner to outer, after the referencing function's own local scope and before the module's global scope or built-ins. However, this sort of code is even less likely to crop up in practice. Per coding aphorism, *flat is better than nested*, and this still holds generally true even with nested scope closures in the toolbox. Except in limited contexts, your life (and the lives of your fellow travelers in the software realm) will generally be better if you minimize nesting in function definitions.

The `nonlocal` Statement

In the prior section, we saw how nested functions can *reference* variables in an enclosing function's scope, even if that function has already returned. As suggested earlier, we can also *change* such enclosing scope variables, as long as we declare them in `nonlocal` statements. With this statement, nested functions gain both read and write access to names in enclosing functions. This makes nested scope closures more useful, by making state changeable.

The `nonlocal` statement, usable in `def` but not `lambda`, is similar in both form and role to `global`, covered earlier. Like `global`, `nonlocal` declares that a name (or names) will be changed in an enclosing scope. Unlike `global`, though, `nonlocal` applies to a name in an enclosing function's scope, not to the global module scope outside all functions. Also unlike `global`, `nonlocal` names must exist in an enclosing function's scope—they are mapped only to enclosing functions and cannot be created by a first assignment in a nested `def` that uses `nonlocal`.

In other words, `nonlocal` both *allows* assignment to names in enclosing function scopes, and *limits* scope lookups for such names to enclosing functions. The net effect is a direct and reliable implementation of changeable state information, for contexts that do not desire or need to use classes or other stateful tools.

nonlocal Basics

The `nonlocal` statement has meaning only inside a function, and is an error to use elsewhere. More specifically, it applies and is usable only when `def` is nested in another `def`: because the body of a `lambda` allows just an *expression*, it supports neither `nonlocal` nor nested `def` statements.

When used within a `def`, the `nonlocal` statement seals the fate of references—much like the `global` statement, `nonlocal` causes searches for the names listed in the statement to begin in the enclosing `defs`' scopes, not in the local scope of the declaring function. That is, `nonlocal` also means “skip my local scope entirely.” In fact, the names listed in a `nonlocal` *must* be assigned in an enclosing `def`, and never refer to names in the global or built-in scopes.

Importantly, the addition of `nonlocal` does not alter name reference scope rules in general; they still work as before, per the LEGB rule described earlier. The `nonlocal` statement simply serves to allow names in enclosing scopes to be changed rather than just referenced. The `global` and `nonlocal` statements, though, tighten up and even restrict the lookup rules within a function for the names that they list:

- `global` makes scope lookup begin in the enclosing module's scope and allows names there to be assigned. Scope lookup continues on to the built-in scope if the name does not exist in the module, but assignments to global names always create or change them in the module's scope.
- `nonlocal` restricts scope lookup to just enclosing `defs`, requires that the names exist there, and allows them to be reassigned. Scope lookup does not continue on to the global or built-in scopes.

nonlocal in Action

Let's move on to examples to make this more concrete. Simple *references* to enclosing `def` scopes work as we've already seen—in the following, `outer` builds and returns the function `inner` to be called later, and the `state` reference in `inner` maps the local scope of `outer` using the normal scope LEGB lookup rules:

```

>>> def outer(start):
    state = start           # Referencing nonlocals works normally
    def inner(label):
        print(label, state) # Remembers state in enclosing scope
    return inner

>>> F = outer(0)
>>> F('code')              # State is the same on every inner run
code 0
>>> F('hack')
hack 0

```

Changing a name in an enclosing def's scope, however, is not allowed by default:

```

>>> def outer(start):
    state = start
    def inner(label):
        state += 1          # Cannot change by default
        print(label, state)
    return inner

>>> F = outer(0)
>>> F('code')
UnboundLocalError: cannot access local variable 'state' ...

```

Now, if we declare `state` in the outer scope as `nonlocal` within `inner`, we get to both reference and *change* it inside the nested function. Again, this works even though `outer` has returned and exited by the time we call the returned `inner` function through the name `F`:

```

>>> def outer(start):
    state = start           # Each call gets its own state
    def inner(label):
        nonlocal state      # <= Remembers state in enclosing scope
        state += 1          # Allowed to change it if nonlocal
        print(label, state)
    return inner

>>> F = outer(0)
>>> F('code')              # Increments state on each call
code 1
>>> F('hack')
hack 2

```


As usual with enclosing scope references, we can call the `outer` factory (closure) function multiple times to get multiple copies of its state in memory. The `state` object in the enclosing scope is essentially attached to the new `inner` function object returned; each call makes a new, distinct `state` object, such that updating one function's state won't impact the other. The following continues the prior listing's interaction:

```
>>> G = outer(24)                # Make a new outer that starts at 24
>>> G('code')
code 25
>>> G('hack')                    # G's state information updated to 26
hack 26

>>> F('more')                   # But F's state is where it left off: at 2
more 3                           # Each call has different state information
```

As you can see, Python's nonlocals are much more functional than “static” function locals typical in some other languages: in a closure function, nonlocals are *per-call, multiple copy* data.

nonlocal Boundary Cases

Though useful, nonlocals come with some subtleties to be aware of. First, unlike the `global` statement, `nonlocal` names really *must* be assigned in an enclosing `def`'s scope—you cannot create them dynamically by assigning them anew in a nested function. The enclosing function's assignment can appear either before or after the nested function, but you'll get an error if it's missing. In fact, nonlocals are checked for an enclosing function assignment at function definition time, before either an enclosing or nested function is ever called:

```
>>> def outer(start):
    def inner(label):
        nonlocal state          # Nonlocals must exist in an enclosing def
        state = 0
        print(label, state)
    return inner
```

SyntaxError: no binding for nonlocal 'state' found

```
>>> def outer(start):
    def inner(label):
```

```

        global state          # But globals don't have to exist when declared
        state = 0
        print(label, state)
    return inner

>>> F = outer(0)
>>> F('glob')
glob 0
>>> state                    # Created by the assignment in nested inner
0

```

Second, `nonlocal` restricts the scope lookup to *just* enclosing `defs`; nonlocals are not looked up in the enclosing module's global scope or the built-in scope outside all `defs`, even if they are already there:

```

>>> state = 0
>>> def outer():
    def inner():
        nonlocal state      # Must be located in a def, not the module
        state += 1          # Use global for globals, not nonlocal
    return inner

```

SyntaxError: no binding for nonlocal 'chapter' found

```

>>> def outer():
    def inner():
        nonlocal ord        # Ditto for built-ins (and immediate calls)
        print(ord)
    inner()

```

SyntaxError: no binding for nonlocal 'ord' found

These restrictions make sense once you realize that Python would not otherwise generally know in which enclosing scope to create a brand-new name. In the prior listing, should `state` be assigned in `outer`, or the module outside? Because this is ambiguous, Python must resolve nonlocals at function *creation* time, not function *call* time.

Finally, this chapter has been careful to say that nonlocal names must be assigned in *an* enclosing function, not *the* enclosing function. This can matter when function nesting runs deep: a name listed in `nonlocal` can technically appear *anywhere* in the hierarchy of enclosing functions—not just one level up—and the closest appearance is used. Even so, because such code seems unlikely

to crop up in practice (and probably qualifies as cruel and unusual punishment), we'll forgo a formal demo here, but see “[Arbitrary Scope Nesting](#)” for most of the morass.

State-Retention Options

Given the extra complexity of nested functions, you might wonder why they're worth the fuss. Although it's difficult to see in our small examples, state information becomes essential in many programs. While functions can return results, their local variables won't normally retain other values that must live on between calls. Moreover, many applications require such values to differ per context of use.

Broadly speaking, there are multiple ways for Python functions to retain state between calls. These include the global variables and enclosing scope references we've already met, but also class-instance attributes and function attributes. To close out this chapter, let's review these options to see how they stack up.

Nonlocals: Changeable, Per-Call, LEGB

First off, the following code is a recap from the prior section, repeated here for compare and contrast. As we've seen, its `nonlocal` allows state in an enclosing scope to be saved and modified. Each call to `outer` makes a self-contained *package of changeable information*, whose names and objects do not clash with any other part of the program:

```
>>> def outer(start):
    state = start                                # Each call gets its own state
    def inner(label):
        nonlocal state                          # Remembers state in enclosing scope
        state += 1                              # Allowed to change it if nonlocal
        print(label, state)
    return inner

>>> F = outer(0)                                # State to be saved in enclosing scope
>>> F('nonlocal1')                             # State visible within closure only
nonlocal1 1
>>> F('nonlocal2')
nonlocal2 2
```

```
>>> F.state
AttributeError: 'function' object has no attribute 'state'
```

We need to declare variables `nonlocal` only if they must be changed (other enclosing scope name references are automatically retained as usual per LEGB), and `nonlocal` names are not visible outside the enclosing function.

While this scheme works well, the next three sections present some alternatives. Some of the code in these sections uses tools we haven't covered yet and is intended partially as preview, but we'll keep the examples simple here so that you can compare and contrast along the way.

Globals: Changeable but Shared

One common suggestion for achieving state retention without `nonlocal` is to simply move saved info out to the *global scope* (the enclosing module):

```
>>> def outer(start):
    global state                # Move it out to the module to change it
    state = start              # global allows changes in module scope
    def inner(label):
        global state
        state += 1
        print(label, state)
    return inner

>>> F = outer(0)
>>> F('global1')              # Each call increments shared global state
global1 1
>>> F('global2')
global2 2
>>> state                    # State accessible as global outside defs
2
```

This works in this case, but it requires `global` declarations in both functions and is prone to name collisions in the global scope (what if “state” is already being used for something else?). A more subtle problem is that it only allows for a *single shared copy* of the state information in the module scope—if we call `outer` again, we'll wind up resetting the module's `state` variable, such that prior calls will see their `state` overwritten:

```

>>> G = outer(24)                # Resets state's single copy in global scope
>>> G('global3')
global3 25

>>> F('global4')                # But F's counter has been overwritten!
global4 26

```

As shown earlier, when you use `nonlocal` and nested function closures instead of `global`, each call to `outer` remembers its own unique copy of the `state` object. For per-call roles, globals don't fit the bill.

Function Attributes: Changeable, Per-Call, Explicit

As another state-retention option, we can often achieve the same effect as nonlocals with *function attributes*—user-defined names attached to functions explicitly. When you attach user-defined attributes to nested functions generated by factory functions, they can also serve as per-call, multiple copy, and writeable state, just like nonlocal scope closures. Such user-defined attribute names won't clash with names Python creates itself, and as for `nonlocal`, need be used only for state variables that must be *changed*; other scope references are retained and work normally.

Because factory functions make a new function on each call anyhow, this does not require extra objects—the new function's attributes become per-call state in much the same way as nonlocals, and are similarly associated with the new function in memory. In contrast, function attributes are perhaps less *magical* than scopes, and allow state variables to be accessed *outside* the nested function. With `nonlocal`, state variables can be seen directly only within the nested `def`; with attributes, state access is a simple function-attribute fetch.

Here's a mutation of our example based on this technique—it replaces a `nonlocal` with an attribute attached to the nested function. This scheme may not seem as intuitive to some at first glance; you must access state explicitly through the function's name instead of as simple variables, and must initialize *after* the nested `def`. Still, it allows state to be accessed externally, saves a line by eliminating a `nonlocal` declaration, and makes state usage more explicit:

```

>>> def outer(start):
    def inner(label):

```

```

        inner.state += 1           # Change object attr, not inner itself
        print(label, inner.state) # inner is in the enclosing scope
    inner.state = start           # Initialize state after inner defined
    return inner

>>> F = outer(0)
>>> F('attr1')                  # F is an inner with state attached
attr1 1
>>> F('attr2')
attr2 2

>>> F.state                      # Can access state outside defs too
2

```

Because each call to the outer function produces a new nested function object, this scheme supports multiple-copy, *per-call* changeable data just like nonlocal closures—a usage mode that global variables cannot provide:

```

>>> G = outer(24)                # G has own state, doesn't overwrite F's
>>> G('attr3')
attr3 25
>>> F('attr4')                  # F's state varies
attr4 3

>>> F.state                      # State is accessible and per-call
3
>>> G.state
25
>>> F is G                       # Different function objects
False

```

Fine points: this code relies on the fact that the function name `inner` is a local variable in the outer scope enclosing `inner`; as such, it can be referenced freely inside `inner` per LEGB’s *E*. This code also relies on the fact that changing an object in place is not an assignment to a name; when it increments `inner.state`, it is changing part of the *object* `inner` references, not the name `inner` itself (much like the `L.append` call we saw earlier). Because we’re not really assigning a *name* in the enclosing scope, no `nonlocal` declaration is required.

We’ll explore function attributes further in [Chapter 19](#). Importantly, you’ll see there that Python uses naming conventions that ensure that the arbitrary names you assign as function attributes won’t clash with names related to internal implementation, making the namespace equivalent to a user scope. At the end of

the day, function attributes both predate `nonlocal` and provide similar utility, making the latter technically redundant in some roles.

NOTE

State with enclosing scope mutables: On a related note, it's also possible to change a mutable object like a *list* in the enclosing scope without declaring its name `nonlocal`. The following, for example, implements changeable per-call state information, and largely works the same as the preceding version (though it does not support access to state info from outside the functions):

```
def outer(start):
    def inner(label):
        state[0] += 1          # Clever hack or dark magic?
        print(label, state[0]) # Leverage in-place mutable change
    state = [start]
    return inner
```

This exploits the mutability of lists, and like function attributes, relies on the fact that in-place object changes do not classify a name as local. This is perhaps more obscure than either function attributes or `nonlocal`, though—it's a technique that predates others, and seems to lie today somewhere on the spectrum from arcane hack to dated workaround. You're probably better off using named function attributes than lists and numeric offsets this way (but you can't control what others code).

Classes: Changeable, Per-Call, OOP

Another standard prescription for changeable state information in Python is to use *classes with attributes*. Like function attributes, this scheme makes state information access more explicit than the implicit magic of scope lookup rules. In addition, each instance of a class gets a fresh copy of the state information, as a natural byproduct of Python's object model. Classes also support inheritance, multiple behaviors, and other OOP tools above and beyond functions.

As an abstract and partial illustration, the following defines two stateful instances of a class:

```
class Book:
    ...code to define method functions that manage and use state...

lp6e = Book()          # Make an instance of the class
```

```
lp6e.year = 2024          # Access instance state via attributes
lp6e.python = 3.12

lp5e = Book()             # Make another instance of the class
lp5e.year = 2013          # Each instance has its own attributes/state
lp5e.python = 3.3
```

Because classes support a broader array of tools, they tend to require more code than closure functions, but may be better in more demanding roles. We haven't explored classes in any sort of detail yet, though, so we'll have to cut this section short here. Watch for classes, and their explicit flavor of multiple-copy state information, in [Part VI](#).

And the Winner Is...

As a wrap-up, globals, nonlocals, function attributes, and classes all offer changeable state-retention options. Globals support only single-copy shared data, but nonlocals, function attributes, and classes all support multiple-copy changeable state. Of the latter, nonlocals rely on an implicit LEGB lookup, function attributes are manual but explicit and allow state access outside of functions, and classes are a larger solution that comes with OOP's complexities.

As usual, the best tool for your program depends upon your program's goals. We've seen that `nonlocal` provides changeable state for nested functions with a dedicated statement, and is especially useful for simpler state-retention needs where global variables do not apply and classes may not be warranted. That being said, function attributes can often serve the same roles as `nonlocal`, with arguably less implicit behavior.

We'll revisit state-retention options introduced here in [Chapter 39](#) for a more realistic use case—*decorators*, a tool that by nature involves multilevel state retention. State options have additional selection factors (e.g., performance), which we'll have to leave unexplored here for space (you'll learn how to time code speed in [Chapter 21](#)). For now, it's time to explore one more technique, which both confounds the LEGB rule and segues to the next chapter.

Scopes and Argument Defaults

In early versions of Python, the enclosing scope references we've used in this chapter failed because nested functions did not do anything about scopes—a reference to an enclosing function's variable would search only the local, global, and built-in scopes. Because it skipped the scopes of enclosing functions, an error would result. To work around this, programmers typically used *default argument values* to pass in and remember the objects in an enclosing scope.

Though a preview of the next chapter's arguments coverage, this also bears on scopes. In the following, a takeoff of an earlier example, a value from the enclosing function's scope is passed into a nested function via an unused argument's default:

```
def f1():
    X = 88
    def f2(X=X):          # Remember enclosing scope X with defaults
        print(X + 1)
        f2()

f1()                      # Prints 89
```

This syntax also works in `lambda`, which, as we've learned, naturally and normally creates nested-function scopes:

```
def f1():
    X = 88
    f2 = lambda X=X: print(X + 1)
    f2()
```

Both of these examples still work in all Python releases, and rely on argument defaults. In short, the syntax *arg=val* in a function header means that the argument *arg* will default to the value *val* if no real value is passed to *arg* in a call. In the preceding code, this is used to explicitly assign enclosing scope state to be retained.

Specifically, in the modified `f2`s here, the `X=X` means that the argument `X` on the left will default to the value of `X` in the enclosing scope—because the `X` on the right is evaluated *before* Python steps into the nested function, it still refers to the `X` in `f1`. In effect, each `f2`'s default argument `X` remembers what `X` was in the enclosing `f1`: the object 88.

That's fairly subtle, and it depends entirely on the *timing* of default-value evaluations. It's also an *incidental*—if not accidental—feature: this works only if a real value is never passed to argument *X* to overwrite the default. In fact, the nested scope lookup rule was added to Python in part to make defaults unnecessary for this role: today, Python *automatically* remembers any values required from the enclosing scope for use in nested `defs` and `lambdas`. As we've seen, this example today works the same sans defaults:

```
def f1():
    X = 88
    def f2():
        print(X + 1)
    f2()
```

Remember enclosing scope X per LEGB rule
And likewise for lambda

That said, flat is generally better than nested again, and function nesting in some such code makes programs more complex than they need be. The following, for instance, is an equivalent of the prior examples that avoids nesting altogether. Notice the forward reference to `f2` inside `f1` in this code—it's OK to call a function defined *after* the function that calls it, as long as the second `def` runs before the first function is actually *called*. Code inside a function's body is never evaluated until the function is later called:

```
def f1():
    X = 88
    f2(X)
```

Pass x along instead of nesting
Forward reference OK

```
def f2(X):
    print(X + 1)
```

Flat is still often better than nested

If you avoid nesting this way, you can almost forget about the nested scopes concept in Python. On the other hand, nested functions can avoid *name clashes* by localizing the names of functions used nowhere else, and nesting is the basis of *closure functions*, which support stateful callable objects useful in a variety of roles. When functions are nested for such reasons, the LEGB rule almost makes defaults unnecessary for saving state from an enclosing function's scope—*except* in the following case.

Loops Require Defaults, Not Scopes

So why bother learning an outdated scope-reference scheme? Because it's still *required* in one common case: if a `lambda` or `def` is nested in a *loop*, and the nested function references an enclosing scope variable that is changed by that loop, then all functions generated within the loop will have the same value—the value the referenced variable had in the *last* loop iteration. In such cases, you must still use defaults to save the variable's *current* value instead.

This may seem obscure, but it can come up in practice more often than you may think, especially in code that generates event-handler functions for a number of widgets in a GUI—for instance, handlers for button-clicks for all the buttons in a panel. If these are created in a loop (and they often will be), you need to be careful to save state with defaults, or all your buttons' callbacks may wind up doing the same thing.

Here's an illustration of this phenomenon reduced to simple code: the following attempts to build up a list of functions that each remember the current variable `i` from the enclosing scope:

```
>>> def makeActions():
    acts = []
    for i in range(5):
        acts.append(lambda x: i ** x)
    return acts
# Try to remember each i in 0..4
# But all remember same last i!

>>> acts = makeActions()
>>> acts[0]
<function makeActions.<locals>.<lambda> at 0x101ae7ec0>
```

Interestingly, this can also be coded as a list comprehension, where a `lambda` can be used as the collection result, and the list comprehension serves as a local scope for its loop variable:

```
>>> acts = [(lambda x: i ** x) for i in range(5)]
>>> acts[0]
<function <lambda> at 0x10de6bce0>
```

Either way, though, this doesn't quite work—because the enclosing scope variable `i` is looked up when the nested functions are later *called*, they all effectively remember the same value: the value the loop variable had on the *last* loop iteration. That is, when we pass a power argument of 2 to `x` in each of the

following calls, we get back 4 to the power of 2 for each function in the list, because *i* is the same in all of them—4:

```
>>> acts[0](2)           # All are 4 ** 2, 4=last i
16
>>> acts[1](2)           # This should be 1 ** 2 (1)
16
>>> acts[2](2)           # This should be 2 ** 2 (4)
16
>>> acts[4](2)           # Only this should be 4 ** 2 (16)
16
```

In this case, we still have to explicitly retain enclosing scope values with default arguments, rather than enclosing scope references. That is, to make this sort of code work, we must pass in the *current* value of the enclosing scope's variable with a default. Here's the required mod for both the *def* and comprehension versions:

```
>>> def makeActions():
    acts = []
    for i in range(5):           # Use defaults instead
        acts.append(lambda x, i=i: i ** x) # Remember _current_ i
    return acts

>>> acts = [(lambda x, i=i: i ** x) for i in range(5)]
```

In either coding, because defaults are evaluated when the nested function is *created* (not when it's later *called*), each remembers its own value for *i*:

```
>>> acts = makeActions()
>>> acts[0](2)           # 0 ** 2
0
>>> acts[1](2)           # 1 ** 2
1
>>> acts[2](2)           # 2 ** 2
4
>>> acts[4](2)           # 4 ** 2
16
```

Nor are function attributes a fix here: because a function's own *name* is a variable from the enclosing scope that changes in the loop too, in every function it may reference the *last* function made by the *last* loop iteration. We'll omit the

gory details here, but keep in mind that *any* enclosing scope reference changed by a loop may require defaults.

This may seem an odd special case, but it reflects Python’s implementation of variable scopes, and will become more likely to crop up as you start writing larger programs. This case is also rooted in both scopes and arguments defaults, and to understand the latter in full, we have to move on to the next chapter.

NOTE

State with argument-default mutables: Also on a related note, it’s possible to retain state too with mutable argument defaults like lists and dictionaries (e.g., `def f(a=[])`). Because defaults are implemented as objects attached to functions at function *creation* time, mutable defaults retain state from call to call, rather than being initialized anew on each call. Defaults can also retain mutables from an enclosing function’s scope, thereby enabling changeable per-call state information.

Depending on whom you ask—and when you ask them—this is either a feature that supports state retention, or a perilous and dark corner of the language to be avoided. Usually, programmers expect defaults initialized with literals like `[]` to be re-created on every call, and are surprised when they retain prior calls’ values. More on this in “[Function Gotchas](#)”.

Chapter Summary

In this chapter, we studied one of two key concepts related to functions: *scopes*, which determine how variables are looked up when used. As we learned, variables are considered local to the function definitions in which they are assigned, unless they are specifically declared to be global or nonlocal. We also explored some more advanced scope concepts here, including nested function scopes and function attributes. Finally, we looked at some general design ideas, such as the need to minimize globals and cross-file changes.

In the next chapter, we're going to continue our function tour with the second key function-related concept: argument passing. As you'll find, arguments are passed into a function by assignment, but Python also provides tools that allow functions to be flexible in how items are passed, including the defaults we previewed here. Before we move on, let's take this chapter's quiz to review the scope concepts we've covered here.

Test Your Knowledge: Quiz

1. What is the output of the following code, and why?

```
>>> X = 'Hack'
>>> def func():
    print(X)

>>> func()
```

2. What is the output of this code, and why?

```
>>> X = 'Hack'
>>> def func():
    X = 'Py!'

>>> func()
```

```
>>> print(X)
```

3. What does this code print, and why?

```
>>> X = 'Hack'
>>> def func():
    X = 'Py!'
    print(X)
```

```
>>> func()
>>> print(X)
```

4. What output does this code produce? Why?

```
>>> X = 'Hack'
>>> def func():
    global X
    X = 'Py!'
```

```
>>> func()
>>> print(X)
```

5. What about this code—what's the output, and why?

```
>>> X = 'Hack'
>>> def func():
    X = 'Py!'
    def nested():
        print(X)
    nested()
```

```
>>> func()
>>> print(X)
```

6. How about this example: what is its output, and why?

```
>>> def func():
    X = 'Py!'
    def nested():
        nonlocal X
        X = 'Hack'
    nested()
    print(X)

>>> func()
```

7. Name three or more ways to retain state information across calls in a Python function.

Test Your Knowledge: Answers

1. The output here is Hack because the function references a global variable in the enclosing module (because it is not assigned in the function, it is considered global).
2. The output here is Hack again because assigning the variable inside the function makes it a local and effectively hides the global of the same name. The `print` statement finds the variable unchanged in the global (module) scope.
3. It prints Py! on one line and Hack on another, because the reference to the variable within the function finds the assigned local and the reference in the `print` statement finds the global.
4. This time it just prints Py! because the global declaration forces the variable assigned inside the function to refer to the variable in the enclosing global scope, even though the variable is assigned inside the function.
5. The output in this case is again Py! on one line and Hack on another, because the `print` statement in the nested function finds the name in the enclosing function's local scope, and the display at the end finds the

variable in the global scope.

6. This example prints Hack because the `nonlocal` statement means that the assignment to `X` inside the nested function changes `X` in the enclosing function's local scope. Without this statement, this assignment would classify `X` as local to the nested function, making it a different variable; the code would then print `Py!` instead.
7. Although the values of local variables go away when a function returns, you can make a Python function retain state information by using shared *global* variables, *nonlocal* enclosing scope references within nested functions, or using *default* argument values. Function *attributes* also allow state to be attached to the function itself, instead of looked up in scopes. Another alternative, using *classes* and OOP, sometimes supports state retention better than any of the scope-based techniques because it makes it explicit with attribute assignments; we'll explore this option in [Part VI](#). Changing *mutable* objects in scopes and defaults works too, but may not be legal in some locales.

WHY YOU WILL CARE: CUSTOMIZING OPEN

For another example of closures at work, consider changing the built-in `open` call to a custom version as suggested earlier in this chapter. If the custom version needs to call the original, it must save it before changing it, and retain it for later use—a classic state retention scenario. Moreover, if we wish to support multiple customizations to the same function, globals won't do: we need per-customizer state.

The following, coded in file *makeopen.py*, is one way to achieve this. It uses a nested scope closure to remember a value for later use, without relying on global variables—which can clash and allow just one value, and without using a class—that may require more code than is warranted here:

```
import builtins

def makeopen(id):
    original = builtins.open
    def custom(*pargs, **kargs):
        print(f'Custom open call {id}' , pargs, kargs)
```

```
    return original(*pargs, **kargs)
builtins.open = custom
```

To change `open` for every module in a process, this code reassigns it in the built-in scope to a custom version coded with a nested `def`, after saving the original in the enclosing scope so the customization can call it later. This code is partially a preview, as it relies on *starred-argument* forms to collect and later unpack arbitrary positional and keyword arguments meant for `open`—a topic coming up in the next chapter. Much of the magic here, though, is scope closures: the custom `open` found by the LEGB rule retains the original:

```
>>> file = '../Chapter14/data.txt'
>>> F = open(file)                # Call built-in open in builtins
>>> F.read()
'Testing file IO\nLearning Python, 6E\nPython 3.12\n'

>>> from makeopen import makeopen    # Import open resetter function
>>> makeopen('MOD1')                # Custom open calls built-in open

>>> F = open(file)                # Call custom open in builtins
Custom open call MOD1 ('../Chapter14/data.txt',) {}
>>> F.read()
'Testing file IO\nLearning Python, 6E\nPython 3.12\n'
```

Because each customization remembers the former built-in scope version in its own enclosing scope, they can even be *nested* naturally in ways that global variables cannot support—each call to the `makeopen` closure function remembers its own versions of `id` and `original`, so multiple customizations may be run:

```
>>> makeopen('MOD2')                # Nested customizers work too
>>> F = open(file)                # Because each retains its own state
Custom open call MOD2 ('../Chapter14/data.txt',) {}
Custom open call MOD1 ('../Chapter14/data.txt',) {}
>>> F.read()
'Testing file IO\nLearning Python, 6E\nPython 3.12\n'
```

As is, our function simply adds possibly nested call tracing to a built-in function, but the general technique may have other applications. A class-based equivalent to this may require more code because it would need to

save the `id` and `original` values explicitly in object attributes—but requires more background knowledge than we yet have; stay tuned for state retention in classes in this book’s [Part VI](#).

-
- ¹ The scope lookup rule was branded the “LGB rule” in the first edition of this book. The enclosing “E” layer was added later in Python to obviate the task of passing in enclosing scope names explicitly with default arguments—an advanced topic that we’ll sample later in this chapter. Since this scope is now addressed by the `nonlocal` statement, the lookup rule might have been better named “LNGB,” but backward compatibility matters in books, too. The present form of this acronym also does not account for the newer obscure scopes of comprehensions and exception handlers, but acronyms longer than four letters tend to defeat their purpose!
 - ² *Multithreading* runs function calls in parallel with the rest of a program and is supported by Python’s standard-library modules `_thread`, `threading`, and `queue`. Because all threaded functions run in the same process, global scopes often serve as one form of shared memory between them (threads may share both names in global scopes, as well as objects in a process’s memory space). Threading is commonly used for long-running tasks in GUIs, to implement nonblocking IO, and to utilize CPU capacity. Threading is also well beyond this language book’s scope (a property it shares with `async` functions, which are nevertheless part of Python syntax today, as you’ll learn in [Chapter 20](#)). See Python’s library manual for more details.

Chapter 18. Arguments

The preceding chapter explored Python’s *scopes*—the places where variables are defined and looked up. As we saw, the place where a name is defined in our code determines much of its meaning. This chapter continues the function story by studying the concepts in Python *argument passing*—the way that objects are sent to functions as inputs. As you’ll see, arguments (a.k.a. parameters) are assigned to names in a function, but have more to do with object references than with variable scopes. You’ll also find that Python provides extra tools, such as keywords, defaults, and argument collectors and extractors, that allow arguments to be sent to functions flexibly.

Argument-Passing Basics

Earlier in this part of the book, we learned that `def` and `lambda` are function *definitions*, and both include argument-list *headers* that name variables which receive values passed by *calls*. These arguments are used in function bodies, and may be matched between call and header by position, name, and other means we’ll explore later in this chapter.

More fundamentally, though, it was also noted that all Python arguments are passed by *assignment*—which means *object reference*. This has some subtle ramifications that aren’t always obvious to newcomers. Let’s start our arguments adventure by exploring how this works. Here is a rundown of the key points in this model:

- **Arguments are passed by automatically assigning objects to local variable names.** Function arguments are just another instance of Python assignment at work: they are *references* to objects sent by, and possibly shared with, the caller. Because references are implemented as pointers, all arguments are passed by opaque pointer. As for all assignments, objects passed as arguments are never automatically copied.

- **Assigning to argument names inside a function does not affect the caller.** Per assignment norms, when the function is run by a call, argument names in the function header simply become new names in the *local* scope of the function. There is no aliasing between function argument names and variable names in the caller's scope.
- **Changing a mutable object argument in a function may impact the caller.** On the other hand, as arguments are simply references to passed-in objects, functions can change passed-in mutable objects in place, and the results may affect the caller. Hence, *mutable* arguments can be both input and output for functions.

For more details on *references*, see [Chapter 6](#); everything we studied there also applies to function arguments, though the assignment to argument names is automatic and implicit.

Python's pass-by-assignment scheme isn't quite the same as C++'s reference parameters option, but it turns out to be similar to the argument-passing model of the C language (and others) in practice. You don't need to know these languages to use Python, of course, but the comparison might help those with backgrounds in other tools:

- **Immutable arguments have the same effect as passing “by value.”** Objects such as integers and strings are passed by object reference instead of by copying, but because you can never *change* immutable objects in place anyhow, the net result is much like making a copy in other languages: the caller's values never morph.
- **Mutable arguments have the same effect as passing “by pointer.”** Objects such as lists and dictionaries are also passed by object reference, which has similar consequences to passing arrays as pointers in C: mutable objects can be changed *in place* within the function, with side effects like those for C's arrays.

If you've never used C, Python's argument-passing mode will seem simpler still—it involves just the assignment of object references to names, and it works the same whether the objects are mutable or not.

Arguments and Shared References

To demo argument-passing properties at work, consider the following code:

```
>>> def f(a):           # a is assigned a reference to the passed object
    a = 99              # Changes local variable a only

>>> b = 88
>>> f(b)               # a and b both reference same 88 initially
>>> b                  # But b is not changed by assignment to a in f
88
```

In this example, the variable `a` is assigned the object 88 at the moment the function is called with `f(b)`, but `a` lives only within the called function's local scope. Changing `a` inside the function has no effect on the place where the function is called; it simply resets the local variable `a` to a completely different object, 99.

That's what is meant by a lack of name *aliasing*—assignment to an argument name inside a function (e.g., `a=99`) does not magically change a variable like `b` in the scope of the function call. Argument names may share passed *objects* initially (they are essentially pointers to those objects), but only temporarily, when the function is first called. As soon as an argument name is reassigned, this relationship ends.

At least, that's the case for assignment to argument *names* themselves. When arguments are passed *mutable* objects like lists and dictionaries, we also need to be aware that in-place changes to such *objects* may live on after a function exits, and hence impact callers. Here's an example that demonstrates this behavior:

```
>>> def changer(a, b):  # Arguments assigned references to objects
    a = 2               # Changes local name's value only
    b[0] = 'mod'        # Changes shared object in place: outlives call

>>> X = 1
>>> L = [1, 2]          # Caller:
>>> changer(X, L)       # Pass immutable and mutable objects
>>> X, L               # X is unchanged, but L is different!
(1, ['mod', 2])
```

In this code, the `changer` function assigns values to argument `a` itself, and to a

component of the *object* referenced by argument *b*. These two assignments within the function are only slightly different in syntax but have radically different results:

- Because *a* is a local variable name in the function's scope, the first assignment has no effect on the caller—it simply changes the local variable *a* to reference a completely different object, and does not change the value of the name *X* in the caller's scope. This is the same as in the prior example.
- Argument *b* is a local variable name, too, but it is passed a mutable object—the list that *L* also references in the caller's scope. Because the assignment to *b*[0] in the function is an in-place change to a shared object, its result impacts the value of *L* after the function returns.

Really, the second assignment statement in *changer* doesn't change *b*—it changes part of the object that *b* currently references. This in-place change impacts the caller only because the changed object outlives the function call. The name *L* hasn't changed either—it still references the same, changed object—but it seems as though *L* differs after the call because the value it references has been modified within the function. In effect, the list name *L* serves as both *input* to and *output* from the function.

Figure 18-1 illustrates the name/object bindings that exist immediately after the function has been called, and before its code has run. When the call begins, two objects are shared among four names.

If this example is still confusing, it may help to notice that the effect of the automatic assignments of the passed-in arguments is the same as running a series of simple assignment statements. In terms of the *first* argument, the assignment has no effect on the caller:

```
>>> X = 1
>>> a = X           # They share the same object
>>> a = 2           # Name change resets 'a' only, 'X' is still 1
>>> X
1
```

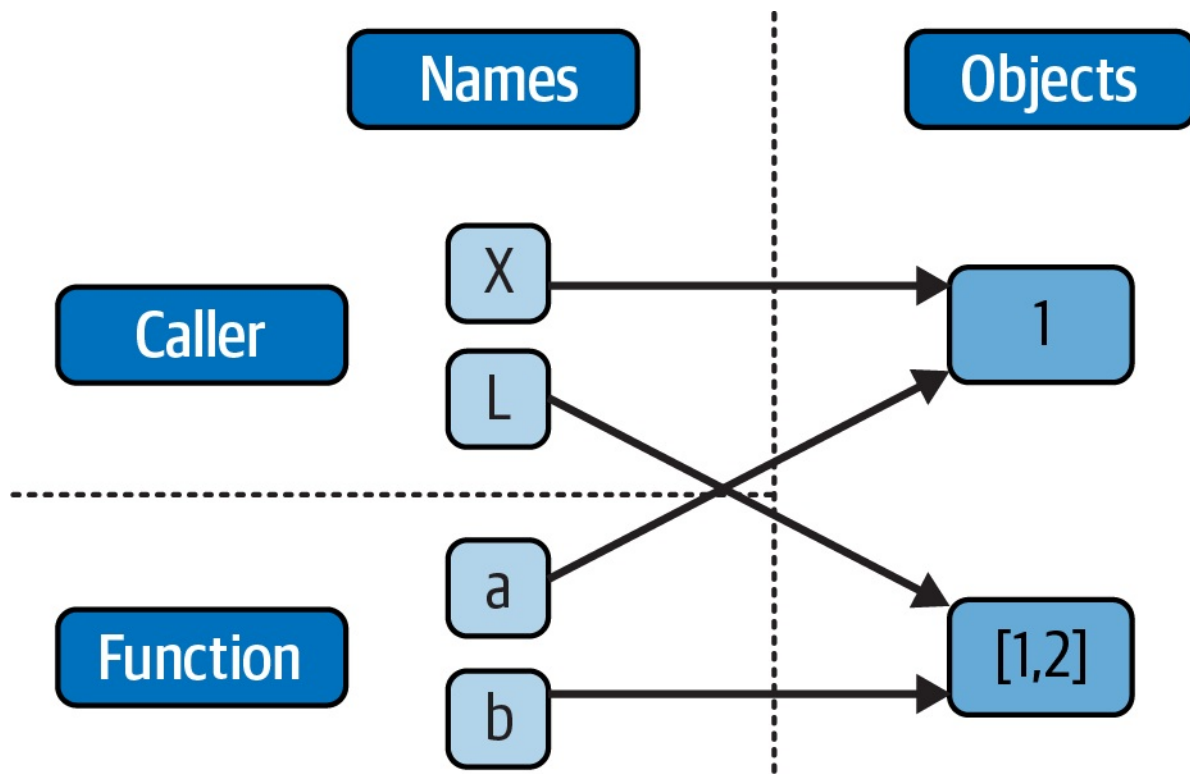


Figure 18-1. Function arguments and shared object references

The assignment through the *second* argument does affect a variable at the call, though, because it is an in-place object change:

```
>>> L = [1, 2]
>>> b = L           # They share the same object
>>> b[0] = 'mod'    # In-place change from 'b': 'L' sees the change too
>>> L
['mod', 2]
```

If you recall our discussions about shared mutable objects in Chapters 6 and 9, you'll recognize the phenomenon at work: changing a mutable object in place can impact other references to that object. Here, the effect is to make one of the arguments work like both an input and an output of the function.

Avoiding Mutable Argument Changes

This behavior of in-place changes to mutable arguments isn't a bug—it's simply the way argument passing works in Python, and turns out to be widely useful in practice. Arguments are normally passed to functions by reference because that is what we normally want. It means we can pass large objects around our

programs without making multiple copies along the way, and we can easily update these objects as we go. In fact, as you’ll see in [Part VI](#), Python’s `class` and OOP model *depends* upon changing a passed-in “self” argument in place, to update mutable object state.

If we don’t want in-place changes within functions to impact objects we pass to them, though, we can simply make explicit copies of mutable objects, as we saw in [Chapter 6](#). For function arguments, we can always copy the list at the point of *call* with tools like `list`, `list.copy`, or an empty slice (dictionaries have similar copy tools):

```
L = [1, 2]
changer(X, L[:])          # Pass a copy, so our 'L' does not change
```

We can also copy within the *function* itself, if we never want to change passed-in objects, regardless of how the function is called:

```
def changer(a, b):
    b = b.copy()          # Copy input list so we don't impact caller
    a = 2
    b[0] = 'mod'          # Changes our list copy only
```

Both of these copying schemes don’t stop the function from changing the object—they just prevent those changes from impacting the caller. To really prevent changes, we can always convert to *immutable* objects to force the issue. Tuples, for example, raise an exception when changes are attempted:

```
L = [1, 2]
changer(X, tuple(L))      # Pass a tuple, so changes are errors
TypeError: 'tuple' object does not support item assignment
```

This scheme uses the built-in `tuple` function, which builds a new tuple out of all the items in a sequence (really, any iterable). It’s also something of an extreme—because it forces the function to be written to never change passed-in arguments, this solution might impose more limitations on the function than it should, and so should generally be avoided (you never know when changing arguments might come in handy for other calls in the future). Using this technique will also make the function lose the ability to call any list-specific methods on the

argument, including methods that do not change the object in place (e.g. `copy`, though tuples have adopted list `count` and `index`).

The main point to remember here is that functions might update mutable objects like lists and dictionaries passed into them. This isn't necessarily a problem if it's expected, and often serves useful purposes. Moreover, functions that change passed-in mutable objects in place are probably designed and *intended* to do so—the change is likely part of a well-defined API that you shouldn't violate by making copies.

However, you do have to be aware of this property—if objects change out from under you unexpectedly, check whether a called function might be responsible, and make copies when objects are passed if needed.

Simulating Output Parameters and Multiple Results

Here's another function topic from the assignments department. We've already discussed the `return` statement and used it in examples. What we haven't yet seen is a common though unusual coding technique it enables: because `return` can send back any sort of object, it can return *multiple values* by packaging them in a tuple or other collection type. In fact, although Python doesn't support what some languages label “call by reference” argument passing, we can simulate it by returning tuples and assigning the results back to the original argument names in the caller:

```
>>> def multiple(x, y):
    x = 2                # Changes local names only
    y = [3, 4]
    return x, y          # Return multiple new values in a tuple

>>> X = 1
>>> L = [1, 2]
>>> X, L = multiple(X, L) # Assign results to caller's names
>>> X, L
(2, [3, 4])
```

It looks like the code is returning two values here, but it's really just one—a two-item *tuple* with the optional surrounding parentheses omitted. After the call returns, we can use tuple (a.k.a. sequence) assignment to unpack the parts of the returned tuple. (If you've forgotten why this works, flip back to “Tuples” in

Chapters 4 and 9, and “Assignments” in Chapter 11.) The net effect of this coding pattern is to both send back multiple results and simulate the *output parameters* of other languages by explicit assignments. Here, X and L change after the call—but only because the code said so. Lists would work, too, but tuples sans `()` are less to type, and hence common.

Special Argument-Matching Modes

As we’ve just seen, arguments are always passed by *assignment* in Python; names in a `def` or `lambda` definition’s header are assigned references to passed-in objects. On top of this model, though, Python provides additional tools that alter the way the argument objects in a call are *matched* with argument names in the definition prior to assignment. These tools are all optional, but allow us to code functions that support more flexible calling patterns and are commonly used by libraries you’re likely to encounter.

By default, arguments are matched between call and definition by *position*, from left to right, and you must pass exactly as many arguments as there are argument names in the function definition. However, you can also specify matching by name, provide default values, unpack and collect arbitrarily many arguments, and even specify passing-mode requirements. This section presents all these extra tools with a quick overview followed by examples, and a formal look at how they interact at the end after we’ve had a chance to see the basics.

Argument Matching Overview

Before we get into syntax details, it’s important to stress that these special modes are optional and deal only with matching objects to names; the underlying passing mechanism after the matching takes place is still assignment. In fact, some of these tools are intended more for people writing libraries than for application developers. That said, you may stumble across these modes even if you don’t code them yourself, so the following summarizes all the options:

Positionals: matched from left to right

The normal case, which we’ve mostly been using so far, is to match

argument values (passed in a call) to argument names (listed in a function definition) by position, from left to right.

Keywords: matched by argument name

Alternatively, callers can explicitly specify which argument in the function is to receive a value, by giving the definition's name for the argument with *name=value* syntax.

Defaults: specify values for optional arguments that aren't passed

Functions themselves can specify default values for arguments to receive if the call passes too few values, again using the *name=value* syntax.

Starred collectors: collect arbitrarily many positional or keyword arguments

Function definitions can use special arguments preceded with one or two * characters to collect an arbitrary number of arguments after other matching. This feature is sometimes referred to as *varargs*, after a variable-length argument list tool in the C language; in Python, the arguments are collected in a normal object.

Starred unpackers: pass arbitrarily many positional or keyword arguments

Function calls can also use the one or two * syntax to unpack argument collections into separate arguments. This is the inverse of a * in a function definition—in the definition it means collect arbitrarily many arguments, while in the call it means unpack arbitrarily many arguments, and pass them individually as discrete values.

Keyword-only arguments: arguments that must be passed by name

Function definitions can also use a * in their headers to specify arguments that must be passed by name as keyword arguments, not position. This is

often used for configuration options that augment primary arguments.

Positional-only arguments: arguments that must be passed by position

As of Python 3.8, function definitions may additionally use a `/` in their header's arguments list to specify that all arguments preceding it must be passed by position, not keyword-argument name.

Argument Matching Syntax

As a reference and preview, [Table 18-1](#) summarizes the syntax that invokes the argument-matching modes.

Table 18-1. Function argument-matching forms

Syntax	Location	Interpretation
<code>func(value)</code>	Caller	Normal argument: matched by position
<code>func(name=value)</code>	Caller	Keyword argument: matched by name
<code>func(*iterable)</code>	Caller	Pass all objects in <i>iterable</i> as individual positional arguments
<code>func(**dict)</code>	Caller	Pass all key/value pairs in <i>dict</i> as individual keyword arguments
<code>def func(name)</code>	Function	Normal argument: matches any passed value by position or name
<code>def func(name=value)</code>	Function	Default argument value, if not passed in the call
<code>def func(*name)</code>	Function	Matches and collects remaining positional arguments in a tuple
<code>def func(**name)</code>	Function	Matches and collects remaining keyword arguments in a dictionary

<code>def func(*name, name)</code>	Function	Arguments that must be passed by keyword only in calls
<code>def func(*, name)</code>	Function	Arguments that must be passed by keyword only in calls
<code>def func(name, /)</code>	Function	Arguments that must be passed by position only in calls

The argument-matching modes listed in this table break down between function *calls* and *definitions* as follows:

In a function call (the first four rows of the table)

Simple values are matched to definition arguments by position, but using the *name=value* form tells Python to match arguments by name instead; these are called *keyword arguments*. Any number of **iterable* or ***dict* forms can be used in a call, allowing us to bundle many positional or keyword objects in iterables and mappings, respectively, and unpack them as separate, individual arguments when they are passed to the function.

In a function definition (the rest of the table)

A simple *name* is matched by position or name depending on how the caller passes it, but the *name=value* form specifies a *default value*. The **name* form collects any extra unmatched positional arguments in a tuple, and ***name* collects unmatched keyword arguments in a dictionary. In addition, any normal or defaulted argument names following a **name* or a bare *** are *keyword-only* arguments and must be passed by keyword in calls, and arguments preceding a */* are *positional-only* arguments that must *not* be passed by keyword name.

Of these, keyword arguments and defaults are probably the most commonly used in Python code. We've informally used both of these earlier in this book:

- We've already used *keywords* to specify options to the `print` function, but they are more general—keywords allow us to label any argument with its name, to make calls more explicit and informational.
- We met *defaults* earlier, too, as a way to pass in values from the enclosing function's scope, but they are also more general—they allow us to make any argument optional, providing its default value in a function definition.

As you'll see ahead, the combination of defaults in a function definition and keywords in a call further allows us to pick and choose which defaults to override per call.

In short, special argument-matching modes let you be fairly liberal about how many arguments must be passed to a function. If a function specifies defaults, they are used if you pass *too few* arguments. If a function uses the `*` argument-collector forms, you can seemingly pass *too many* arguments; the `*` names collect the extra arguments in data structures for processing within the function.

Argument Passing Details

If you choose to use and combine the special argument-matching modes, Python will ask you to follow some ordering rules among the modes' optional components. We're going to defer full and formal coverage of these until we've had a chance to observe these modes in action, but as some initial tips:

- In a function *call*, any number of positionals, keywords, and starred unpackings can be used, but positional arguments must precede keyword arguments and `**dict` unpackings, and `*iterable` unpackings must precede `**dict` unpackings.
- In a function *definition*, arguments must appear in this order: any positional-only arguments; followed by any positional-or-keyword arguments; followed by the optional `*name` positional collector or `*` and any keyword-only arguments; followed by the optional `**name`

keyword collector. Arguments can have optional defaults (*name=value*), but once a default is used, all arguments must use defaults up to a *, after which keyword-only arguments allow defaults and nondefaults to be freely mixed.

If you mix arguments in any other order, you will get a syntax error because the combinations can be ambiguous. The steps that Python internally carries out to match arguments before assignment can roughly be described as follows:

1. Unpack all **args* at the call into nonkeyword arguments.
2. Unpack all ***args* at the call into keyword arguments.
3. Assign nonkeyword arguments by position.
4. Assign keyword arguments by matching names.
5. Collect extra nonkeyword arguments in the **name* tuple.
6. Collect extra keyword arguments in the ***name* dictionary.
7. Assign default values to unassigned arguments.

After arguments in call and definition are matched, Python checks to make sure each argument is passed just one value (or else an error is raised) and then assigns argument names to the objects passed to them and runs the function body.

The actual matching algorithm Python uses is a bit more complex, so we'll defer to Python's standard language manual for a more exact description. It's not required reading, but tracing Python's matching algorithm may help you to understand some convoluted cases, especially when modes are mixed.

We'll return to the ordering rules in function calls in definitions with higher fidelity after we've had a chance to meet all the players. Let's get started in the next section with the most common of the bunch.

NOTE

But annotations are moot: Argument names in a `def` statement (only) can also have *annotation* values, specified as `name:annot`, or `name:annot=default` when defaults are present. This is

simply additional syntax for arguments and does not augment or change argument-ordering rules. The function itself can also have an annotation value, given as `def f(...)->annot`, and Python attaches all annotation values to the function object. See the discussion of function annotation in [Chapter 19](#) for more details, and the section on their role in unused type hinting in [Chapter 6](#).

Keyword and Default Examples

Argument passing is simpler in code than the preceding descriptions may imply. First off, if you don't use any special matching syntax, Python matches names by *position* from left to right. For instance, if you define a function that requires three arguments, you must call it with three arguments:

```
>>> def f(a, b, c): print(a, b, c)

>>> f(1, 2, 3)
1 2 3
```

Here, we pass by position—a is matched to 1, b is matched to 2, and so on. This works like it does in most other programming languages.

Keywords

In Python, though, you can be more specific about what goes where when you call a function. Keyword arguments allow us to match by *name*, instead of by position. Using the same function definition but a different call:

```
>>> f(c=3, b=2, a=1)
1 2 3
```

The `c=3` in this call, for example, means send 3 to the argument named `c`. More formally, Python matches the name `c` in the call to the argument named `c` in the function definition, and then assigns the value 3 to that argument. The net effect of this call is the same as that of the prior call, but notice that the left-to-right order of the arguments no longer matters when keywords are used because arguments are matched by name, not by position.

It's even possible to combine positional and keyword arguments in a single call. In this case, all positionals are matched first from left to right in the definition,

before keywords are matched by name:

```
>>> f(1, c=3, b=2)           # a gets 1 by position, b and c passed by name
1 2 3
```

When most people see this the first time, they wonder why one would use such a tool. Keywords typically have two roles in Python. First, they make your calls a bit more self-documenting (assuming that you use better argument names than *a*, *b*, and *c*!). For example, a call of this form:

```
func(title='Learning Python', edition=6, year=2024, python=3.12)
```

is much more meaningful than a call with three naked values separated by commas, especially in larger programs—the keywords serve as labels for the data in the call:

```
func(title='Learning Python', 6, 2024, 3.12)
```

The second major use of keywords occurs in conjunction with defaults, which we turn to next.

Defaults

We talked about defaults in brief earlier, when discussing nested function scopes. In short, defaults allow us to make selected function arguments *optional*; if not passed a value, the argument is assigned its default before the function runs. For example, here is a function that requires one argument and defaults two others:

```
>>> def f(a, b=2, c=3): print(a, b, c)           # a required, b and c optional
```

As noted earlier, defaults must appear after nondefaults at this point in a header (they can be mixed after a *** as you'll see ahead). When we call this function, we must provide a value for *a*, either by position or by keyword; however, providing values for *b* and *c* is optional. If we don't pass values to *b* and *c*, they default to 2 and 3, respectively:

```
>>> f(1)           # Use defaults
1 2 3
```

```
>>> f(a=1)
1 2 3
```

If we pass two values, only `c` gets its default, and when passing three values, no defaults are used:

```
>>> f(1, 4)                # Override defaults
1 4 3
>>> f(1, 4, 5)
1 4 5
```

Finally, here is how the keyword and default features interact. Because they subvert the normal left-to-right positional mapping, keywords allow us to essentially skip over arguments with defaults:

```
>>> f(1, c=6)              # Choose defaults: b in the middle
1 2 6
```

Here, `a` gets 1 by position, `c` gets 6 by keyword, and `b`, in between, defaults to 2.

Be careful not to confuse the special *name=value* syntax in a function definition and a function call; in the *call* it means a match-by-name keyword argument, while in the *definition* it specifies a default for an optional argument. In both cases, this is not an assignment statement (despite its appearance); it is special syntax for these two contexts, which modifies the default argument-matching mechanics.

Combining keywords and defaults

Here is a slightly larger example that demonstrates keywords and defaults in action. In the following, the caller must always pass at least two arguments (to match `code` and `hack`), but the other two are optional. If they are omitted, Python assigns `script` and `app` to the defaults specified in the definition:

```
def func(code, hack, script=0, app=0):    # First 2 required
    print((code, hack, script, app))

func(1, 2)                               # Output: (1, 2, 0, 0)
func(1, app=1, hack=0)                   # Output: (1, 0, 0, 1)
func(code=1, hack=0)                     # Output: (1, 0, 0, 0)
func(script=1, hack=2, code=3)           # Output: (3, 2, 1, 0)
```

```
func(1, 2, 3, 4)
```

```
# Output: (1, 2, 3, 4)
```

Notice again that when keyword arguments are used in the call, the order in which the arguments are listed doesn't matter; Python matches by name, not by position. The caller must supply values for `code` and `hack`, but they can be given by position or by name.

Also keep in mind that all the special definition-side argument-matching syntax we're exploring in this chapter works the same in `def` and `lambda`, though most examples use the former, and the latter returns results implicitly:

```
>>> func = lambda code, hack, script=0, app=0: (code, hack, script, app)
>>> func(1, 2)
(1, 2, 0, 0)
>>> func(script=1, hack=2, code=3)
(3, 2, 1, 0)
```

NOTE

Beware mutable defaults: As noted in the prior chapter, if you code a default to be a mutable object (e.g., `def f(a=[])`), the same, *single* mutable object is reused every time the function is later called—even if it is changed in place within the function. The net effect is that the argument's default retains its value from the prior call and is not reset to its original value coded in the function header. To reset anew on each call, move the assignment into the function body instead. Mutable defaults allow state retention, but this is often an unpleasant surprise. Since this is such a common trap, we'll postpone further exploration until this part's “gotchas” list at the end of [Chapter 21](#).

Arbitrary Arguments Examples

The last two argument-matching extensions, `*` and `**`, are designed to support *any number* of arguments. Both can appear in either the function definition or a function call, and they have related purposes in the two locations.

Definitions: Collecting arguments

The first use, in the function definition, collects unmatched *positional* arguments into a tuple:

```
>>> def f(*args): print(args)
```

When this function is called, Python *collects* all the positional arguments into a new *tuple* and assigns the variable `args` to that tuple. Because it is a normal tuple object, it can be indexed, stepped through with a `for` loop, and so on:

```
>>> f()
()
>>> f(1)
(1,)
>>> f(1, 2, 3, 4)
(1, 2, 3, 4)
```

The `**` feature is similar, but it only works for *keyword* arguments—it collects them into a new *dictionary*, which can then be processed with normal dictionary tools. In a sense, the `**` form allows you to convert from keywords to dictionaries, which you can then step through with `keys` calls, dictionary iterators, and the like (this is roughly what the `dict` call does when passed keywords, but it *returns* the new dictionary):

```
>>> def f(**args): print(args)

>>> f()
{}
>>> f(a=1, b=2)
{'a': 1, 'b': 2}
```

Finally, function definitions can combine normal arguments, the `*`, and the `**` to implement wildly flexible call signatures. For instance, in the following, `1` is passed to `a` by position, `2` and `3` are collected into the `pargs` positional tuple, and `x` and `y` wind up in the `kargs` keyword dictionary:

```
>>> def f(a, *pargs, **kargs): print(a, pargs, kargs)

>>> f(1, 2, 3, x=1, y=2)
1 (2, 3) {'x': 1, 'y': 2}
```

Notice that the dictionary’s keys are ordered here: as of Python 3.6, the `**` collector’s keys preserve the order in which keyword arguments were passed to the function. This relies on the insertion order of keys in dictionaries at large (see [Chapter 8](#) for a refresher if you’ve forgotten what that means).

Functions with both `*` and `**` may be rare, but they show up in functions that need to support multiple call patterns (for backward compatibility, for instance). In fact, these features can be combined in even more complex ways that may seem ambiguous at first glance—an idea we will revisit later in this chapter when we lock down ordering rules. First, though, let's see what happens when `*` and `**` are coded in function *calls* instead of definitions.

Calls: Unpacking arguments

It turns out that we can use the `*` syntax when we call a function, too. In this context, its meaning is the inverse of its meaning in the function definition—it *unpacks* a collection of arguments, rather than building a collection of arguments. For example, we can pass four arguments to a function in a tuple or other iterable, and let Python unpack them into individual positional arguments:

```
>>> def func(a, b, c, d): print(a, b, c, d)

>>> args = (1, 2)
>>> args += (3, 4)
>>> func(*args)                # Same as func(1, 2, 3, 4)
1 2 3 4
```

Similarly, the `**` syntax in a function call unpacks a dictionary or other mapping of key/value pairs into separate keyword arguments:

```
>>> args = {'a': 1, 'b': 2, 'c': 3}
>>> args['d'] = 4
>>> func(**args)               # Same as func(a=1, b=2, c=3, d=4)
1 2 3 4
```

Moreover, we can use other iterables like lists and other dictionary syntax like `dict`, and we may combine star, positional, and keyword arguments in the call in very flexible ways:

```
>>> func(*(1, 2), **{'d': 4, 'c': 3})    # Same as func(1, 2, d=4, c=3)
1 2 3 4
>>> func(1, *[2, 3], **dict(d=4))        # Same as func(1, 2, 3, d=4)
1 2 3 4
>>> func(1, c=3, *[2], **{'d': 4})       # Same as func(1, 2, c=3, d=4)
1 2 3 4
>>> func(1, *(2, 3), d=4)                # Same as func(1, 2, 3, d=4)
```

```

1 2 3 4
>>> func(1, *[2], c=3, **dict(d=4))      # Same as func(1, 2, c=3, d=4)
1 2 3 4

```

While the preceding serves to demo the possibilities, its use of stars and literals is overkill when arguments are known; more typical code would build up argument collections ahead of the call and unpack by names:

```

>>> pargs, kargs = (1, 2), dict(d=4, c=3)
>>> func(*pargs, **kargs)
1 2 3 4

```

Star unpacking is convenient when you cannot predict the number of arguments that will be passed to a function when you write your script; you can build up a collection of arguments at runtime instead and call the function generically this way. Here again, though, don't confuse the `*/**` starred-argument syntax in the function definition and the function call—in the *definition* it collects any number of arguments, while in the *call* it unpacks any number of arguments. In both, one star means positionals, and two applies to keywords.

Finally, as of Python 3.5, and as noted in [Chapter 11](#)'s sidebar “[The Many Stars of Python](#)”, we can even use *multiple* `*` and `**` items in calls to unpack multiple iterables and mappings, respectively. The `*` unpacks into positional arguments, and `**` into keyword arguments, though single stars must precede double stars, per formal rules coming up ahead. Again, the following uses literals to demo, but these would usually be names assigned to prebuilt values:

```

>>> def func(a, b, c, d): print(a, b, c, d)

>>> func(*[1], *(2,), **dict(c=3), **{'d': 4})      # *Positionals + **keywords
1 2 3 4
>>> func(*[1], *(2,), *[3, 4])                    # All *positionals
1 2 3 4
>>> func(**dict(a=1, b=2), **dict(c=3), **{'d': 4}) # All **keywords
1 2 3 4
>>> func(*[1], 2, **dict(c=3), d=4)                # All call modes at once!
1 2 3 4

>>> func(*[1], **dict(b=2, c=3), *[4])
SyntaxError: iterable argument unpacking follows keyword argument unpacking

```

NOTE

Unpacking generality—and inconsistency: As previewed in [Chapter 14](#), the `*` form in a call is an *iteration tool*, so it accepts any iterable object, not just tuples or other sequences used in examples here. For instance, iterables like `zip` and `range` work after a `*` too and unpack into individual arguments, and file objects automatically read and unpack their lines:

```
func(*range(4))           # Same as func(0, 1, 2, 3)
func(*open('filename'))   # Read+pass lines as arguments
```

Watch for more examples of this utility in [Chapter 20](#), after we study generators.

On the downside, stars also come with *inconsistencies*. For one, this generality holds true only for *calls*—a `*` unpacking in a call allows any iterable, but a `*` in a function *definition* always collects extra arguments into a *tuple*. Moreover, this collection behavior in definitions is similar in spirit and syntax to the `*` in the extended-unpacking *assignment* forms we met in [Chapter 11](#) (e.g., `x, *y = z`), but that star usage always creates *lists*, not tuples. Again: these are different rules for different tools—despite the same syntax.

Why arbitrary arguments?

The prior section’s examples may seem academic (if not downright esoteric), but they are used more often than you might expect. Some programs need to call arbitrary functions in a generic fashion, without hardcoding their names or arguments ahead of time. In fact, the real power of the special starred-unpacking call syntax is that you don’t need to know how many arguments a function call requires before you write a script. For example, you can use `if` logic to select from a set of functions and argument lists, and call any of them generically (functions here are hypothetical):

```
if sometest:
    action, args = func1, (1,)           # Call func1 with one arg in this case
else:
    action, args = func2, (1, 2, 3)       # Call func2 with three args instead
...
action(*args)                           # Dispatch generically
```

This leverages both the `*` form and the fact that functions are objects that may be both referenced by, and called through, any variable. More generally, this unpacking call syntax is useful anytime you cannot predict the arguments list. If your user selects an arbitrary function via a user interface, for instance, you may

be unable to hardcode a function call when writing your script. To work around this, simply build up the arguments list with sequence operations, and call it with starred-argument syntax to unpack the arguments:

```
args = (2,3)
args += (4,)
...
func3(*args)
```

Because the arguments list is passed in as a tuple here, the program can build it at runtime. This technique also comes in handy for functions that test or time other functions. For instance, the code in [Example 18-1](#) supports any function with any arguments by passing along whatever arguments were sent in (see *tracer0.py* in the example package).

Example 18-1. tracer0.py

```
def tracer(func, *pargs, **kargs):           # Accept arbitrary arguments
    print('calling:', func.__name__)
    return func(*pargs, **kargs)           # Pass along arbitrary arguments

def func(a, b, c, d):
    return a + b + c + d

print(tracer(func, 1, 2, c=3, d=4))
```

This code uses the built-in `__name__` attribute attached to every function (as you might expect, it's the function's name string), and uses stars to collect and then unpack the arguments intended for the traced function. In other words, when this code is run, arguments are intercepted by the tracer and then *propagated* with unpacking call syntax:

```
$ python3 tracer0.py
calling: func
10
```

For another example of this technique, see the preview near the end of the preceding chapter, where it was used to reset the built-in `open` function (though it probably makes more sense to you now). We'll code additional examples of such roles later in this book; see especially the sequence timing examples in [Chapter 21](#) and the various *decorator* utilities we will code in [Chapter 39](#) (after a

preview in the next chapter). It's a common technique in general tools.

Keyword-Only Arguments

Besides accepting keyword (i.e., pass-by-name) arguments in general, function *definitions* can also specify that some arguments must always be passed by keyword only and will never be filled in by a positional argument. This extension, known as *keyword-only arguments*, can be useful if we want a function to both process any number of arguments and accept possibly optional configuration options.

Syntactically, keyword-only arguments are coded as named arguments that may appear after **name* in the arguments list. All such arguments must be passed using keyword syntax in the call. For example, in the following, *a* may be passed by name or position, *b* collects any extra positional arguments, and *c* must be passed by keyword only:

```
>>> def kwonly(a, *b, c): print(a, b, c)

>>> kwonly(1, 2, c=3)
1 (2,) 3
>>> kwonly(a=1, c=3)
1 () 3
>>> kwonly(1, 2, 3)
TypeError: kwonly() missing 1 required keyword-only argument: 'c'
```

If we don't need to collect arbitrary positionals, we can also use a bare *** character by itself in the arguments list to introduce keyword-only arguments. This indicates that a function does not accept a variable-length argument list but still expects all arguments following the *** to be passed as keywords. In the next function, *a* may be passed by position or name again, but *b* and *c* must be keywords, and no extra positionals are allowed:

```
>>> def kwonly(a, *, b, c): print(a, b, c)

>>> kwonly(1, c=3, b=2)
1 2 3
>>> kwonly(c=3, b=2, a=1)
1 2 3
>>> kwonly(1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: kwonly() takes 3 positional arguments but 4 were given
```

```
TypeError: kwonly() takes 1 positional argument but 3 were given
>>> kwonly(1)
TypeError: kwonly() missing 2 required keyword-only arguments: 'b' and 'c'
```

You can still use defaults for keyword-only arguments, even though they appear after the `*` in the function. In the following, `a` may be passed by name or position, and `b` and `c` are optional but must be passed by keyword if used:

```
>>> def kwonly(a, *, b='code', c='app'): print(a, b, c)

>>> kwonly(1)
1 code app
>>> kwonly(1, c='hack')
1 code hack
>>> kwonly(a='py')
py code app
>>> kwonly(c=3, b=2, a=1)
1 2 3
>>> kwonly(1, 2)
TypeError: kwonly() takes 1 positional argument but 2 were given
```

In fact, keyword-only arguments with defaults are optional, but those without defaults effectively become *required keywords* for the function—like `b` in the following:

```
>>> def kwonly(a, *, b, c='hack'): print(a, b, c)

>>> kwonly(1, b='code')
1 code hack
>>> kwonly(1, c='code')
TypeError: kwonly() missing 1 required keyword-only argument: 'b'
>>> kwonly(1, 2)
TypeError: kwonly() takes 1 positional argument but 2 were given
```

As noted earlier, keyword-only arguments also allow defaults and nondefaults to be *mixed*, unlike their otherwise more flexible cohorts coded before the optional `*`—an ostensible inconsistency we'll return to later:

```
>>> def kwonly(a, *, b=2, c, d=4): print(a, b, c, d)

>>> kwonly(1, c=3)
1 2 3 4
>>> kwonly(1, c=5, b=6)
```

```

1 6 5 4
>>> konly(1)
TypeError: konly() missing 1 required keyword-only argument: 'c'
>>> konly(1, 2, 3)
TypeError: konly() takes 1 positional argument but 3 were given

>>> def badmix(b=2, c, d=5): ...
SyntaxError: parameter without a default follows parameter with a default

```

Finally, note that keyword-only arguments must be specified after a single star, not two—nothing can appear after the `**args` arbitrary-keywords form, and, unlike `*`, a `**` can't appear by itself in the arguments list. More generally, keyword-only arguments must be coded *between* the `*` and the optional `**`, and an argument that appears before `*` is a possibly default argument that can be passed by position or keyword, not keyword-only:

```

>>> def konly(a, **kargs, b, c):
SyntaxError: arguments cannot follow var-keyword argument
>>> def konly(a, **, b, c):
SyntaxError: invalid syntax
>>> def mixed(a, *b, **d, c=6):
SyntaxError: arguments cannot follow var-keyword argument

```

These failures will make more sense after we get formal about argument ordering rules later in this chapter. They may also appear to be worst cases in the artificial examples here, but they can come up in real practice, especially for people who write libraries for other Python programmers to use—which leads to the next point.

Why keyword-only arguments?

So why care about keyword-only arguments? In short, they make it easier to allow a function to accept both any number of positional arguments to be processed, and configuration options passed as keywords. While their use is optional, without keyword-only arguments extra work may be required to provide defaults for such options and to verify that no superfluous keywords were passed.

Imagine a function that processes a set of passed-in objects and allows a tracing flag to be passed:

```
process(X, Y, Z)                # Use flag's default
process(X, Y, notify=True)      # Override flag default
```

Without keyword-only arguments we have to use both `*args` and `**args` and manually inspect the keywords, but with keyword-only arguments less code is required. The following guarantees that no positional argument will be incorrectly matched against `notify` and requires that it be a keyword if passed:

```
def process(*args, notify=False): ...
```

Since we’re going to explore a more realistic example of this later in this chapter, in [“Example: Rolling Your Own Print”](#), we’ll postpone the rest of this story until then. For an additional example of keyword-only arguments in action, see the upcoming iteration-options timing case study in [“Timer Module: Take 2”](#).

Positional-Only Arguments

Beginning with Python 3.8, function *definitions* may also include a `/` in the arguments list to designate that all arguments preceding it (i.e., to its left) must be passed by *position*, not by keyword-argument name. Though arguably ad hoc on first sighting in an arguments list, this notation was being used in documentation for built-in functions that did not accept keywords; making it available to function coders as part of Python’s syntax was deemed a logical extension for library developers who may not want to expose argument names for use by clients as keywords.

To demo, the following function specifies that `a` and `b` must be passed by position, though `c` is more flexible:

```
>>> def mostlypos(a, b, /, c): print(a, b, c)

>>> mostlypos(1, 2, 3)
1 2 3
>>> mostlypos(1, 2, c=3)
1 2 3
```

Passing either of the first two arguments by keyword, however, fails:

```
>>> mostlypos(1, b=2, c=3)
```

```
TypeError: mostlypos() got some positional-only arguments passed as keyword arguments: 'b'
```

```
>>> mostlypos(c=3, b=2, a=1)
TypeError: mostlypos() got some positional-only arguments passed as keyword arguments: 'a, b'
```

To define a function that allows *only* positional arguments, simply code the slash at the end:

```
>>> def allpos(a, b, c, /): print(a, b, c)

>>> allpos(1, 2, 3)
1 2 3
>>> allpos(1, 2, c=3)
TypeError: mostlypos() got some positional-only arguments passed as keyword ...
```

As you should expect, the slash works the same in a `lambda` argument list:

```
>>> f = lambda a, b, /, c: print(a, b, c)

>>> f(1, 2, c=3)
1 2 3
>>> f(1, b=2, c=3)
TypeError: <lambda>() got some positional-only arguments passed as keyword ...
```

And functions can *combine* positional- and keyword-only arguments to be as rigid as they wish:

```
>>> def combo(a, b, /, *, c, d): print(a, b, c, d)

>>> combo(1, 2, c=3, d=4)
1 2 3 4
>>> combo(1, 2, 3, 4)
TypeError: combo() takes 2 positional arguments but 4 were given
>>> combo(a=1, b=2, c=3, d=4)
TypeError: combo() got some positional-only arguments passed as keyword ...
```

It's up to you to ponder whether or not use cases for this syntax justify its convolution of function definitions that follows in the next section. Given that Python users somehow got by without it for over three decades, though, this seems a tough sell. For more on the rationale and usage of the positional-only slash, see Python's standard manuals. Also watch for an example in [Chapter 21](#)

that uses it to avoid name clashes—and may or may not be compelling.

Argument Ordering: The Gritty Details

So far, we've been fairly loose about the rules surrounding argument-matching tools, because they don't crop up very often in the simpler usage patterns of typical code. In more sophisticated roles, though, you need to verify that your code follows Python's expectations. Now that we've seen all of its subjects, the ordering rules for function arguments can finally be summarized in full. While function *definitions* and *calls* share some similar syntax, their rules are completely different, owing to their different roles. Let's take a more formal look at both.

Definition Ordering

In function *definitions*, argument lists are enclosed in parentheses in a `def` statement and coded before a colon in a `lambda` expression, but follow the same format in both. In short, they consist of four optional parts that must appear in the following order, where *position* means a simple value in calls, and *keyword* means a *name=value* pair:

1. One or more arguments that must be passed by *position* only, followed by a single `/`
2. Any number of arguments that can be passed by either *position* or *keyword*
3. A single `*` by itself or a single `*name` positional-argument *collector*, optionally followed by any number of arguments that must be passed by *keyword* only
4. A single `**name` keyword-argument *collector*

In all cases, individual arguments, including a bare `/` or `*`, are separated by commas. In addition, any nonstarred argument name can have a *name=expression* default, but all names must have defaults after the first that does, up to the `*` (keyword-only argument runs following a star may freely mix

default and nondefault names).

Inherent in this ordering, positional-only arguments must appear first, **name* ends positional-argument runs and collects unmatched positionals, and ***name* ends the entire arguments list and collects unmatched keywords. As noted earlier, the ***name* collector's keys retain the order in which keyword arguments were passed to the function.

Formal definition

More concisely, the ordering of arguments in function definitions can be defined as follows, where *-or-* is notation for a choice and `[ ]` encloses an optional part (neither is part of the actual code you type):

```
def name(arguments-list): statements
lambda arguments-list: expression

arguments-list =
    [positional-only-arguments, /]
    [positional-or-keyword-arguments]
    [* -or- *positional-collector, [keyword-only-arguments]]
    [**keyword-collector]
```

As a real example, the following defines a function with all these parts in actions:

```
>>> def f(a, /, b, c=3, *ps, e=4, **ks):
        print(f'{a=}, {b=}, {c=}, {ps=}, {e=}, {ks=}')

>>> f(0, 1, 2, 3, 4, e=5, f=6, g=7)
a=0, b=1, c=2, ps=(3, 4), e=5, ks={'f': 6, 'g': 7}
```

Boundary cases

As consequences of the ordering in function definitions, `/` must precede a star, and the double-star keyword collector must be coded last:

```
>>> def f(a, *ps, /, b): pass
SyntaxError: / must be ahead of *

>>> def f(a, **ks, b): pass
SyntaxError: arguments cannot follow var-keyword argument
```


Though enforced separately from basic argument-ordering rules, once a *default* is used in a definition, all subsequent arguments must also have defaults—including those following the positional-only / delimiter:

```
>>> def f(a, b, c=3, d, e): pass
SyntaxError: parameter without a default follows parameter with a default

>>> def f(a, b=2, /, c): pass
SyntaxError: parameter without a default follows parameter with a default
```

However, this constraint applies only to arguments preceding a *** or **name*—defaults and nondefaults *can* be mixed freely in keyword-only arguments, which seems inconsistent, though a case for sanity could be made here on the grounds that keyword-only arguments never match by position, which reduces ambiguity of defaults:

```
>>> def f(a, *, x=1, y): ...      # OK: x and y must be keywords

>>> def f(a=1, b, *, x=1): ...    # Not OK: a and b may be positional
SyntaxError: parameter without a default follows parameter with a default
```

Not shown here, `def` can also be preceded by a *decorator* (e.g., `@value`), and, as noted earlier, any of its arguments may be followed by *annotations* (e.g., `:value`)—extensions we’ll explore in the next chapter and later in this book. Neither of these impact argument ordering or matching, and neither work in `lambda` due to its limited syntax.

Calls Ordering

On the other side of the fence, the syntax is similar but the rules differ. In function *calls*, all positional arguments must precede all keyword arguments, and any number of starred *unpackings* can be mixed in with individual values: one star unpacks into multiple positional arguments, two unpacks into keywords, and all positional arguments and one-star unpackings must precede all two-star unpackings.

In more detail, function (and by extension, method) calls consist of three optional parts in the following order:

1. Any combination of one or more *expression* positional arguments, and **expression* iterable unpackings transformed into positional arguments
2. Any combination of one or more *name=expression* keyword arguments, and **expression* iterable unpackings transformed into positional arguments
3. Any combination of one or more *name=expression* keyword arguments, and ***expression* mapping unpackings transformed into keyword arguments

In all cases, all arguments, starred or otherwise, are separated by commas.

Formal definition

More concisely again, the ordering of arguments in function calls can be defined as follows, where *function* is an expression that evaluates to a callable object, and *-or-* and *[]* here again mean a choice and optional part, respectively (they're not part of the code you type):

```
function([positional-values -or- *iterable-positional-unpackings]  
          [keyword-arguments -or- *iterable-positional-unpackings]  
          [keyword-arguments -or- **mapping-keyword-unpackings])
```

As a concrete example, the following passes a variety of positional, keyword, and unpacking arguments (this code serves as a demo here, but is not exactly the sort of thing you should strive to craft in practice!):

```
>>> def f(a, b, c, d, e, f, g, h, i):  
    print(a, b, c, d, e, f, g, h, i)  
  
>>> f(*[1], 2, *[3], 4, f=6, *[5], **dict(g=7), h=8, **{'i': 9})  
1 2 3 4 5 6 7 8 9
```

Boundary cases

As one consequence of the argument ordering in calls, once you use a *keyword* argument, you can no longer use any unstarred positionals—all subsequent arguments must also be keywords, or single or double stars:

```
>>> f(1, 2, c=3, 4)
SyntaxError: positional argument follows keyword argument unpacking
```

This is similar to defaults in definitions—but not exactly. Again, try not to conflate function definitions and calls. Despite their reuse of similar syntax, it has very different roles and rules in these tools. The `*` and `=`, for example, are used for unpacking and keywords in calls, but mean collection and defaults in definitions.

Once you code a *double star*, both positionals and single stars are also out of the game, though this seems inconsistent too—single stars can be freely mixed with keyword arguments (which is what a double star unpacks into), and keyword arguments cannot be mixed with positionals (which is what a single star unpacks into):

```
>>> f(1, 2, **{'d': 3}, 4)
SyntaxError: positional argument follows keyword argument unpacking
>>> f(1, 2, **{'d': 3}, *[4])
SyntaxError: iterable argument unpacking follows keyword argument unpacking

>>> f(1, 2, d=3, *[4])      # OK, but why? - like first error above
>>> f(1, 2, d=3, 4)         # Not OK, but why? - like preceding line
SyntaxError: positional argument follows keyword argument
```

Perspective

And if this is starting to make your head spin, it probably should. Python's argument-matching rules have been accumulated over time to incorporate new convolutions, and they are complex and perhaps even kludgy. The first rule of programming applies to function definitions and calls as everywhere else: keep it simple, unless it has to be complex. If you have to agonize over argument-ordering rules to understand code, it's probably time to reevaluate priorities.

Example: The min Wakeup Call

OK—it's time for something more realistic. To make the concepts here more concrete, the rest of this chapter works through a set of examples that demonstrate practical applications of argument-matching tools. First up is an exercise borrowed from live classes and used to rouse learners like you starting

to succumb to the knottiness of argument rules.

Here's the problem statement: suppose you're asked to code a function that is able to compute the *minimum value* from an arbitrary set of arguments, which may be arbitrary sorts of objects. That is, the function should accept *zero or more* arguments—as many as you wish to pass. Moreover, the function should work for all kinds of Python object *types*: numbers, strings, lists, lists of lists, files, and even `None`. To keep this fair, you don't need to support *dictionaries* or *mixed* nonnumeric types, because neither supports direct comparisons, per Chapters 8 and 9.

The first requirement provides a natural example of how the `*` feature can be put to good use—we can handle arbitrary arguments by collecting them in a tuple, and stepping over each with a simple `for` loop. The second part of the problem definition is easy in Python: because nearly every object type supports comparisons, we don't have to specialize the function per type (an application of *polymorphism*); we can simply compare objects blindly and let Python worry about what sort of comparison to perform according to the objects being compared.

Full Credit

The following script file, *mins.py* in [Example 18-2](#), shows four ways to code this operation (some of which were suggested by students in a group exercise designed to prevent post-lunch napping):

- The first function fetches the first argument from its `args` tuple, and traverses the rest by slicing off the first (there's no point in comparing the first object to itself, especially if it might be a large structure).
- The second version lets Python pick off the first and rest of the arguments automatically, and so avoids an index and slice; the code is simpler, and may be faster (though it would take many calls to matter).
- The third converts from a tuple to a list with the built-in `list` call and employs the list `sort` method: the first item has lowest value after an ascending-value sort.

- The fourth sorts too, but skips the list conversion (and two lines) by using the `sorted` built-in function.

Python sorting tools are coded in C, so they can be quicker than the other approaches at times, but the linear scans of the first two techniques may often make them faster.¹

Example 18-2. mins.py

"Find minimum value among all passed arguments of comparable types"

```
def min1(*args):
    res = args[0]
    for arg in args[1:]:
        if arg < res:
            res = arg
    return res

def min2(first, *rest):
    for arg in rest:
        if arg < first:
            first = arg
    return first

def min3(*args):
    tmp = list(args)
    tmp.sort()
    return tmp[0]

def min4(*args):
    return sorted(args)[0]

for func in (min1, min2, min3, min4):
    print(func.__name__ + '...')
    print(func(3, 4.0, 1, 2))
    print(func('bb', 'aa'))
    print(func([2, 2], [1, 1], [3, 3]))
    print(func(*'hack'))
```

Test all 4 functions
Mixed numerics
Strings: code points
Lists: recursive
Unpacked characters

This script's testing code uses the `__name__` attribute we met earlier, along with a `for` loop to run each function one at a time (remember, functions are objects that work in a tuple too). All four solutions produce the same result when the file is run, so we'll list just the first's output here. Run this file live, or import it as a module and type a few calls to its function interactively to experiment with them on your own (see [Chapter 3](#) for tips on both modes):

```
$ python3 mins.py
min1...
1
aa
[1, 1]
a
...and the same for others...
```

Notice that none of these four variants tests for the case where *no* arguments are passed in. They could, but there’s probably no point in doing so here—in all four solutions, Python will automatically raise an exception to signal the error if no arguments are sent. The first variant raises an exception when we try to fetch argument 0, the second when Python detects an argument list mismatch, and the third and fourth when we try to return item 0 post sort.

This is exactly what we want to happen—because these functions support any object (including None), there is no value that we could pass back to designate an error, so we may as well let the exception be raised. There are exceptions to this exceptions rule (e.g., you might test for errors yourself if you’d rather avoid actions that run before reaching the code that triggers an error automatically). But in general—and especially when errors aren’t common—it’s better to assume that arguments will work in your functions’ code, and let Python raise errors for you when they do not.

Bonus Points

You can get bonus points here for changing these functions to compute the arguments’ *maximum*, rather than minimum, value. This one’s trivial: the first two versions only require changing < to >, and the last two simply require that we return item [-1] instead of [0]. For an extra point, be sure to mod the function name to “max” as well (though this part is strictly optional).

True curve busters might also note that it’s possible to generalize a *single* function to compute either a minimum *or* a maximum value, by evaluating comparison expression strings with a tool like the `eval` built-in function (described in Python’s library manual, and at various appearances here, especially its note in [Chapter 10](#)), or by passing in an arbitrary comparison function. [Example 18-3](#) shows how to implement the latter scheme for one of the

coding options.

Example 18-3. minmax.py

```
"Find minimum -or- maximum value of arguments"

def minmax(test, *args):
    res = args[0]
    for arg in args[1:]:
        if test(arg, res):
            res = arg
    return res

def lessthan(x, y): return x < y           # See also: lambda, eval
def grtrthan(x, y): return x > y

print(minmax(lessthan, 4, 2, 1, 5, 6, 3))  # Self-test code
print(minmax(grtrthan, 4, 2, 1, 5, 6, 3))
```

Running this script prints both minimum and maximum, per its self-test code at the end:

```
$ python3 minmax.py
1
6
```

Again, functions are just another kind of object, which allows them to be passed into other functions as done here. To make this a `max` (or other) function, for example, we simply pass in the right sort of `test` function to `minmax`. This may seem like extra work, but the main point of *generalizing* functions this way—instead of cutting and pasting to change just a single character—is that we’ll only have one version to change in the future, not two.

The Punch Line

Of course, all this was just a coding exercise. There’s really no reason to write `min` or `max` functions, because both are built-ins in Python! We met them briefly in [Chapter 5](#) in conjunction with numeric tools, and again in [Chapter 14](#) when exploring iteration contexts. The built-in versions work almost exactly like ours, but they’re coded in C for optimal speed and accept either a single iterable or multiple arguments. Still, though it’s superfluous in this context, the general coding pattern we used here might be useful in other scenarios.

Example: Generalized Set Functions

Our next example also demos special argument-matching modes at work. At the end of [Chapter 16](#), we wrote a function that returned the intersection of two sequences (really, it picked out items that appeared in both). [Example 18-4](#) codes an augmented version that intersects an *arbitrary* number of sequences (one or more) by using the argument-matching form `*args` to collect all the passed-in arguments. Because the arguments come in as a tuple, we can process them in a simple for loop. Just for fun, we'll code a `union` function that also accepts an arbitrary number of arguments to collect items that appear in *any* of the operands.

Example 18-4. inter2.py

```
"""
Implement intersection and union for one or more arguments.
Inputs may be any sort of iterable that supports multiple in
tests, and results are always lists. This intersect avoids
duplicates in results by in test, but may be slow: improve me.
"""

def intersect(*args):
    res = []
    for x in args[0]:
        if x in res: continue
        for other in args[1:]:
            if x not in other: break
        else:
            res.append(x)
    return res

def union(*args):
    res = []
    for seq in args:
        for x in seq:
            if not x in res:
                res.append(x)
    return res
```

Because these tools are potentially worth reusing (and are too big to retype interactively), we'll store the functions in a *module* file called `inter2.py`. Again, if you're unsure about how modules and imports work, see the introduction in [Chapter 3](#), or stay tuned for in-depth coverage in [Part V](#). This chapter's module usage is simple, but per [Chapter 3](#), be sure to launch your REPL in the same

folder as the file, so imports can find it.

Like [Chapter 16](#)'s original `intersect`, both of the functions in this module work on any kind of sequence. Here they are live at the REPL, processing strings, mixed types, and more than two sequences:

```
$ python3
>>> from inter2 import intersect, union
>>> s1, s2, s3 = 'HACKK', 'CODE', 'CASH'

>>> intersect(s1, s2), union(s1, s3)           # Two operands
(['C'], ['H', 'A', 'C', 'K', 'S'])

>>> intersect([1, 2, 3, 4], (1, 4))           # Mixed types
[1, 4]

>>> intersect(s1, s2, s3)                     # Three operands
['C']

>>> union(s1, s2, s3)
['H', 'A', 'C', 'K', 'O', 'D', 'E', 'S']
```

Testing the Code

To test more thoroughly, the following continues our REPL session to code a function that applies the two tools to arguments in different orders using a simple *shuffling* technique that we saw in [Chapter 13](#)—it slices to move the first to the end on each loop, uses a `*` to unpack arguments, and sorts so results are comparable. Notice that arguments for the function are sent as a sequence (not discrete items), and the `trace` configuration option is keyword-only here:

```
>>> def tester(func, items, *, trace=True):
    for i in range(len(items)):
        items = items[1:] + items[:1]           # Move front item to back
        if trace: print(items)
        print(sorted(func(*items)))             # Test with reordered items

>>> tester(intersect, (s1, s2, s3))             # Use strings from prior listing
('CODE', 'CASH', 'HACKK')
['C']
('CASH', 'HACKK', 'CODE')
['C']
('HACKK', 'CODE', 'CASH')
['C']
```

```
>>> tester(union, (s1, s2, s3), trace=False)
['A', 'C', 'D', 'E', 'H', 'K', 'O', 'S']
['A', 'C', 'D', 'E', 'H', 'K', 'O', 'S']
['A', 'C', 'D', 'E', 'H', 'K', 'O', 'S']

>>> tester(intersect, (s1, s2, s3), trace=False)
['C']
['C']
['C']
```

Two context notes here: first, because *duplicates* won't appear in these intersection and union functions, they qualify as set operations mathematically, but may not be optimal in term of speed. Still, there's not much point in improving this demo's code—intersection and union, like `min` and `max`, are built-in operations today: the *set* object we explored in [Chapter 5](#) does intersection and union with `&` and `|`, and has methods that take multiple operands too. Hence, optimizing this code is left as suggested exercise, but see *inter3.py* in the examples package for pointers.

Second, the argument scrambling in the tester here doesn't generate all possible argument orders (that would require a full permutation, and 6 orderings for 3 arguments), but suffices to check if argument order impacts results. As a preview, though, the tester would be simpler and more flexible if it delegated scrambling to a reusable *function*. Watch for this revision in [Chapter 20](#), after we've explored how to code user-defined *generators*. We'll also recode set tools one last time in [Chapter 32](#) and a solution to a [Part VI](#) exercise, as classes that add them to the list object as methods.

Example: Rolling Your Own Print

To close out the chapter, let's look at one last example of argument matching at work: this section uses both `*` and `**` arbitrary-arguments collectors up front, and keyword-only arguments later, to emulate most of what Python's `print` function does. Like the preceding examples, there's no urgent reason to code tools that Python provides. Also like the others, though, this is both instructive and may be a basis for custom variants of built-in tools.

For both purposes, the module file in [Example 18-5](#) does roughly the same job

as `print` in a small amount of reusable and modifiable code, by building and routing the print string per configuration arguments.

Example 18-5. print3.py

```
r"""
Emulate most of the Python 3.X print function as customizable code.
Call signature: print3(*args, sep=' ', end='\n', file=sys.stdout).
"""

import sys

def print3(*args, **kwargs):
    sep = kwargs.get('sep', ' ')          # Keyword arg defaults
    end = kwargs.get('end', '\n')
    file = kwargs.get('file', sys.stdout)
    output = ''                          # Build+print a string
    first = True
    for arg in args:
        output += ('' if first else sep) + str(arg)
        first = False
    file.write(output + end)
```

Notice that this module’s docstring uses a *raw string* to retain its backslash for `help` (per [Chapter 15](#)). Also note that this module’s function need not be called “`print3`” because “`print`” is a built-in but not a reserved word, but using a different name avoids inadvertently hiding the built-in. To test it, import this into another file or the interactive prompt, and use it like the `print` built-in.

[Example 18-6](#) codes a test script that imports our printer as a demo.

Example 18-6. test-print3.py

```
from print3 import print3
print3(1, 2, 3)                # Defaults
print3(1, 2, 3, sep='')        # Suppress separator
print3(1, 2, 3, sep='...')     # Custom separator
print3(1, [2], (3,), sep='...') # Various object types

print3(4, 5, 6, sep='', end='') # Suppress newline
print3(7, 8, 9)                # Finish line
print3()                       # Blank line

import sys
print3(1, 2, 3, sep='?', end='.\n', file=sys.stderr) # Redirect to stream

print3('LP6E was here', file=open('log.txt', 'w'))    # Redirect to a file
print3(open('log.txt').read())
```

When this is run, our `print3` produces the same results as the `print` built-in. Fine points here: `stderr` goes to your console by default, and it's OK to use a *dash* in this script's name because it's run, not imported (again, we'll be focusing on such module details in this book's next part):

```
$ python3 test-print3.py
1 2 3
123
1...2...3
1...[2]...(3,)
4567 8 9

1?2?3.
LP6E was here
```

As usual, the generality of its toolset allows us to prototype or develop concepts in the Python language itself. In this case, argument-matching tools are as flexible in Python code as they are in Python's internal implementation.

Using Keyword-Only Arguments

Our `print` emulator works, but has a minor flaw baked in: it assumes that all positional arguments are to be printed, and all keywords are for options only. Any extra keyword arguments are silently ignored, and neither printed nor reported. A call like the following, for instance, will ignore the extra—and likely erroneous—`sap` argument:

```
$ python3
>>> from print3 import print3
>>> print3(3.12, 'py', sap='@')
3.12 py
```

It may not make sense to print superfluous keywords, but we can detect them manually by using `dict.pop()` to delete fetched keywords, and checking if the dictionary is not empty when we're done. The version in [Example 18-7](#) does—it's equivalent to the original, but triggers a built-in exception with a `raise` statement when unexpected keyword arguments are sent by a call (this is partly preview: we'll study exceptions and `raise` in depth in [Part VII](#)).

Example 18-7. `print3_pops.py`

```
"Use keyword-collector arguments with deletion and defaults"
import sys
```

```
def print3(*args, **kargs):
    sep = kargs.pop('sep', ' ')
    end = kargs.pop('end', '\n')
    file = kargs.pop('file', sys.stdout)
    if kargs: raise TypeError(f'extra keywords: {kargs}')
    output = ''
    first = True
    for arg in args:
        output += ('' if first else sep) + str(arg)
        first = False
    file.write(output + end)
```

Notice that this file's name uses an *underscore* instead of a dash: because it's to be imported, its name must follow the rules for variables of [Chapter 11](#). This version works as before, but it now catches extraneous keyword arguments:

```
>>> from print3_pops import print3
>>> print3(3.12, 'py', sep='@')
3.12@py
>>> print3(3.12, 'py', sap='@')
TypeError: extra keywords: {'sap': '@'}
```

It's OK to reimport the same `print3` name from a different file here: this simply replaces the prior version just like reassigning any other variable (more on this later in this book). That being coded, this example could also use keyword-only arguments to *automatically* validate configuration arguments, as the final variant in [Example 18-8](#) illustrates.

Example 18-8. print3_kwonly.py

```
"Use keyword-only arguments to emulate print"
import sys

def print3(*args, sep=' ', end='\n', file=sys.stdout):
    output = ''
    first = True
    for arg in args:
        output += ('' if first else sep) + str(arg)
        first = False
    file.write(output + end)
```

This version works the same way for valid calls, but catches invalid keywords with keyword-only arguments instead of manual code, and is a prime example of

how keyword-only arguments can address coding needs:

```
>>> from print3_kwonly import print3
>>> print3(3.12, 'py', sep='@')
3.12@py
>>> print3(3.12, 'py', sap='@')
TypeError: print3() got an unexpected keyword argument 'sap'
```

Given that this version of the function also requires *four fewer lines* of code than its predecessor, keyword-only arguments can simplify a specific category of functions that accept both arguments and options. A similar case can be made for positional-inly arguments versus manual code, but it's more obscure, and this chapter has run out of space. For another example of keyword-only arguments at work, stay tuned for the iteration-timing case study in [Chapter 21](#).

And for more inspiration, also see the sidebar [“Why You Will Care: Customizing open”](#). Much as we did there, our print emulator could be assigned to `builtins.print` to replace the built-in with our custom version everywhere in a program. There's no reason to do that for a version that's the *same*, of course, but this technique can be used to install a replacement printer that mods or extends the built-in (e.g., with logging).

Chapter Summary

In this chapter, we studied the second of two key concepts related to functions: the *arguments* used to send objects to a function. As we saw, arguments are passed to a function by assignment, which means by object reference (which really means by pointer), and are open to the usual side effects for shared mutable objects, desired or not. We also studied some advanced extensions that generalize argument matching, including default and keyword arguments, tools for collecting and unpacking arbitrarily many arguments, and keyword- and positional-only arguments. Finally, we explored a few larger examples that employed argument tools, and previewed module topics of this book's next part.

The next chapter continues our look at functions with a grab bag of more advanced function-related ideas: function annotations, recursion, and more on `lambda` and functional tools such as `map` and `filter`. Many of these concepts stem from the fact that functions are normal objects in Python, and so support flexible processing tools and modes. Before diving into those topics, however, take this chapter's quiz to review the argument ideas we've studied here.

Test Your Knowledge: Quiz

This quiz asks you to trace through examples of function definitions and calls to predict their outputs. Try to work out the answers on your own before resorting to cut and paste in a REPL:

1. What is the output of the following code, and why?

```
>>> def func(a, b=4, c=5):  
    print(a, b, c)  
  
>>> func(1, 2)
```

2. What is the output of this code, and why?

```
>>> def func(a, b, c=5):
```

```
print(a, b, c)
```

```
>>> func(1, c=3, b=2)
```

3. How about this code: what is its output, and why?

```
>>> def func(a, *pargs):  
    print(a, pargs)
```

```
>>> func(1, 2, 3)
```

4. What does this code print, and why?

```
>>> def func(a, **kargs):  
    print(a, kargs)
```

```
>>> func(a=1, c=3, b=2)
```

5. What gets printed by this, and why?

```
>>> def func(a, b, c=3, d=4): print(a, b, c, d)
```

```
>>> func(1, *(5, 6))
```

6. One last time: what is the output of this code, and why?

```
>>> def func(a, b, c):  
    a = 2; b[0] = 'x'; c['a'] = 'y'
```

```
>>> L=1; M=[1]; N={'a': 0}
```

```
>>> func(L, M, N)
```

```
>>> L, M, N
```

Test Your Knowledge: Answers

1. The output here is 1 2 5, because 1 and 2 are passed to a and b by position, and c is omitted in the call and hence defaults to 5.
2. The output this time is 1 2 3: 1 is passed to a by position, and b and c are passed 2 and 3 by name (the left-to-right order doesn't matter when keyword arguments are used like this).
3. This code prints 1 (2, 3), because 1 is passed to a and the *args collects the remaining positional arguments into a new tuple object. We can step through the extra positional arguments tuple with any iteration tool (e.g., for arg in args: ...).
4. This time the code prints 1 {'c': 3, 'b': 2}, because 1 is passed to a by name and the **kwargs collects the remaining keyword arguments into a dictionary. We could step through the extra keyword arguments dictionary by key with any iteration tool (e.g., for key in kwargs: ...). Note that the order of the dictionary's keys reflects the order in which keyword arguments are passed, in recent Pythons.
5. The output here is 1 5 6 4: the 1 matches a by position, 5 and 6 match b and c by * positional unpacking (6 overrides c's default), and d defaults to 4 because it was not passed a value.
6. This displays (1, ['x'], {'a': 'y'})—the first assignment in the function doesn't impact the caller, but the second two do because they change passed-in mutable objects in place.

WHY YOU WILL CARE: KEYWORD ARGUMENTS

As you can probably tell, some argument-matching mode combos can be complex. They are also largely optional in your code; in fact, you can get by with just simple positional matching, and it's probably a good idea to do so when you're starting out. However, because many Python tools make good use of them, some general knowledge of these modes is important.

For example, keyword arguments play a key role in `tkinter`, the de facto standard GUI API for Python. We touch on `tkinter` only briefly at various points in this book, but in terms of its call patterns, keyword arguments set

configuration options when GUI components are built. For instance, a call of the form:

```
from tkinter import Button
widget = Button(text='Press me', command=someFunction)
```

creates a new button and specifies its text displayed in the GUI and callback function run on a press, using the `text` and `command` keyword arguments. Since the number of configuration options for a widget can be large, keyword arguments let you pick and choose which to apply. Without them, you might have to either list all the possible options by position or hope for a judicious positional-argument defaults protocol that would handle every possible option arrangement.

Many built-in functions in Python expect us to use keywords for usage-mode options as well, which may or may not have defaults. As we learned in [Chapter 8](#), for instance, the `sorted` built-in:

```
sorted(iterable, key=None, reverse=False)
```

expects us to pass an iterable object to be sorted, but also allows us to pass in optional keyword arguments to specify both a value-transform function and a reversal flag, which default to `None` and `False`, respectively. Since we normally don't use these options, they may be omitted to apply defaults.

As we've also seen, the `dict`, `str.format`, and `print` calls accept keywords as well—other usages we had to introduce in earlier chapters because of their forward dependence on argument-passing modes we've finally studied here (alas, those who change Python already know Python!).

¹ Actually, it's complicated. CPython's `sort` (used by both `list.sort` and `sorted`) is coded in C and uses a heavily optimized algorithm that attempts to take advantage of partial ordering in the items to be sorted. Still, sorting is an inherently busy operation (it must chop up the sequence and put it back together many times), and the other versions simply perform one linear left-to-right scan. This suggests that sorting may be quicker if the arguments are partially *ordered*, but is likely to be slower otherwise. Even so, Python performance changes regularly; the fact that sorting is implemented in the C language can help greatly; and the speed difference may not matter in many programs. For an exact analysis, you should time the alternatives with the `time` or `timeit` modules—you'll see how soon in

Chapter 21, but file *mins-timings.txt* in the examples package demos the idea if you can't wait. The gist: in CPython, the nonsort mins are faster for random arguments, but slower for ordered—today!

Chapter 19. Function Odds and Ends

This chapter presents a medley of function-related topics: recursive functions; function attributes, annotations, and decorations; and more on both the `lambda` expression and functional-programming tools such as `map` and `filter`. These are all somewhat advanced tools that, depending on your job description, you may not encounter on a regular basis. Because of their roles in some domains, though, a basic understanding can be useful. `lambda`, for instance, makes regular appearances in GUIs, and functional programming techniques have grown common in Python code.

Some of the art of using functions lies in the *interfaces* between them, so we will also explore some general function design principles here. The next chapter continues the advanced themes here with an exploration of generator functions and expressions and a revival of list comprehensions in the context of the functional tools we will study here.

Function Design Concepts

Now that we've studied function essentials in Python, let's open this chapter with some perspective. When you start using functions in earnest, you're faced with choices about how to glue components together—for instance, how to decompose a task into purposeful functions (known as *cohesion*), and how your functions should communicate (called *coupling*). You also need to take note of the *size* of your functions because it directly impacts code usability. Some of this falls into the category of structured analysis and design, but it applies to Python code as to any other.

We explored some ideas related to function and module coupling in [Chapter 17](#) when studying scopes, but here is a review of a few general guidelines for readers new to function design principles:

- **Coupling: use arguments for inputs and return for outputs.**
Generally, you should strive to make a function independent of the world outside of it. Arguments and return statements are often the best ways to isolate external dependencies to a small number of well-known places in your code.
- **Coupling: use global variables only when truly necessary.** As we've seen, global variables (i.e., names in the enclosing module) are usually a poor way for functions to communicate. They can create dependencies and timing issues that make programs difficult to debug, change, and reuse.
- **Coupling: don't change mutable arguments unless the caller expects it.** As we've also seen, functions can change parts of passed-in mutable objects, but as with global variables, this creates a tight coupling between the caller and callee, which can make a function too specific and brittle.
- **Cohesion: each function should have a single, unified purpose.**
When designed well, each of your functions should do one thing—something you can summarize in a simple declarative sentence. If that sentence is very broad (e.g., “this function implements my whole program”) or contains lots of conjunctions (e.g., “this function gives employee raises *and* submits a pizza order”), you might want to think about splitting it into separate and simpler functions. Otherwise, there is no way to reuse the code of the individual steps embedded in the function.
- **Size: each function should be relatively small.** This naturally follows from the preceding goal, but if your functions start spanning multiple pages on your display, it's probably time to split them. Especially given that Python code is so concise to begin with, a long or deeply nested function is often a symptom of design problems. Keep it simple, and keep it short.
- **Coupling: avoid changing variables in another module file directly.**
We also introduced this concept in [Chapter 17](#), and we'll revisit it in the next part of the book when we focus on modules. For reference, though,

remember that changing variables across file boundaries sets up a coupling between modules similar to how global variables couple functions—the modules become difficult to understand and reuse separately. Use accessor functions whenever possible, instead of direct assignment statements.

Figure 19-1 summarizes the ways functions can talk to the outside world; inputs may come from items on the left side, and results may be sent out in any of the forms on the right. Nonlocals might belong in this sketch too, but they're mostly a state-retention tool in the same category as other local variables. Despite the array of options, good function designs prefer to use only arguments for inputs and return statements for outputs, whenever possible.

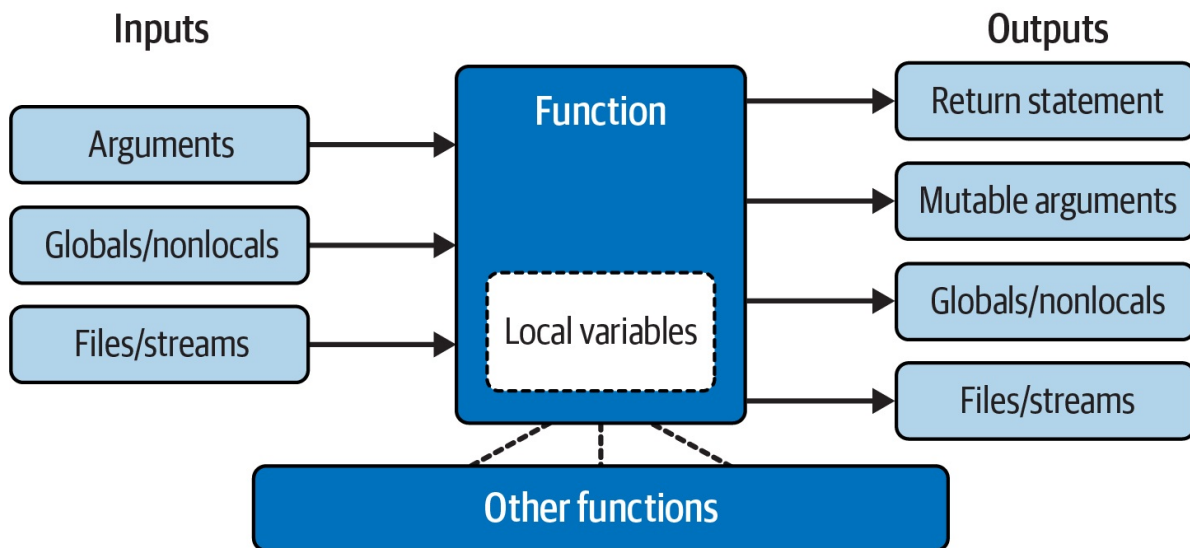


Figure 19-1. Function execution environment

Of course, there are plenty of exceptions to the preceding design rules, including some related to Python's OOP support. As you'll see in **Part VI**, Python classes *depend* on changing a passed-in mutable object—class functions set attributes of an automatically passed-in argument called `self` to change per-object state information (e.g., `self.edition=6`). Moreover, if classes are not used, global variables are a straightforward way for functions in modules to retain single-copy state between calls. Side effects are usually dangerous only if they're unexpected.

In general though, you should strive to minimize external dependencies in functions and other program components. The more *self-contained* a function is,

the easier it will be to understand, reuse, and modify. Making code as freestanding as possible is especially important when functions go multilevel with recursion, per the next section.

Recursive Functions

We mentioned recursion in relation to comparisons of core types in [Chapter 9](#). While discussing scope rules near the start of [Chapter 17](#), we also briefly noted that Python supports *recursive functions*—functions that call themselves either directly or indirectly in order to loop. In this section, we’ll explore what this looks like in our functions’ code.

Recursion is a somewhat advanced topic, and it’s relatively uncommon to see in Python, partly because Python’s procedural shed includes simpler looping tools. Still, it’s a useful technique to know about, as it allows programs to traverse structures that have arbitrary and unpredictable shapes and depths—planning travel routes, analyzing language, and crawling links on the web, for example. Recursion is even an alternative to simple loops and iterations, though not necessarily the simplest or most efficient one.

Summation with Recursion

Let’s turn to some examples. To sum a list (or other sequence) of numbers, we can either use the built-in `sum` function or write a more custom version of our own. [Example 19-1](#) shows what a custom summing function might look like when coded with recursion.

Example 19-1. mysum.py

```
def mysum(L):
    if not L:
        return 0
    else:
        return L[0] + mysum(L[1:])    # Call myself recursively
```

To use, either add self-test code to the bottom of this file and run it as a script, or import it as a module and test at the REPL (again, the file may need to be in the folder where you’re working either way, per [Chapter 3](#)). With the latter:

```
>>> from mysum import mysum    # Import file as a module in a REPL
```

```
>>> mysum([1, 2, 3, 4, 5])           # Sum all the numbers in any sequence
15
```

At each level, this function calls itself recursively to compute the sum of the *rest* of the list, which is later added to the item at the *front*. This recursive loop ends and zero is returned when the list becomes empty. When using recursion like this, each open level of call to the function has its own copy of the function's local scope on the runtime call stack. Here, that means *L* is different in each level, so each remembers its own segment of the list.

If this is difficult to understand (and it often is for new programmers), try adding a `print` of *L* to the function and run it again, to trace the current list at each call level; here's the required mod pasted at the REPL for variety:

```
>>> def mysum(L):
    print(L)                # Trace recursive levels
    if not L:               # L shorter at each level
        return 0
    else:
        return L[0] + mysum(L[1:])

>>> mysum([1, 2, 3, 4, 5])
[1, 2, 3, 4, 5]
[2, 3, 4, 5]
[3, 4, 5]
[4, 5]
[5]
[]
15
```

As you can see, the list to be summed grows smaller at each recursive level, until it becomes empty—the termination of the recursive loop. The sum is then computed as the recursive calls unwind on returns.

Coding Alternatives

Interestingly, we can use Python's `if/else` ternary expression (described in [Chapter 12](#)) to save some code real estate here. We can also generalize for any summable type (which is easier if we assume at least one item in the input) and use extended-unpacking assignment to make the first/rest unpacking simpler (as covered in [Chapter 11](#)). [Example 19-2](#) collects all three of these mods, ready to

be run in a file or pasted into a REPL.

Example 19-2. mysum_alts.py

```
def mysum(L):
    return 0 if not L else L[0] + mysum(L[1:])          # Use ternary expression

def mysum(L):
    return L[0] if len(L) == 1 else L[0] + mysum(L[1:]) # Any type, assume one+

def mysum(L):
    first, *rest = L
    return first if not rest else first + mysum(rest)   # Use extended unpacking
```

When tested individually, all three of these alternatives handle numeric summation the same as the original:

```
>>> mysum([1, 2, 3, 4, 5])
15
```

Uniquely, the latter two fail for empties (e.g., `mysum([])`), but handle sequences of *any* object type that supports `+`, not just numbers (for strings, the effect is similar to `' '.join(L)`):

```
>>> mysum(('h', 'a', 'c', 'k'))          # The last two fail on mysum([])
'hack'
>>> mysum(['hack', 'app', 'code'])        # But they support nonnumeric types
'hackappcode'
```

Run some tests on your own for more insight. If you study these three variants, you'll also find that:

- The latter two work on a single *string* argument (e.g., `mysum('hack')`), because strings are sequences of one-character strings (though this use case isn't very useful: you get back the same string).
- The third variant also works on arbitrary *iterables*, including open input files (`mysum(open(name))`), but the others' indexing generally fails on nonsequences (see [Chapter 14](#) for extended-unpacking demos).

You may also notice that the third variant's unpacking assignment is similar to a `*` collector in a function header, and it's tempting to recode it as such. This won't

quite work, though, because it would expect *individual* arguments, not a single iterable—unless we *also* star both the top-level input and recursive call. Here's the end result, though by summing discrete arguments, it solves a different problem than both the prior versions and built-in `sum`:

```
>>> def mysum(first, *rest):
        return first if not rest else first + mysum(*rest)

>>> mysum(*[1, 2, 3, 4, 5])
15
>>> mysum(*'hack')
'hack'
```

Finally, bear in mind that recursion can be either *direct*, as in the examples so far, or *indirect*, as in the following—a function that calls another function, which calls back to its caller. The net effect is the same, though there are two function calls at each level instead of one:

```
>>> def mysum(L):
        if not L: return 0
        return nonempty(L)                                # Call a function that calls me

>>> def nonempty(L):
        return L[0] + mysum(L[1:])                        # Indirectly recursive

>>> mysum([1.1, 2.2, 3.3, 4.4])
11.0
```

Loop Statements Versus Recursion

Though recursion works for summing in the prior sections' examples, it's probably overkill in this context. In fact, recursion is not used nearly as often in Python as in more esoteric languages like Prolog or Lisp, because Python emphasizes simpler procedural statements like loops, which are usually more natural. The `while`, for example, often makes things more concrete, and it doesn't require that a function be defined to allow recursive calls:

```
>>> L = [1, 2, 3, 4, 5]
>>> tot = 0
>>> while L:
        tot += L[0]
        L = L[1:]
```

```
>>> tot
15
```

Better yet, for loops iterate for us automatically, making recursion largely extraneous in many cases (and, in all likelihood, less efficient in terms of memory space and execution time):

```
>>> L = [1, 2, 3, 4, 5]
>>> tot = 0
>>> for x in L: tot += x

>>> tot
15
```

With looping statements, we don't require a fresh copy of a local scope on the call stack for each iteration, and we avoid the speed costs associated with function calls in general. (Stay tuned for [Chapter 21](#)'s timer case study for ways to compare the execution times of alternatives like these.)

Handling Arbitrary Structures

On the other hand, recursion—or equivalent and explicit stack-based algorithms we'll explore shortly—can be *required* to traverse arbitrarily shaped structures. As a simple example of recursion's role in this context, consider the task of computing the sum of all the numbers in a nested sublists structure like this:

```
[1, [2, [3, 4], 5], 6, [7, 8]]           # Arbitrarily nested sublists
```

Neither our prior summers nor simple looping statements will work here because this is not a linear iteration. Nested looping statements do not suffice either—because the sublists may be nested to arbitrary *depth* and in an arbitrary *shape*, there's no way to know how many nested loops to code to handle all cases. Instead, the function in [Example 19-3](#) accommodates such general nesting by using recursion to visit sublists along the way.

Example 19-3. sumtree.py

```
def sumtree(L, trace=False):
    tot = 0
    for x in L:
        # For each item at this level
```

```

    if not isinstance(x, list):
        tot += x                    # Add numbers directly
        if trace: print(x, end=', ')
    else:
        tot += sumtree(x, trace)    # Recur for sublists
return tot

```

In this file's function, each recursive level runs a for loop to add numbers, or recur into sublists to open new levels. Recall from [Chapter 9](#) that `isinstance` compares object types; it's used here to detect nested sublists.

This code is also instrumented to trace items as they are added to the total: if `trace` is passed a true value, you can see how the object is scanned left to right. Because it also steps down into sublists with recursion along the way, though, the traversal is really both horizontal and vertical:

```

>>> from sumtree import sumtree
>>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]])
36
>>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]], trace=True)
1, 2, 3, 4, 5, 6, 7, 8, 36

```

Testing with a separate script

At this point, we could test other cases by typing them interactively or by adding code to the bottom of the file, but you're probably starting to see that this can be a bit limiting. Instead, the script in [Example 19-4](#) makes the process automatic and easily repeatable. As a bonus, it can be used for other summers we'll code in a moment.

Example 19-4. sumtree_tester.py

```

tests = (
    [1, [2, [3, 4], 5], 6, [7, 8]],      # Mixed nesting => 36
    [1, [2, [3, [4, [5]]]]],             # Right-heavy nesting => 15
    [1, [2, [3, [4, [5]]]]],             # Left-heavy nesting => 15
)

def tester(sumtree, trace=True):
    for test in tests:
        print(sumtree(test, trace))

```

To use this tester, simply import both summer and tester, and pass the former to the latter. When run, our summer prints numbers as added with the final sum at the end, for each of three canned tests:

```

>>> from sumtree import sumtree           # Get the summer
>>> from sumtree_tester import tester     # Get the tester
>>> tester(sumtree)                       # Run the tester on the summer
1, 2, 3, 4, 5, 6, 7, 8, 36
1, 2, 3, 4, 5, 15
1, 2, 3, 4, 5, 15

```

Within `tester`, `sumtree` refers to the `summer` function passed into it. Again, because functions are objects, passing them around this way is natural, and makes code flexible.

Recursion versus queues and stacks

It sometimes helps recursion newcomers to understand that internally, Python implements recursion by pushing information on a *call stack* at each recursive call, so it remembers where it must return and continue later. In fact, it's generally possible to implement recursive-style procedures *without* recursive calls, by using an explicit stack or queue of your own to keep track of remaining steps.

For instance, [Example 19-5](#) computes the same sums as the prior example, but uses an explicit list to schedule when it will visit items in the subject, instead of issuing recursive calls. The item at the front of the list is always the next to be processed and summed.

Example 19-5. sumtree_queue.py

```

def sumtree(L, trace=False):               # Breadth-first, explicit queue
    tot = 0
    items = list(L)                        # Start with copy of top level
    while items:
        front = items.pop(0)               # Fetch/delete front item
        if not isinstance(front, list):
            tot += front                    # Add numbers directly
            if trace: print(front, end=', ')
        else:
            items.extend(front)             # <== Append all in nested list
    return tot

```

Technically, this code traverses the list in *breadth-first* fashion (across before down), because it adds nested lists' contents to the *end* of the list—forming a FIFO (first-in-first-out) *queue*. The net effect sums by horizontal levels. To test, we can either import and use the new `summer` directly, or route it to the

Example 19-4 tester to be exercised automatically with tracing:

```
>>> from sumtree_queue import sumtree           # Get the new summer
>>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]])
36
>>> from sumtree_tester import tester           # Unless already imported
>>> tester(sumtree)                             # Run tester on _this_ summer
1, 6, 2, 5, 7, 8, 3, 4, 36
1, 2, 3, 4, 5, 15
5, 4, 3, 2, 1, 15
```

Fine points: we don't have to reimport the tester again if it's already been imported in this session, and importing the same-named summer just works—the new summer's filename makes it unique, and `sumtree` is always the latest version imported if you import more than one, because imports *assign* names (see [Chapter 18](#)'s print emulators for another example of this pattern at work, and watch for more on imports in this book's next part).

More importantly, notice how the order in which numbers are visited here is different than in the original recursive-call version, due to the breadth-first queue. Trace through the tester's tests to see how this pans out.

If we instead want to emulate the traversal of the recursive-call version more closely, we can change this code to perform *depth-first* traversal (down before across) simply by adding the contents of nested lists to the *front* of the list—forming a last-in-first-out (LIFO) *stack*. [Example 19-6](#) makes the required mods, but the only way it differs from the breadth-first version is the line that adds to the front instead of the end, marked with `<==` in a comment.

Example 19-6. sumtree_stack.py

```
def sumtree(L, trace=False):                     # Depth-first, explicit stack
    tot = 0
    items = list(L)                             # Start with copy of top level
    while items:
        front = items.pop(0)                    # Fetch/delete front item
        if not isinstance(front, list):
            tot += front                        # Add numbers directly
            if trace: print(front, end=', ')
        else:
            items[:0] = front                  # <== Prepend all in nested list
    return tot
```

As before, we can use this function directly, or pass it to the same tester; its file

makes it distinct, and its name refers to the latest import. When run, this summer visits numbers in the same order as the recursive-calls original, but manages the traversal with an explicit stack instead of recursion:

```
>>> from sumtree_stack import sumtree           # Same name, different file
>>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]])
36
>>> from sumtree_tester import tester           # Optional if already imported
>>> tester(sumtree)
1, 2, 3, 4, 5, 6, 7, 8, 36
1, 2, 3, 4, 5, 15
1, 2, 3, 4, 5, 15
```

For more on the last two examples (plus another breadth-first coding variant omitted here), see file *sumtree_etc.py* in the book's examples package. It adds additional tracing so you can watch it walk structures in more detail.

In general, though, once you get the hang of recursive calls, they may be more natural than the explicit scheduling lists they automate, and are generally preferred unless you need to traverse structures in specialized ways. Some programs, for example, perform a *best-first* search that requires an explicit search queue ordered by relevance or other criteria. If you think of a web crawler that scores sites visited by content, the applications may start to become clearer.

Cycles, paths, and stack limits

As is, these programs suffice as demos, but larger recursive applications can sometimes require a bit more infrastructure than shown here: they may need to avoid cycles or repeats, record paths taken for later use, and expand stack space when using recursive calls instead of explicit queues or stacks.

For instance, neither the recursive-call nor the explicit queue/stack examples in this section do anything about avoiding *cycles*—visiting a location already visited. That's not required here, because we're traversing strictly hierarchical trees of list objects. If data can be a cyclic graph, though, both these schemes will fail: the recursive-call scheme will fall into an infinite recursive loop (and may run out of call-stack space), and the others will fall into simple infinite loops, re-adding the same items to their lists (and may or may not run out of general memory). In fact, it's easy to demo the perils by creating a cyclic object with the strange code we met in an exercise at the end of **Chapter 3**:

```

>>> L = [1, 2]
>>> L.append(L)      # Make a cyclic object: L references itself
>>> L
[1, 2, [...]]

>>> from sumtree import sumtree
>>> sumtree(L)
RecursionError: maximum recursion depth exceeded

>>> from sumtree_queue import sumtree
>>> sumtree(L)
...hang or crash, and ditto for stack...

```

Some programs also need to avoid repeated processing for a state reached more than once, even if that wouldn't lead to a loop. To do better, a recursive-call traversal might make and pass along a mutable set, dictionary, or list of states visited so far and check for repeats as it goes. We will use this scheme in later recursive examples in this book:

```

if state not in visited:
    visited.add(state)      # x.add(state), x[state]=True, or x.append(state)
    ...proceed...

```

Nonrecursive alternatives might similarly avoid adding states already visited with code like the following. Subtly, object cycles may require `is` (not `in`), and simply checking for duplicates already on the `items` list would avoid scheduling a state twice but would not prevent revisiting a state visited earlier and hence removed from that list:

```

visited.add(front)
...proceed...
items.extend([x for x in front if x not in visited])

```

This model doesn't quite apply to this section's use case that simply adds numbers in lists, but other applications will generally be able to identify repeated states—a URL of a previously visited web page, for instance. In fact, we'll use such techniques to avoid cycles and repeats in the later examples listed in the next section.

Some programs may also need to record complete *paths* for each state followed so they can report solutions when finished. In such cases, each item in the

nonrecursive scheme’s stack or queue may be a full path list that suffices for a record of states visited, and contains the next item to explore at either end.

Also note that standard Python limits the *depth* of its runtime call stack—crucial to recursive-call programs—to trap infinite recursion errors. To expand it for deeper journeys, use the `sys` module:

```
>>> sys.getrecursionlimit()      # 1000 calls deep default
1000
>>> sys.setrecursionlimit(10000) # Allow deeper nesting
>>> help(sys.setrecursionlimit)   # Read more about it
```

The maximum allowed setting can vary per platform. This isn’t required for programs that use stacks or queues to avoid recursive calls and gain more control over the traversal process (though they also won’t catch infinite loops).

More recursion examples

Although this section’s example is artificial, it is representative of a larger class of programs; inheritance trees and module import chains, for example, can exhibit similarly general structures, and computing tools such as permutations can require arbitrarily many nested loops. In fact, we’ll use recursion again in such roles later in this book:

- In [Chapter 20](#)’s *permute.py*, to shuffle arbitrary sequences
- In [Chapter 25](#)’s *reloadall.py*, to traverse import chains
- In [Chapter 29](#)’s *classtree.py*, to traverse class inheritance trees
- In [Chapter 31](#)’s *lister.py*, to traverse class inheritance trees again
- In [Appendix B](#), “Solutions to End-of-Part Exercises” at the end of this part of the book: countdowns and factorials

The second and third of these will also detect states already visited to avoid cycles and repeats. Although simple loops should generally be preferred to recursion for linear iterations on the grounds of simplicity and efficiency, you’ll find that recursion is essential in scenarios like those in these later examples.

Moreover, you sometimes need to be aware of the potential of *unintended*

recursion in your programs. As you'll also see later in the book, some operator-overloading methods in classes such as `__setattr__` and `__getattr__` and even `__repr__` have the potential to recursively loop if used incorrectly. Recursion is a powerful tool, but it tends to be best when both understood and expected!

Function Tools: Attributes, Annotations, Etc.

Let's move on to a category of tools that may seem less ethereal than recursion to some earthlings. As we've seen in this part of the book, functions in Python are much more than code-generation specifications for a compiler—they are full-blown *objects*, stored in pieces of memory all their own. As such, they can be freely passed around a program and called indirectly. They also support operations that have little to do with calls at all, including attributes and annotation. We've sampled some of these topics earlier, but this section provides expanded coverage.

The First-Class Object Model

Because Python functions are objects, you can write programs that process them generically. Function objects may be reassigned to other names, passed to other functions, embedded in data structures, returned from one function to another, and more, as if they were simple numbers or strings. Function objects happen to support a special operation—they can be *called* by listing arguments in parentheses—but they belong to the same general category as other objects.

As we've seen, this is usually called a *first-class object model*; it's ubiquitous in Python, and a necessary part of *functional programming*. We'll explore this programming mode more fully in this and the next chapter; because its motif is founded on the notion of applying functions, it treats functions as a kind of data.

We've explored some generic use cases for functions in earlier examples, but a quick review helps to underscore the model. For example, there's really nothing special about the name used in a `def` statement: it's just a variable assigned in the current scope, as if it had appeared on the left of an `=` sign. Because the function name is simply a reference to an object after a `def` runs, you can

reassign that object to other names freely and call it through any reference:

```
>>> def exclaim(message):           # Name exclaim assigned to function object
    print(message + '!')

>>> exclaim('Direct call')         # Call object through original name
Direct call!

>>> x = exclaim                    # Now x references the function too
>>> x('Indirect call')             # Call object through name x by adding ()
Indirect call!
```

And because arguments are passed by assigning objects, it's just as easy to *pass* functions to other functions as arguments. The callee may then call the passed-in function just by adding arguments in parentheses (see the earlier summer tester in [Example 19-4](#) for another example of this pattern):

```
>>> def generic(func, arg):
    func(arg)                       # Call the passed-in object by adding ()

>>> generic(exclaim, 'Argument call') # Pass the function to another function
Argument call!
```

You can even stuff function objects into data structures, as though they were integers or strings. The following, for example, *embeds* the function twice in a list of tuples, as a sort of actions table. Because Python compound types like these can contain any sort of object, there's no special case here, either ([Example 18-2](#) used similar code):

```
>>> schedule = [ (exclaim, 'Hack'), (exclaim, 'Code') ]
>>> for (func, arg) in schedule:
    func(arg)                       # Call functions embedded in containers

Hack!
Code!
```

This code simply steps through the `schedule` list, calling the `exclaim` function with one argument each time through. As we saw in [Chapter 17](#)'s examples, functions can also be created and *returned* for use elsewhere—the *closure* created in this mode also retains state from the enclosing scope:

```

>>> def implore(verb):                # Make a function but don't call it
    def exclaim(noun):
        print(f'{verb} {noun}!')
    return exclaim

>>> I = implore('Hack')               # Label in enclosing scope is retained
>>> I('Code')                         # Call the function that make returned
Hack Code!
>>> I('App')
Hack App!

```

Python's first-class object model and lack of type constraints make for a very flexible programming language.

Function Introspection

In fact, functions are more flexible than you might expect. Because they are objects, we can also process functions with normal object tools. For instance, by now we know that once we make a function we can call it as usual:

```

>>> def func(a):
    b = 'Hack'
    return b * a

>>> func(8)
'HackHackHackHackHackHackHackHack'

```

But the call expression is just one of a set of operations defined to work on function objects. For instance, we can also inspect their attributes generically:

```

>>> func.__name__
'func'
>>> dir(func)
['__annotations__', '__builtins__', '__call__', '__class__', '__closure__',
...more omitted: 38 total...
'__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__type_params__']

```

Introspection—internals access—tools like this allow us to explore implementation details. For example, functions have attached *code objects*, which provide details on aspects such as the functions' local variables and arguments:

```

>>> func.__code__
<code object func at 0x103ef3910, file "<stdin>", line 1>

>>> dir(func.__code__)
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
...more omitted: 48 total...
'co_posonlyargcount', 'co_qualname', 'co_stacksize', 'co_varnames', 'replace']

>>> func.__code__.co_varnames
('a', 'b')
>>> func.__code__.co_argcount
1

```

Tool writers can make use of such information to manage functions—in fact, we will too in [Chapter 39](#), to implement validation of function arguments with the decorators introduced ahead. Whether you code or use such tools, object introspection boosts function utility.

Function Attributes

Nor are function objects limited to the system-defined attributes of the prior section: it’s also possible to attach arbitrary *user-defined* attributes to them. This topic was introduced in [Chapter 17](#), but this section expands on it with additional context and examples. As usual in Python, function attributes are created by simple assignments:

```

>>> def func(): pass
>>> func
<function func at 0x1043771a0>
>>> func.count = 0
>>> func.count += 1
>>> func.count
1
>>> func.handles = 'Button-Press'
>>> func.handles
'Button-Press'
>>> dir(func)
...most double-underscore names omitted...
['__str__', '__subclasshook__', '__type_params__', 'count', 'handles']

```

Python’s own implementation-related data stored on functions follows naming conventions that prevent them from clashing with the more arbitrary attribute names you might assign yourself. Specifically, all function internals’ names have

leading and trailing double underscores (“__X__”):

```
>>> len(dir(func))
40
>>> [x for x in dir(func) if not x.startswith('__')]
['count', 'handles']
```

If you’re careful not to name attributes the same way as Python, you can safely use the function’s namespace as though it were your own namespace or scope. Naturally, all of this works the same for functions made with `lambda`:

```
>>> F = lambda: None
>>> len(dir(F))
38
>>> F.book = 'LP6E'
>>> F.book
'LP6E'
```

As covered in [Chapter 17](#), such attributes can be used to attach *state information* to function objects directly, instead of using other techniques such as globals, nonlocals, and classes. Unlike nonlocals, such attributes are accessible anywhere the function itself is, even from outside its code.

In a sense, this is also a way to emulate “static locals” in other languages—variables whose names are local to a function, but whose values are retained after a function exits. Attributes are related to objects instead of scopes (and must be referenced through the function name within its code), but the net effect is similar.

Moreover, as also explored in [Chapter 17](#), when attributes are attached to functions generated by other *factory* functions, they also support multiple copy, writeable, and *per-call* state retention, as an alternative to closures and class-instance attributes. This makes function attributes a broadly useful tool—like the topics of the next section.

Function Annotations and Decorations

For tools roles, functions also support attached *annotations*—arbitrary user-defined info about a function’s arguments and result that augment the function. Python provides syntax for coding annotations, but it doesn’t do anything with

them itself—annotations are completely optional, don't impact function behavior in any way, and when present are simply attached to the function object's `__annotations__` attribute for use by other tools.

While not of general interest to most application programmers, third-party tools and libraries might use annotations in the context of enhanced error checking, or API directives. Type hinting, discussed in [Chapter 6](#), is also based on annotations, but is optional and unused, and doesn't preclude other roles for annotations today (more on this ahead).

We studied the formal rules for arguments in function definitions in the preceding chapter. Annotations don't modify these rules but simply extend their syntax to allow extra expressions to be associated with named arguments and function results. Consider the following nonannotated function, coded with three arguments and a returned result:

```
>>> def func(a, b, c):  
    return a + b + c  
  
>>> func(1, 2, 3)  
6
```

Syntactically, function annotations are coded in `def` header lines (only), as arbitrary expressions associated with arguments and return values. For arguments, they appear after a colon immediately following the argument's name; for return values, they are written after a `->` following the arguments list's closing parenthesis. The following code, for example, annotates all three of the prior function's arguments, as well as its return value:

```
>>> def func(a: 'hack', b: (1, 10), c: float) -> int:  
    return a + b + c  
  
>>> func(1, 2, 3)  
6
```

Calls to an annotated function work as usual, but when annotations are present Python collects them in a *dictionary* and attaches it to the function object itself as its `__annotations__`. In this, argument names become keys; the return value annotation is stored under key `return` if coded (which suffices because this

reserved word can't be used as an argument name); and the values of annotation keys are assigned to the results of the annotation expressions:

```
>>> func.__annotations__
{'a': 'hack', 'b': (1, 10), 'c': <class 'float'>, 'return': <class 'int'>}
```

Because they are just Python objects attached to a Python object, annotations are straightforward to process. The following annotates just two of three arguments and steps through the attached annotations generically:

```
>>> def func(a: 'python', b, c: 3.12):
    return a + b + c

>>> func(1, 2, 3)
6
>>> func.__annotations__
{'a': 'python', 'c': 3.12}

>>> for arg in func.__annotations__:
    print(arg, '=>', func.__annotations__[arg])

a => python
c => 3.12
```

There are two fine points to note here. First, you can still use *defaults* for arguments if you code annotations—the annotation (and its `:` character) appears *before* the default (and its `=` character). In the following, for example, `a: 'hack' = 4` means that argument `a` defaults to 4 and is annotated with the string `'hack'`:

```
>>> def func(a: 'hack' = 4, b: (1, 10) = 5, c: float = 6) -> int:
    return a + b + c

>>> func(1, 2, 3)                # No defaults used
6
>>> func()                       # a=4 + b=5 + c=6 (all defaults)
15
>>> func(1, c=10)                # 1 + b=5 + 10 (keywords work normally)
16
>>> func.__annotations__
{'a': 'hack', 'b': (1, 10), 'c': <class 'float'>, 'return': <class 'int'>}
```

Second, note that the *blank spaces* in the prior example are all optional—you can use spaces between components in function headers or not, but omitting them

might degrade your code’s readability to some observers (and probably improve it to others):

```
>>> def func(a:'hack'=4, b:(1,10)=5, c:float=6)->int:
    return a + b + c

>>> func(1, 2)                # 1 + 2 + c=6
9
>>> func.__annotations__
{'a': 'hack', 'b': (1, 10), 'c': <class 'float'>, 'return': <class 'int'>}
```

While annotations are optional and uncommon, they may be used to specify constraints for types or values, and larger APIs might use this feature as a way to register function interface information. In fact, you’ll see a potential application in [Chapter 39](#), when we code annotations as an alternative to *function decorator arguments*—a more general concept in which augmentation info is coded *outside* the function header and so is not limited to a single role.

Function decorators alternative: Preview

Because we’re going to devote an entire chapter to decorators, we’ll omit most of their story here. But as a very brief preview, decorators are simply functions that augment other functions. They are applied to another function’s `def` with an `@` prefix that rebinds the other function’s name to the result of passing it to the decorator. This syntax:

```
@decorator
def func(args): ...           # Decorated function def
```

is automatically morphed into the following equivalent, where `decorator` is a one-argument callable object (or an expression that returns one), which returns a callable object having arguments compatible with `func` (or `func` itself):

```
def func(args): ...
func = decorator(func)       # Rebind func to decorator's result
```

Decorators can use this hook to wrap another function in an extra layer of code for nearly arbitrary purposes, as in the following code that adds a message when a decorated function is called:

```

def echo(F):
    def proxy(*args):
        print('calling', F.__name__)    # Add actions here
        return F(*args)                # Run decorated function
    return proxy

@echo
def func(x, y):
    print('I am running...', x, y)      # func is run by the proxy closure

>>> func(1, 2)                        # Runs a proxy, which runs func
calling func
I am running... 1 2

```

As you'll learn later, decorators can also take *arguments* (e.g., `@echo(args)`) whose utility can overlap with annotations—argument info can be sent to the decorator instead of being embedded in headers as annotations. Because this is an advanced tool that can also be applied to classes, we'll pause this thread until Chapters 32 and 39.

Compared to annotations, though, decorators don't complicate function headers themselves with extra syntax, and more naturally support multiple roles. Annotations may make functions difficult to read, and generally can have just one role—one that's hijacked by the optional *type hinting* of Chapter 6 in programs that choose to employ it.

In fact, type hinting advocates have tried to deprecate all other roles for annotations. While these divisive (and perhaps even rude) efforts have thankfully failed to date, decorators face no such challenge, and may be a safer bet going forward. Today, though, both annotations and decorations are tools whose roles are limited only by your imagination.

Finally, note that annotations and decorations work in `def` statements—but not in `lambda` expressions, whose syntax by design limits the functions they can define. Coincidentally, this brings us to this potpourri's next topic.

Anonymous Functions: `lambda`

We first met the `lambda` expression back in Chapter 16 and have used it in a handful of examples since then. This section reviews the basics and takes a

deeper second look, to both reinforce and expand on this topic.

As we've seen, besides the `def` statement, Python provides an expression that creates function objects. Because of its similarity to a tool in other languages, it's called `lambda`. Like `def`, this expression creates a function to be called later, but it returns the function instead of assigning it to a name. This is why `lambdas` are sometimes known as *anonymous* functions. In practice, they are used to *inline* function definitions, or *defer* execution of code.

NOTE

What's in a name?: The `lambda` tends to intimidate people, largely due to the name “lambda” itself—a name that comes from the Lisp language, which got it from lambda calculus, which is a form of symbolic logic. Obscure mathematical heritage aside, though, `lambda` in Python is really just a word that introduces an expression syntactically, and its expression is simply an alternative way to code a function—albeit without statements, decorators, or annotations.

lambda Basics

As a refresher, the `lambda`'s general form is the keyword `lambda`, followed by zero or more arguments (just like the arguments you enclose in parentheses in a `def` header, sans annotations), followed by an expression after a colon:

```
lambda argument1, argument2,... argumentN : expression-using-arguments
```

Parentheses are not allowed around `lambda` arguments and are generally optional around the entire `lambda` itself, though they're required in some contexts and may boost clarity in others. Functions returned by `lambda` work the same as those assigned by `def`, but `lambda` differs in ways that make it useful in specialized roles:

- **lambda is an expression, not a statement.** Because of this, a `lambda` can appear in places a `def` is not allowed by Python's syntax—inside a list literal or a function call's arguments, for example. With `def`, functions can be referenced by name in such locations, but must be created elsewhere. As an expression, `lambda` returns a value (a new

function) that can be assigned a name, or used where the `lambda` appears.

- **lambda's body is a single expression, not a block of statements.** The `lambda`'s body is similar to what you'd put in a `def` body's `return` statement; you simply type the result as a naked expression, instead of explicitly returning it. Because it is limited to an expression, a `lambda` is less general than a `def`—you can only squeeze so much logic into a `lambda` body without full-blown statements. This is by design, to limit program nesting: `lambda` is designed for coding simple functions, and `def` handles larger tasks.

Apart from those distinctions, `defs` and `lambdas` do the same sort of work. For instance, by this point we should be pros at making a function with a `def` statement:

```
>>> def func(x, y, z): return x + y + z
>>> func(2, 3, 4)
9
```

But we can achieve the same effect with a `lambda` expression by explicitly assigning its result to a name through which you can later call the otherwise-anonymous function:

```
>>> func = lambda x, y, z: x + y + z
>>> func(2, 3, 4)
9
```

Here, `func` is manually assigned the function object the `lambda` expression creates; this is how `def` works, too, but its assignment is automatic. Defaults and other *argument-matching* syntax work in `lambda` too, just like in a `def`:

```
>>> x = (lambda a='hack', b='python', c='code': a + b + c)
>>> x('write')
'writepythoncode'
```

The code in a `lambda` body also follows the same *scope* lookup rules as code inside a `def`. `lambda` expressions introduce a new local scope much like a nested

`def`, which automatically sees names in enclosing functions, the module, and the built-in scope—via the LEGB rule we studied in [Chapter 17](#):

```
>>> def editions(title):           # title in enclosing scope
    return (lambda e: title + ', ' + e)  # Return a function object

>>> labeler = editions('Learning Python')  # Make+save nested function
>>> labeler('6E')                        # '6E' passed to e in lambda
'Learning Python, 6E'
```

With those basic `lambda` “hows” in hand, the natural next question is “why,” taken up in the next section.

Why Use `lambda`?

Generally speaking, `lambda` is a sort of function *shorthand* that allows you to embed a function’s definition within the code that uses it. It is entirely optional—you can always use `def` instead, and *should* if your function requires the power of full statements that the `lambda`’s expression cannot provide. But `lambda` may be easier to use in scenarios where you just need to embed a small bit of executable code inline at the place where it is to be later used.

For instance, you’ll see later that callback handlers are frequently coded as inline `lambda` expressions embedded directly in a registration call’s arguments list, instead of being defined with a `def` elsewhere in a file and referenced by name (see the sidebar [“Why You Will Care: `lambda` Callbacks”](#) for an example).

`lambda` is also commonly used to code *jump tables*, which are lists or dictionaries of actions to be performed on demand. For example:

```
L = [lambda x: x * 2,           # Inline function definition
     lambda x: x ** 2,
     lambda x: x // 2]          # A list of three callable functions

for f in L:
    print(f(5))                 # Prints 10, 25, and 2

print(L[1](5))                 # Prints 25
```

The `lambda` expression works well when you need to stuff small pieces of

executable code into places where statements like `def` are not allowed. The preceding code snippet, for example, builds up a list of three functions by embedding `lambda` expressions inside a list literal; a `def` won't work inside a literal like this because it is a statement, not an expression. The equivalent `def` coding would require temporary function names (which might clash with others) and function definitions outside the context of intended use (which might be hundreds of lines away):

```
def f1(x): return x * 2
def f2(x): return x ** 2          # Define named functions
def f3(x): return x // 4         # Separate from their place of use

L = [f1, f2, f3]                # Reference by name

for f in L:
    print(f(2))                  # Also prints 10, 25, and 2
```

Multiway branches: The finale

In fact, you can do the same sort of thing with dictionaries and other data structures in Python to build up more general sorts of action tables. Here's another example to illustrate at the interactive prompt:

```
>>> key = 'loop'
>>> {'hack': lambda s: s.upper(),
    'code': lambda s: s.lower(),
    'loop': lambda s: f'{s * 4}!'}[key]('Py')
'PyPyPyPy!'
```

Here, when Python makes the temporary dictionary, each of the nested `lambdas` generates and leaves behind a function to be called later. Indexing by key fetches one of those functions, and parentheses force the fetched function to be called. When coded this way, a dictionary becomes a more general multiway branching tool than this book could fully reveal in [Chapter 12](#)'s coverage of `if` statements.

To make this work without `lambda`, you'd need to instead code three `def` statements somewhere else in your file, outside the dictionary in which the functions are to be used, and reference the functions by name:

```
>>> def f1(s): return s.upper()
```

```
>>> def f2(s): return s.lower()
>>> def f3(s): return f'{s * 4}!'

>>> key = 'loop'
>>> {'hack': f1, 'code': f2, 'loop': f3}[key]('Py')
'PyPyPyPy!'
```

This works, too, but your `defs` may be arbitrarily far away in your file, even if they are just little bits of code. The *code proximity* that `lambda` provides is especially useful for functions that will only be used in a single context—if the three functions here are not useful anywhere else, it makes sense to embed their definitions within the dictionary as `lambdas`. Moreover, the `def` form requires you to make up names for these little functions that, again, may clash with other names in this file (perhaps unlikely, but always possible).

You can use the `match` statement today with similar results, but it may require even more code than the `def` scheme, especially since you’d have to nest it within a `def` to support an argument like `s`; see [Chapter 12](#) for more info.

`lambda` is also handy in function-call *argument lists* as a way to inline temporary function definitions not used anywhere else in your program; you’ll see examples of such other uses later in this chapter, when we study `map`.

NOTE

The siren song of eval: In principle, you could skip the dispatch table in the preceding code if the function name is the same as its string lookup key—an `eval(key)(arg)` would kick off the call. While true in this case and sometimes useful, as we saw earlier (e.g., [Chapter 10](#)), `eval` is relatively slow (it must compile and run code) and insecure (you must trust the string’s source). More fundamentally, jump tables are generally subsumed by *polymorphic* method dispatch in Python: calling a method does the “right thing” based on the type of object that’s the subject of the call, no switching logic required. To see why, stay tuned for [Part VI](#).

How (Not) to Obfuscate Your Python Code

The fact that the body of a `lambda` has to be a single expression (not a series of statements) would seem to place severe limits on how much logic you can pack into a `lambda`. If you know what you’re doing, though, you can code most statements in Python as expression-based equivalents.

For example, if you want to *print* from the body of a `lambda` function, simply use `print` or `sys.stdout.write` (recall from [Chapter 11](#) that this is what `print` really does). And to execute *multiple* actions, code a sequence like a tuple, to evaluate nested multiple expressions from left to right (tuples require parentheses in this context):

```
>>> series = lambda a, b: (print(a.upper()), print(b.lower()))    # > 1 actions
>>> ignore = series('Hack', 'Code')
Hack
code
```

Similarly, to nest logic in a `lambda`, you can use the `if/else` ternary expression introduced in [Chapter 12](#), or the equivalent but trickier and/or combination also described there. As you learned earlier, the following statement:

```
if a:
    b
else:
    c
```

can be emulated by either of these equivalent expressions (the second is only roughly the same, but close enough):

```
b if a else c
((a and b) or c)
```

Because expressions like these can be placed inside a `lambda`, they may be used to implement *selection* logic within a `lambda` function (the `lambda` is parenthesized here only for variety and subjective clarity):

```
>>> lower = (lambda x, y: x if x < y else y)    # Ifs in lambda: ternary
>>> lower('bb', 'aa')
'aa'
>>> lower('aa', 'bb')
'aa'
```

Furthermore, if you need to perform *loops* within a `lambda`, you can also embed things like list comprehension expressions and `map` calls—tools we met in earlier chapters and will revisit in this and the next chapter:


```
>>> showall = lambda x: [print(y) for y in x]           # Loops in lambda: list comp
>>> t = showall(['lp5e', 'lp6e'])
lp5e
lp6e
>>> showall = lambda x: list(map(print, x))             # Loops in lambda: maps
>>> t = showall(('py3.3', 'py3.12'))
py3.3
py3.12
>>> showall = lambda x: print(*x, sep='', end='')      # Another option: *unpacking
```

And while it's limited by the local scope of the function that `lambda` makes, *assignment* is in scope (pun intended) for `lambda` expressions that use the `:=` named-assignment expression:

```
>>> namer = lambda x: (res := x + 1) + res             # Assignment in lambda: :=
>>> namer(2)
6
>>> res                                                # But it doesn't persist
NameError: name 'res' is not defined
```

There is a limit to emulating statements with expressions: you can't assign nonlocals, for instance, though tools like the `setattr` built-in, the `__dict__` of namespaces, and methods that change mutable objects in place can sometimes stand in—and can quickly lead you deep into the heart of expression-convolution darkness.

But now that this book has shown you these tricks, it must also humbly implore you to use them only as a last resort. Without due care, they can yield unreadable (a.k.a. *obfuscated*) Python code, despite the language's best intents. In general, simple is better than complex, explicit is better than implicit, and full statements are better than arcane expressions. That's why `lambda` is limited to expressions. If you have larger logic to code, use `def`; `lambda` is for small pieces of inline code. On the other hand, you may find these techniques useful—when taken in moderation.

Scopes: lambdas Can Be Nested Too

One last `lambda` note: as we saw in [Chapter 17](#), `lambda` is also the main beneficiary of enclosing-function scope lookup—the *E* in the LEGB scope rule. As a review, in the following the `lambda` appears inside a `def`, its typical coding,

and so can access the value that name `x` had in the enclosing function's scope during its call:

```
>>> def action(x):
    return (lambda y: x + y)           # Make function, remember x

>>> act = action(99)
>>> act(2)                             # Call what action returned
101
```

What wasn't illustrated in the prior discussion of nested function scopes is that a `lambda` also has access to the names in any enclosing `lambda`. This case is somewhat obscure, but imagine if we recoded the prior `def` with a `lambda`:

```
>>> action = (lambda x: (lambda y: x + y))   # lambda scopes nest too
>>> act = action(99)
>>> act(3)
102
>>> ((lambda x: (lambda y: x + y))(99))(4)   # Even if you don't save them
103
```

Here, the nested `lambda` structure makes a function that makes a function when called. In both cases, the nested `lambda`'s code has access to the variable `x` in the enclosing `lambda`. This works, but it seems fairly convoluted code; in the interest of readability, nested `lambdas` may be best avoided. The good news, perhaps, is that `def` cannot be nested in `lambda`: because `lambda`'s body is an expression, statements like `def` won't work—thankfully!

Functional Programming Tools

Last up in this chapter's gumbo is a reprisal of a category of tools we met in earlier in this book, with a few new tips to round out the topic. By most definitions, today's Python blends support for multiple programming paradigms: *procedural* (with its basic statements), *object-oriented* (with its classes), and *functional*.

The criteria for the latter of these categories are somewhat loose, but by most measures Python's functional programming toolbox includes its *first-class object*

model, nested scope *closures*, and anonymous function *lambdas* that we met earlier; its *generators* and *comprehensions*, which we'll be expanding on in the next chapter; and perhaps its function and class *decorators* previewed here but fleshed out in this book's final part.

This toolbox also includes built-in functions that apply other functions to iterables automatically—including functions that call other functions on an iterable's items (`map`); select items based on a test function (`filter`); and apply functions to pairs of items and running results (`reduce`). Let's wrap up this chapter with a quick survey of this trio.

Mapping Functions over Iterables: `map`

One of the more common things programs do with lists and other sequences is apply an operation to each item and collect the results—selecting columns in database tables, incrementing pay fields of employees in a company, parsing email attachments, and so on. Python has multiple tools that make such collection-wide operations easy to code. For instance, we've seen that updating all the counters in a list can be done easily with a `for` loop:

```
>>> counters = [1, 2, 3, 4]

>>> updated = []
>>> for x in counters:
    updated.append(x + 10)                # Add 10 to each item

>>> updated, counters
([11, 12, 13, 14], [1, 2, 3, 4])
```

But because this is such a common operation, Python also provides built-ins that do most of the work for you. Among them, the `map` function applies a passed-in function to each item in an iterable object and returns a list containing all the function-call results. For example, assuming `counters` is intact:

```
>>> def inc(x): return x + 10                # Function to be run

>>> list(map(inc, counters))                # Collect call results
[11, 12, 13, 14]
```

We met `map` briefly in Chapters 13 and 14, as a way to apply a built-in function to items in an iterable. Here, we make more general use of it by passing in a *user-defined* function to be applied to each item in the list—`map` calls our `inc` on each list item and collects all the return values into a new list. Remember that `map` is a nonsequence iterable, so a `list` call is used to force it to produce all its results for display here (per Chapter 14 coverage).

Because `map` expects a function to be passed in and applied, it also happens to be one of the places where `lambda` commonly appears:

```
>>> list(map((lambda x: x + 3), counters))    # Function expression
[4, 5, 6, 7]
```

Here, the function adds 3 to each item in the `counters` list; as this little function isn't needed elsewhere, it was written inline as a `lambda`. Because such uses of `map` are equivalent to `for` loops, with a little extra code you can always code a general mapping utility yourself:

```
>>> def mymap(func, iter):
    res = []
    for x in iter: res.append(func(x))
    return res
```

Assuming the function `inc` is still as it was when it was shown previously, we can map it across a sequence (or other iterable) with either the built-in or our equivalent:

```
>>> list(map(inc, [1, 2, 3]))                # Built-in is an iterable
[11, 12, 13]
>>> mymap(inc, [1, 2, 3])                    # Ours builds a list (see generators)
[11, 12, 13]
```

However, as `map` is a built-in, it's always available, always works the same way, and may have performance benefits (as we'll prove in Chapter 21, it's faster than a manually coded `for` loop in some usage modes). Moreover, `map` can be used in more advanced ways than shown here. For instance, given multiple sequence arguments, it sends items taken from sequences in parallel as distinct arguments to the function:

```
>>> pow(3, 4)                                # 3**4
81
>>> list(map(pow, [1, 2, 3], [2, 3, 4]))      # 1**2, 2**3, 3**4
[1, 8, 81]
```

With multiple sequences, `map` expects an N-argument function for N sequences. Here, the `pow` function takes two arguments on each call—one from each sequence passed to `map`. It’s not much extra work to simulate this multiple-sequence generality in code, too, but we’ll postpone doing so until later in the next chapter, after we’ve explored some additional iteration tools (hint: it would be better to *generate* items on demand, like the built-in).

The `map` call is also similar to the list comprehension expressions we studied in [Chapter 14](#) and will revisit in the next chapter from a functional perspective:

```
>>> list(map(inc, [1, 2, 3, 4]))
[11, 12, 13, 14]
>>> [inc(x) for x in [1, 2, 3, 4]]
[11, 12, 13, 14]
```

In some cases, `map` may be faster to run than a list comprehension, and it may also require less code. On the other hand, because `map` applies a *function* call to each item instead of an arbitrary *expression*, it is a somewhat less general tool, and often requires extra helper functions or `lambdas`. Moreover, wrapping a comprehension in parentheses instead of square brackets creates an object that *generates* values on request to save memory and increase responsiveness, much like `map`—a topic we’ll take up in the next chapter.

Selecting Items in Iterables: `filter`

The `map` function is a primary and relatively straightforward representative of Python’s functional programming toolset. Its close relatives, `filter` and `reduce`, select an iterable’s items based on a test function and apply functions to item pairs, respectively.

Because it also returns an iterable, `filter` (like `range`) requires a `list` call to display all its results in a REPL. For example, the following `filter` call picks out items in a sequence that are greater than zero:

```
>>> list(range(-5, 5))
[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4]

>>> list(filter((lambda x: x > 0), range(-5, 5)))
[1, 2, 3, 4]
```

We met `filter` briefly earlier in the sidebar “Why You Will Care: Booleans” and while exploring iterables in [Chapter 14](#). Items in the sequence or iterable for which the function returns a true result are added to the result list. Like `map`, this function is roughly equivalent to a `for` loop, but it is built-in, concise, and often fast:

```
>>> res = []
>>> for x in range(-5, 5):
    if x > 0:
        res.append(x)
# The statement equivalent of filter
# Simple but slower today, probably

>>> res
[1, 2, 3, 4]
```

Also like `map`, `filter` can be emulated by *list comprehension* syntax with often simpler results (especially when it can avoid creating a new function), and with a similar *generator expression* when coded with enclosing parentheses to delay production of results—though again, the generator story will have to remain a teaser for the next chapter:

```
>>> [x for x in range(-5, 5) if x > 0]
[1, 2, 3, 4]
```

Combining Items in Iterables: reduce

The functional `reduce` call—once a built-in but now a resident of the `functools` standard-library module—is more complex. It accepts an iterable to process, but it’s not an iterable itself: it returns a *single* result that aggregates an iterable’s items. To demo, here are two `reduce` calls that compute the sum and product of the items in a list:

```
>>> from functools import reduce
>>> reduce((lambda x, y: x + y), [1, 2, 3, 4])
10
```

```
>>> reduce((lambda x, y: x * y), [1, 2, 3, 4])
24
```

At each step, `reduce` passes the *current* sum or product, along with the *next* item from the list, to the passed-in `lambda` function. By default, the first item in the sequence initializes the starting value. To make that more concrete again, here's the `for` loop equivalent to the first of these calls, with the addition hardcoded inside the loop:

```
>>> L = [1, 2, 3, 4]
>>> res = L[0]
>>> for x in L[1:]:
    res += x

>>> res
10
```

In fact, coding your own reusable and customizable version of `reduce` is fairly straightforward too. The following function emulates most of the built-in's behavior and helps demystify its operation in general:

```
>>> def myreduce(function, sequence):
    tally = sequence[0]
    for next in sequence[1:]:
        tally = function(tally, next)
    return tally

>>> myreduce((lambda x, y: x + y), [1, 2, 3, 4])
10
>>> myreduce((lambda x, y: x * y), [1, 2, 3, 4])
24
```

The built-in `reduce` also allows an optional third argument, effectively placed before the items in the sequence to serve as an initial value and a default result when the sequence is empty, but we'll leave this extension as a suggested exercise (again, emulating built-in tools is instructive, but superfluous).

If this coding technique has sparked your interest, you might also be interested in the standard-library `operator` module, which provides functions that correspond to built-in expressions and so comes in handy for some uses of functional tools (consult `help` in a REPL or Python's library manual for more details on this

module):

```
>>> import operator, functools
>>> functools.reduce(operator.add, [2, 4, 6])          # Function-based +
12
>>> functools.reduce((lambda x, y: x + y), [2, 4, 6])
12
```

Together, `map`, `filter`, and `reduce` support powerful functional programming techniques. As mentioned, many observers would also extend the functional programming toolset in Python to include the nested function closures and anonymous function `lambdas` we've explored, as well as the *generators* and *comprehensions* we've met in piecemeal fashion along the way. To fully grok the latter two, though, we must move on to the next chapter.

Chapter Summary

This chapter's collage took us on a tour of function-related topics: function design guidelines; recursive functions; function attributes, annotations, and decorators; `lambda` expressions; and the `map`, `filter`, and `reduce` built-ins. Some of these are advanced, but most are common in Python programming. The next chapter continues the advanced-topics motif with a reprisal of comprehensions and an unmasking of generators—tools that are just as related to functional programming as to looping statements. Before you move on, though, make sure you've mastered the concepts covered here by working through this chapter's quiz.

Test Your Knowledge: Quiz

1. How are `lambda` expressions and `def` statements related?
2. What's the point of using `lambda`?
3. Compare and contrast `map`, `filter`, and `reduce`.
4. What are function annotations, and how are they used?
5. What are recursive functions, and how are they used?
6. What are some general design guidelines for coding functions?
7. Name three or more ways that functions can communicate results to a caller.

Test Your Knowledge: Answers

1. Both `lambda` and `def` create function objects to be called later. Because `lambda` is an expression, though, it returns a function object instead of assigning it to a name, and it can be used to nest a function definition in places where a `def` will not work syntactically. A `lambda` allows for

only a single implicit return value expression, though; because it does not support a block of statements, it is not ideal for larger functions.

2. `lambda` allows us to “inline” small units of executable code, defer its execution, and provide it with state in the form of default arguments and enclosing scope variables. Using a `lambda` is never required; you can always code a `def` instead and reference the function by name. `lambda` comes in handy, though, to embed small pieces of deferred code that are unlikely to be used elsewhere in a program. It commonly appears in callback-based programs such as GUIs, and has a natural affinity with functional tools like `map` and `filter` that expect a processing function.
3. These three built-in functions all apply another function to items in a sequence (or other iterable) object and collect results. `map` passes each item to the function and collects call results, `filter` collects items for which the function returns a true value, and `reduce` computes a single value by applying the function to an accumulator and successive items. Unlike the other two, `reduce` is available in the `functools` module, not the built-in scope (in modern Python history, at least).
4. Function annotations are syntactic embellishments of a function’s arguments and result, which are collected into a dictionary assigned to the function’s `__annotations__` attribute. Python places no semantic meaning on these annotations, but simply packages them for potential use by other tools.
5. Recursive functions call themselves either directly or indirectly in order to loop (i.e., repeat). They may be used to traverse arbitrarily shaped structures, but they can also be used for iteration in general (though the latter role is often more simply and efficiently coded with looping statements). Recursion can often be simulated or replaced by code that uses explicit stacks or queues to have more control over traversals.
6. Functions should generally be small and as self-contained as possible, have a single unified purpose, and communicate with other components through input arguments and return values. They may use mutable arguments to communicate results too if changes are expected, and

some types of programs imply other communication mechanisms.

7. Functions can send back results with `return` statements, by changing passed-in mutable arguments, and by setting global variables. Globals are generally frowned upon (except for very special cases, like multithreaded programs) because they can make code more difficult to understand and use. `return` statements are usually best, but changing mutables is fine (and even useful), if expected. Functions may also communicate results with system devices such as files and sockets, but these are beyond our scope here.

WHY YOU WILL CARE: LAMBDA CALLBACKS

Another common role for `lambda` is to define inline *callback* functions for Python's `tkinter` GUI API. For example, the following creates a button that prints a message on the console when pressed, assuming `tkinter` is present in your Python (it is by default on most PCs and at least one Android app):

```
from tkinter import Button, mainloop
x = Button(
    text='Press me',
    command=lambda: print('Tapped!'))
x.pack()
mainloop() # This may be optional in some REPLs
```

Here, we register the callback handler by passing a function generated with a `lambda` to the `command` keyword argument. The advantage of `lambda` over `def` in this is that the code that handles a button press is right here, embedded in the button-creation call.

In effect, the `lambda` *defers* execution of the handler until the event occurs: the `print` call happens on button presses, not when the button is created, and effectively “knows” the string it should write when the event occurs. Real GUIs rarely print to consoles, of course, but this demos the coding pattern.

Because the nested function scope rules apply to `lambda`, they automatically see names in the functions in which they are coded and hence don't require passed-in defaults in most cases (except for loop variables, per [Chapter 17](#)).

This is especially useful for accessing the special `self` instance argument that is a local variable in enclosing class method functions (which we'll study in [Part VI](#), so take this as preview only):

```
class MyGui:
    def makewidgets(self):
        Button(command=lambda: self.onPress('Tapped!'))
    def onPress(self, message):
        ...use message...
```

As you'll see later, both class objects with `__call__` and *bound methods* often serve in callback roles too—watch for coverage of these in [Chapters 30](#) and [31](#).

And all of this applies to coding event callbacks in other commonly used and portable GUI toolkits for Python—including *Kivy*, *Toga* (in *BeeWare*), *PyQT*, and *wxPython*. *tkinter* gets more press here only because it's shipped with Python's standard library. As for all tools, you should vet GUIs for yourself.